



INFOSYS LATERAL INTERVIEW • JL3A / JL4

Infosys Interview Prep

Company-నిర్దిష్ట తయారీ – ఏ rounds, ఏ ప్రశ్నలు, ఏం చెప్పాలి.
నిజమైన candidate అనుభవాల ఆధారంగా.

ఇది topic doc కాదు – ఇది మిగతా library మీద ఒక **filter**. ఏ topic కి ఎంత
సంభావ్యత ఉందో చెప్పి, లోతుగా చదవాల్సిన చోటికి దారి చూపుతుంది.

విషయ సూచిక (Table of Contents)

Part 0 – Ground Reality (ముందు ఇది చదువు)

1. Infosys lateral hiring process – నిజంగా ఏం జరుగుతుంది (rounds, timeline, platform)
2. నీ Job Level (JL) ఏమిటి, Salary ఎంత ఆశించాలి – 3 years experience కి
3. "Full Stack" trap – Infosys లో Full Stack అంటే చాలాసార్లు **Java/Spring**, MERN కాదు
4. Infosys ఎలా evaluate చేస్తుంది – rating scale, ఎందుకు reject చేస్తారు

Part 1 – Round 0: Recruiter Screening & Resume

5. Screening call లో ఏం అడుగుతారు – word-by-word script (CTC, notice period సహా)
6. Resume – Infosys కి ఎలా tune చెయ్యాలి (ATS + interview syllabus)

Part 2 – Round 1: Technical Interview (అతి ముఖ్యం)

7. Technical round structure – 1 hour లో ఏం జరుగుతుంది + **topic probability table**
8. "Tell me about yourself" – 90-second pitch (template + complete example)
9. Project deep-dive – ఇదే round ని decide చేస్తుంది (12 follow-up questions)
10. JavaScript question bank – 32 questions + answers (Infosys depth)
11. JavaScript output-based puzzles – 12 snippets, **అన్నీ node లో verify చేయబడ్డాయి**
12. JavaScript polyfills – 11 implementations, **అన్నీ assertions తో test చేయబడ్డాయి**
13. React question bank – 34 questions, Level 1 → Level 3 (Infosys-specific list)
14. React scenario questions – నిజంగా అడిగినవి + model answers
15. Redux / State management – Infosys ఇది తప్పకుండా అడుగుతుంది
16. HTML, CSS & Browser – underrated కానీ అడుగుతారు
17. Git – Infosys technical round లో regular question
18. Machine coding / Live coding (CodeSandbox) – నిజంగా ఇచ్చిన tasks
19. Node.js / Express / MongoDB – Full Stack అయితే
20. DBMS + SQL – Infosys favourite queries
21. API, REST, HTTP & Security basics
22. Testing (Jest, RTL) – 3 YOE కి expected level
23. DSA – Infosys ఏ level అడుగుతుంది (spoiler: LeetCode Easy)

Part 3 – Round 2: Managerial Round

24. Managerial round అంటే ఏమిటి – technical round కంటే ఎందుకు risky
25. STAR method – నీ 5 stories ముందే ready చెయ్యి (2 complete model answers)
26. Scenario question bank – 24 questions + model answers (client, deadline, conflict, fit)
27. ఈ round లో reject అయ్యే 6 తప్పులు

Part 4 – Round 3: HR Round & Negotiation

28. HR round questions + answers
29. Salary negotiation – script, numbers, 4 levers
30. Documents checklist (Form 16, payslips, BGV) + offer letter timeline

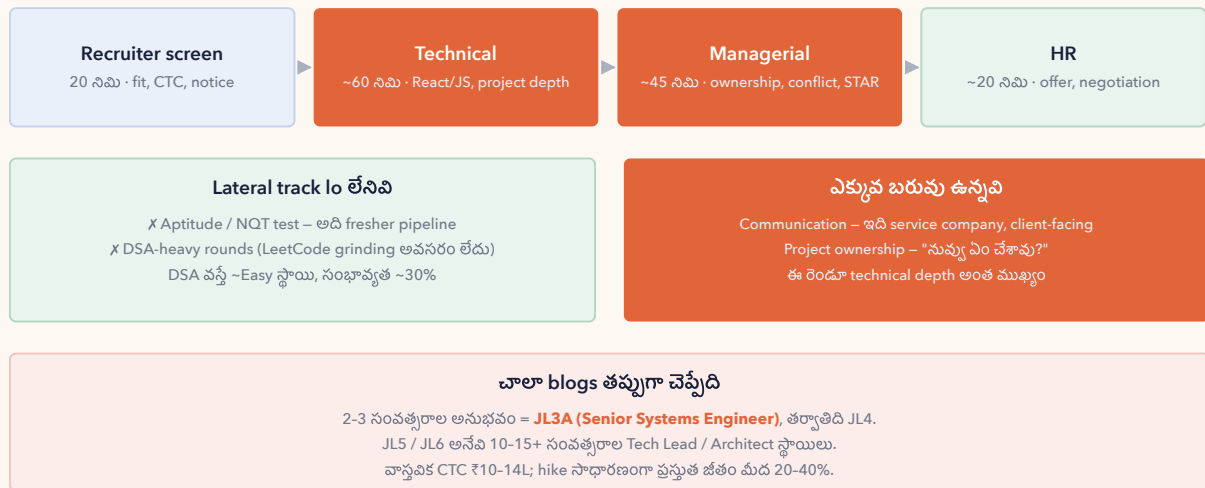
Part 5 – Execution

31. 4-week study plan (day-by-day) + time allocation
32. 1-week crash plan (interview దగ్గరపడితే)
33. Interview ముందు రోజు + ఆ రోజు checklist
34. Mock interview scripts – 3 full mocks + self-evaluation
35. Rapid-fire cheat sheet (చివరి 2 గంటల revision)

Part 0 – Ground Reality

INFOSYS LATERAL · ROUNDS మరియు ఏది ఎంత ముఖ్యం

LATERAL (experienced) TRACK – నాలుగు rounds



అతి ముఖ్యమైన వ్యూహం: Infosys "Full Stack" requisitions తరచుగా **Java/Spring**. MERN నేపథ్యం ఉంటే – "Frontend Developer (React)" / "UI Developer" requisitions కి apply చేయడం, మరియు recruiter తో backend stack ని ముందే నిర్ధారించుకోవడం.

1. Infosys lateral hiring process – నిజంగా ఏం జరుగుతుంది

వివరణ

Infosys లో **fresher hiring** (InfyTQ, HackWithInfy, NQT) కి, **lateral hiring** (2+ years experience) కి process పూర్తిగా వేరు. నువ్వు 2-3 years experience తో apply చేస్తున్నావు కాబట్టి **నీకు aptitude test, verbal ability test, pseudo-code test ఏమీ ఉండవు**. Fresher content (IRT, NQT papers) చదవడం **waste of time** – ఆ trap లో పడకు.

నే actual pipeline ఇది:

Stage	ఏమిటి	Duration	ఎవరు తీసుకుంటారు
0. Application	Career portal / Naukri / referral	–	ATS + recruiter
1. Recruiter screening	Phone call – experience, skills, notice period, CTC verify	10-15 min	HR recruiter
2. Technical Round 1	JavaScript, React, Redux, Git + live coding	~1 hour	Senior dev / Tech Lead
3. Technical Round 2 (కొన్నిసార్లు మాత్రమే)	Deeper tech, లేదా backend split	45-60 min	Architect / PM
4. Managerial Round	Scenario, behavioural + కొంత technical	15-45 min	Delivery/Project Manager
5. HR Round	Salary, location, notice period	15-30 min	HR

గమనిక: కొంతమంది candidates కి **3 rounds** (Tech → Managerial → HR), కొంతమందికి **2 rounds** (Tech → HR) మాత్రమే వచ్చాయి. ఇది **business unit** మీద, **project urgency** మీద ఆధారపడి ఉంటుంది. Client project urgent గా ఉంటే rounds తగ్గుతాయి. కాబట్టి **3 rounds కి prepare అవ్వు, 2 వస్తే bonus.**

Real-life Scenario

Infosys hiring ని ఒక **restaurant chain** లా ఊహించుకో. Google/Amazon అంటే ఒక Michelin-star restaurant – వాళ్ళు ప్రతి chef ని "నువ్వు molecular gastronomy చెయ్యగలవా?" అని test చేస్తారు. Infosys అంటే ఒక **పెద్ద chain** – వాళ్ళకి 100 outlets ఉన్నాయి (clients), ప్రతి outlet కి chefs కావాలి. వాళ్ళు అడిగేది: "నీకు basics వచ్చా? నువ్వు kitchen లో ఇతరులతో కలిసి పని చెయ్యగలవా? Customer ముందు మర్యాదగా మాట్లాడగలవా? Recipe follow చెయ్యగలవా?" – molecular gastronomy కాదు.

అందుకే Infosys interview లో "**Explain closures**" అని అడుగుతారు కానీ "**Design a distributed rate limiter**" అని అడగరు.

Timeline (candidates report చేసినది)

- Application → slot booking email: **5-6 days**
- Application → offer: **average 19 days** (React Developer role), overall Infosys average ~14 days
- కొన్ని cases లో **4-8 weeks** (business unit slow అయితే)
- HR round → offer letter: **1-4 weeks** (BGV తర్వాత)

Gotchas (సాధారణ తప్పులు)

- 1. Fresher content చదవడం.** "Infosys previous year question paper", "Infosys aptitude questions" – ఇవన్నీ నీకు **సంబంధం లేదు**. నీది lateral track.

2. **Coding OA ఉంటుందా లేదా అని confusion.** Lateral కి సాధారణంగా ఉండదు, కానీ కొన్ని **Specialist/Digital roles** కి HackerRank screen ఉంటుంది. నీ recruiter ని direct గా అడుగు – ఇది guess చేసే విషయం కాదు.
3. **Rounds అన్నీ ఒకే రోజు అనుకోవడం.** Infosys rounds సాధారణంగా వేరు వేరు రోజుల్లో ఉంటాయి, slot booking ద్వారా.
4. **Managerial round ని తక్కువ అంచనా వెయ్యడం.** ఇక్కడే ఎక్కువమంది experienced candidates reject అవుతారు – technical round కాదు.

Key Points

- Lateral = **no aptitude test**. Technical + Managerial + HR.
- Rounds **2 నుండి 4** వరకు ఉండొచ్చు – business unit మీద ఆధారపడి.
- Interviews **Microsoft Teams** లో జరుగుతాయి (Infosys standard).
- Live coding ఉంటే **CodeSandbox** లో ఇస్తారు.
- Difficulty: candidates consistently "**easy to medium**" అని report చేశారు.

Interview దృష్టి

మొదటి recruiter call లోనే ఈ 3 questions అడుగు (ఇది professional గా కనిపిస్తుంది, panic కాదు):

1. "How many interview rounds are there for this position?"
2. "Is there a coding assessment before the technical round?"
3. "Could you share the exact tech stack for this requisition – is the backend Node or Java?"

ఈ 3వ question అత్యంత ముఖ్యం. తర్వాతి section చూడు.

2. నీ Job Level (JL) ఏమిటి, Salary ఎంత ఆశించాలి

INFOSYS JOB LEVELS • 2-3 YOЕ ఎక్కడ నిలుస్తుంది

JOB LEVELS – చాలా blogs ఇక్కడ తప్పు చెప్తాయి

JL3A	Senior Systems Engineer	2-4 YOЕ • ₹8-14L
JL4	Technology Analyst	4-7 YOЕ • ₹14-22L
JL5	Technology Lead	8-12 YOЕ
JL6	Senior Tech Lead / Architect	12+ YOЕ

2-3 YOЕ = JL3A – JL5 కాదు

"3 సంవత్సరాలకి JL5" అని చెప్పే blogs తప్పు. JL5 అనేది 8-12 ఏళ్ళ Tech Lead స్థాయి.

తప్పు level ఆశిస్తే – recruiter call lo నమ్మకం పోతుంది.

వాస్తవిక hike: ప్రస్తుత CTC మీద **20-40%**. అంతకంటే ఎక్కువ అరుదు.

వివరణ

Infosys లో అందరికీ ఒక **Job Level (JL)** ఉంటుంది. ఇది నీ designation, salary band, promotion path అన్నీ decide చేస్తుంది. HR round లో negotiation చెయ్యాలంటే **నీ level ఏమిటో నీకు తెలియాలి** – లేకపోతే

వాళ్ళు చెప్పిందే నిజం అవుతుంది.

Infosys **delivery/technology stream** ladder:

Job Level	Designation	సాధారణంగా ఎంత experience
JL3B	Systems Engineer	0-2 years (entry)
JL3A	Senior Systems Engineer	2-4 years ← నువ్వు ఇక్కడ
JL4	Technology Analyst	4-7 years ← నీ next target
JL5	Technology Lead	8-12 years
JL6A / JL6B	Senior Technology Architect / Project Manager	15+ years
JL7, JL8	Senior/Principal Architect, Delivery Head	18+ years

ముఖ్యం: చాలా blogs లో "3 years = JL6" అని తప్పుగా రాస్తారు. **అది తప్పు.** JL6 అనేది 15+ years architect/PM level. నీ realistic level **JL3A (Senior Systems Engineer)**, బాగా negotiate చేస్తే **JL4 (Technology Analyst)**.

Real-life Scenario

JL అనేది army లో **rank** లాంటిది. నువ్వు interview లో "నాకు 20 LPA కావాలి" అని అడిగితే, HR మనసులో ఆలోచించేది "ఈ candidate JL3A. JL3A band max ఇంత. ఇది band దాటుతోంది."

Band దాటిన demand ని వాళ్ళు negotiate చెయ్యరు – silently reject చేస్తారు. అందుకే నీ band తెలుసుకుని అందులో top end అడగాలి, band దాటి కాదు.

ఒక candidate 40 LPA offer hold చేస్తున్నానని చెప్పాడు – Infosys direct గా "we would not be able to match that" అన్నారు. Band దాటితే conversation అక్కడితో ఆగిపోతుంది.

Salary – realistic expectations (3 YOE, React/Full Stack)

–	Range
JL3A typical CTC	₹6.5L - ₹12L
Strong React + good negotiation	₹10L - ₹14L
JL4 (అరుదు, 3 YOE కి)	₹12L - ₹18L

Hike logic: Infosys lateral hikes సాధారణంగా current CTC మీద **20-40%**. నీ current 8L అయితే realistic ask **11-12L**. 25L అడిగితే call అక్కడితో అయిపోతుంది.

నిజం: Infosys **premium payer** కాదు. వాళ్ళ strength = stability, brand name, client exposure, onsite opportunity. Money maximize చెయ్యడం goal అయితే product companies target చెయ్యి. Infosys ని ఒక **stable platform + brand** గా చూడు.

Gotchas (సాధారణ తప్పులు)

1. **"Current CTC has no bearing"** అని కొందరు చెప్తారు – **అది పూర్తిగా నిజం కాదు.** Infosys HR దాదాపు ఎప్పుడూ current CTC ఆధారంగా % hike calculate చేస్తుంది. Payslips అడుగుతారు, verify చేస్తారు. **CTC గురించి అబద్ధం చెప్పకు** – Form 16 లో పట్టుబడితే offer revoke అవుతుంది.
2. **CTC ≠ in-hand.** CTC లో PF (employer + employee), gratuity, insurance, variable pay అన్నీ ఉంటాయి. ₹12L CTC అంటే in-hand సుమారు **₹80-85K/month**. Offer వచ్చాక **breakup అడుగు**.
3. **Variable pay ని fixed అనుకోవడం.** Infosys లో 5-15% variable ఉంటుంది, అది performance/company results మీద ఆధారపడుతుంది. **Fixed component మీద negotiate చెయ్యి.**
4. **Designation negotiate చెయ్యకపోవడం.** Salary band fix అయితే, **"Can this be considered for Technology Analyst instead?"** అని అడుగు. Level పెరిగితే future hikes అన్నీ పెరుగుతాయి.

Key Points

- 3 YOE = **JL3A, Senior Systems Engineer.** JL6 కాదు.
- Realistic CTC: **₹10-14L** (strong React profile).
- Hike: current మీద **20-40%**.
- **Payslips + Form 16 ready ఉంచు** – వాళ్ళు అడుగుతారు.

Interview దృష్టి

HR "What is your expected CTC?" అంటే – **ఒక number కాదు, ఒక range ఇవ్వు, justification తే:**

"Based on my 3 years of React and full-stack experience, and the market range for this role, I'm looking at 12 to 14 lakhs fixed. But I'm flexible for the right role and growth path."

Part 4 లో పూర్తి negotiation script ఉంది.

3. "Full Stack" trap – Infosys లో Full Stack అంటే ఏమిటి

"FULL STACK" TRAP • STACK ని ముందే నిర్ధారించుకోండి

"FULL STACK" – Infosys lo దీని అర్థం

మీరు అనుకునేది

MERN – React + Node + MongoDB
JavaScript అంతటా

తరచుగా నిజం

React + **Java / Spring Boot**
Backend పూర్తిగా వేరే ప్రపంచం

వ్యూహం

MERN నేపథ్యం అయితే – **"Frontend Developer (React)" / "UI Developer"** requisitions కి apply.
Full Stack req కి apply చేసేమొందు – recruiter ని **నేరుగా** అడగండి: "backend stack ఏమిటి?"
Java req కి React నైపుణ్యంతో వెళ్లే – technical round lo Spring questions వస్తాయి.

వివరణ

ఇది ఈ doc లో అత్యంత ముఖ్యమైన warning. చాలామంది MERN developers Infosys "Full Stack Developer" role కి apply చేసి, interview లో **Java, Spring Boot, Microservices** questions వచ్చి shock అవుతారు.

కారణం: Infosys ఒక enterprise service company. వాళ్ళ clients (banks, insurance, retail) ఎక్కువగా **Java backend** వాడతారు. కాబట్టి Infosys లో "Full Stack" అంటే చాలాసార్లు:

Java + Spring Boot + Microservices (backend) + React/Angular (frontend)

Node.js/MERN కాదు.

Real-life Scenario

ఒక candidate Infosys Full Stack interview కి వెళ్ళాడు. **Round 1 పూర్తిగా backend** – Java, Spring, Microservices. **Round 2 frontend** – JavaScript, React. మరో candidate కి core Java, Spring Boot fundamentals, Collections (ArrayList vs LinkedList, Java 8 తర్వాత HashMap internals), మరియు ఒక **LRU Cache coding problem** వచ్చాయి.

అతను React లో brilliant. కానీ Java రాకపోవడం వల్ల Round 1 లోనే out. **Prep తప్పు కాదు – role selection తప్పు.**

నీకు ఏం చెయ్యాలి – 3 options

Option A (recommended): "Frontend Developer - React" role కి apply చెయ్యి, "Full Stack" కి కాదు. నీ background MERN/React. Infosys కి **pure React/UI developer** roles చాలా ఉన్నాయి (UI Developer, Frontend Developer, React JS Developer). ఇక్కడ నీకు Java అవసరం లేదు. **ఇదే నీ highest-probability path.**

Option B: Full Stack అయితే – requisition లో stack confirm చేసుకో. Recruiter call లో word-for-word ఇలా అడుగు:

"Could you confirm the backend stack for this requisition – is it Node.js or Java/Spring Boot?"

Java అని చెప్తే: **ఆ role వదిలేయ్**, లేదా నిజాయితీగా "My backend experience is Node.js and Express; I'd be a strong fit for a Node-based or frontend-focused requirement" అని చెప్పు. Recruiters సాధారణంగా నిన్ను వేరే matching req కి move చేస్తారు.

Option C: Java నేర్చుకో – కానీ ఇది **2-3 నెలల పని**, 2-3 వారాల పని కాదు. Interview దగ్గరపడితే ఇది option కాదు.

Gotchas (సాధారణ తప్పులు)

1. JD లో **"Full Stack" చూసి Node అనుకోవడం.** JD చదువు – "J2EE", "Spring", "Microservices", "Hibernate" అనే words ఉంటే అది **Java role**.

2. "నేను త్వరగా నేర్చుకుంటాను" అని చెప్పడం. Service company interview లో ఇది pass కాదు. వాళ్ళకి **billable from day one** కావాలి – client కి bill చెయ్యాలి కాబట్టి.
3. నా repo లో **Java doc లేదు**. ఈ repo లో `00PS_TeLugu.md` OOP concepts (Java-based examples) cover చేస్తుంది కానీ **Spring Boot / Microservices doc లేదు**. Java route వెళ్తే అది వేరే prep.

Key Points

- Infosys "Full Stack" ≈ **Java + React** (చాలాసార్లు), MERN కాదు.
- **Recruiter ని stack confirm చెయ్యి** – ఇది ఒక్క question, చాలా time save చేస్తుంది.
- నీకు best fit = "**Frontend Developer (React)**" లేదా "**UI Developer**" requisitions.
- MERN backend role దొరికితే – ఈ doc లోని Node/Express/MongoDB section (Part 2, §19) చాలు.

Interview దృష్టి

Interviewer "Do you know Java?" అని అడిగితే – **అబద్ధం చెప్పకు**, కానీ transferable strength చూపించు:

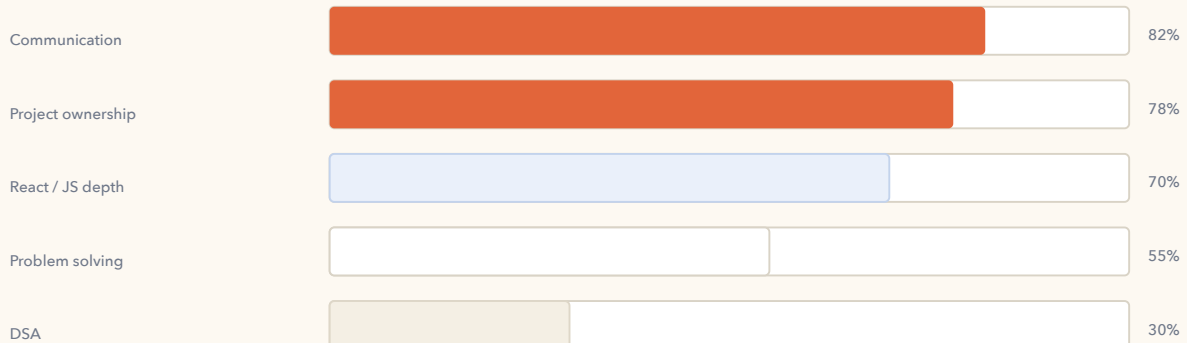
"My professional backend work has been Node.js and Express. I have strong OOP fundamentals – encapsulation, inheritance, polymorphism, SOLID – and I've worked with Java in academic projects. For this role my depth is on the React and Node side, and I pick up new backend stacks quickly, but I want to be upfront that Java/Spring isn't my production experience."

నిజాయితీ ఇక్కడ strength. Service companies BGV + skill validation చేస్తారు – అబద్ధం తర్వాత దొరుకుతుంది.

4. Infosys ఎలా evaluate చేస్తుంది – ఎందుకు reject చేస్తారు

INFOSYS EVALUATION • ఏది నిజంగా బరువు

EVALUATION – ఏది ఎంత బరువు



ఇది service company – అందుకే

Client-facing billable role. మీరు రేపు client meeting lo మాట్లాడగలరా అన్నది ముఖ్యం.
అందుకే communication మరియు ownership – technical depth అంత బరువు.
DSA grinding కంటే – **project narrative + STAR stories** మీద సమయం పెట్టడం మేలు.

వివరణ

Infosys interviewer ఒక **feedback form** నింపుతాడు. Typical parameters:

Parameter	Weight	ఏం చూస్తారు
Technical depth	High	Fundamentals clear గా ఉన్నాయా
Communication	చాలా High	Client ముందు మాట్లాడగలడా – ఇది service company
Project ownership	High	నిజంగా code రాశాడా, లేక "team చేసింది" అంటాడా
Attitude / flexibility	High	Location, shift, technology change కి ok నా
Learnability	Medium	కొత్తది నేర్చుకుంటాడా
Culture fit	Medium	Process follow చేస్తాడా

గమనించు: Communication కి product company కంటే చాలా ఎక్కువ weight. కారణం – నువ్వు client calls లో ఉంటావు, daily standups US/UK client తో ఉంటాయి.

Real-life Scenario

ఇద్దరు candidates. **Candidate A:** React internals చాలా బాగా తెలుసు, Fiber architecture explain చేయగలడు, కానీ ముక్కలు ముక్కలుగా మాట్లాడతాడు, "మా team చేసింది", "నాకు ఆ part తెలియదు" అని చెప్తాడు. **Candidate B:** React basics మాత్రమే తెలుసు, కానీ clear గా, structured గా మాట్లాడతాడు, తన project ని end-to-end explain చేస్తాడు, "నేను ఈ feature own చేశాను, ఇలా చేశాను, ఈ problem వచ్చింది, ఇలా solve చేశాను" అంటాడు.

Infosys లో Candidate B select అవుతాడు. ఇది product company అయితే A. ఈ తేడా తెలుసుకోవడమే ఈ doc యొక్క ముఖ్య ఉద్దేశం.

ఎందుకు reject చేస్తారు – top 8 కారణాలు

1. **Project లో ownership లేకపోవడం.** "మా team చేసింది", "నేను ఆ module చూడలేదు" – ఇది instant red flag.
2. **Fundamentals లో గుంతలు.** Closures, `this`, event loop, `var/let/const`, virtual DOM – ఇవి రాకపోతే out. **Depth కాదు, gap problem.**
3. **Communication.** Answer structure లేకపోవడం, rambling, English లో struggle.
4. **Resume లో లేని skill claim చెయ్యడం** – resume లో ఉన్న ప్రతి technology గురించి అడుగుతారు.
5. **Managerial round లో "నాకు conflict ఎప్పుడూ రాలేదు"** అనడం – ఇది "ఇతను team లో పని చెయ్యలేదు" అని అర్థం.
6. **Location/shift కి rigid గా "no" చెప్పడం.** Infosys కి flexibility కావాలి.
7. **Notice period చాలా ఎక్కువ (90 days) + negotiate చెయ్యలేకపోవడం** – వాళ్ళకి fast onboarding కావాలి.
8. **Current company/manager గురించి negative గా మాట్లాడటం** – HR round killer.

Gotchas (సాధారణ తప్పులు)

1. **Over-preparing DSA, under-preparing project story.** Infosys లో నీ project గురించి **20-25 minutes** మాట్లాడతారు. DSA మీద 5 minutes. Time allocation అలా ఉంచు.
2. **"I don't know" ని అస్సలు చెప్పకపోవడం.** తెలియకపోతే – "I haven't worked with that directly, but based on my understanding of X, I'd approach it this way..." ఇది **మంచి signal**. కానీ bluff చెయ్యడం **worst signal**.
3. **Camera off, background noise.** Teams interview – camera on, quiet room, decent light. ఇది communication score లో లెక్కిస్తారు.

Key Points

- **Communication ≈ Technical** weight (service company).
- **Ownership language వాడు:** "I built", "I decided", "I fixed" – "we did" కాదు.
- Resume లో రాసిన ప్రతి skill = fair game.
- Reject కారణం సాధారణంగా **depth లేకపోవడం కాదు, gaps + presentation.**

Interview దృష్టి

ప్రతి answer కీ ఈ structure వాడు (ఇది Infosys interviewers కీ బాగా నచ్చుతుంది):

1. **ఏమిటి** (one-line definition)
2. **ఎందుకు / ఎప్పుడు** (why it exists, when to use)
3. **నా project లో ఎక్కడ వాడాను** (concrete example)

ఉదాహరణ – "What is useMemo?"

"useMemo caches the result of an expensive computation between renders. (ఏమిటి) You use it when a calculation is genuinely costly and its inputs don't change every render – otherwise it adds overhead rather than removing it. (ఎందుకు) In my dashboard project we had a filter-and-sort over about 5,000 rows running on every keystroke; wrapping it in useMemo keyed on the filter state cut the input lag noticeably. (ఎక్కడ)"

ఈ 3-part structure ప్రతి answer కీ వాడితే, నువ్వు 3 YOE కాదు, 5 YOE లా విసిపిస్తావు.

Part 1 – Round 0: Recruiter Screening Call

5. Screening call లో ఏం అడుగుతారు – word-by-word script

వివరణ

ఇది **10-15 minutes phone call**. Technical కాదు – ఇది ఒక **filter**. HR recruiter నీ resume లో ఉన్నది నిజమేనా, నువ్వు available నా, నీ expectation వాళ్ళ budget లో ఉందా – ఇవి check చేస్తారు.

ఇక్కడ చాలామంది careless గా ఉండి fail అవుతారు. "ఇది just HR call కదా" అని casual గా తీసుకుంటారు. కానీ ఈ call లో చెప్పిన CTC number **మొత్తం negotiation ని lock చేస్తుంది.**

Real-life Scenario

ఇది ఒక **house rent** discussion లాంటిది. Owner "నీ budget ఎంత?" అని అడిగినప్పుడు నువ్వు మొదట చెప్పిన number మీదే మిగతా bargaining మొత్తం జరుగుతుంది. తక్కువ చెప్తే, తర్వాత "నేను ఎక్కువ ఇవ్వగలను" అన్నా పని కాదు – anchor set అయిపోయింది.

Screening call లో expected CTC ఇలాగే పని చేస్తుంది. **Anchor ని నువ్వు set చెయ్యి, తక్కువగా కాదు.**

ఏం అడుగుతారు – పూర్తి list + నీ answer

Q1. "Tell me briefly about your experience."

60 seconds. Full story కాదు – headline. "I'm a frontend developer with 3 years of experience, primarily React and JavaScript. Currently at [company] working on [domain – e.g. a customer-facing e-commerce dashboard]. My core stack is React, Redux, TypeScript, and Node on the backend."

Q2. "What is your current CTC?"

నిజం చెప్పు. **Fixed + variable breakup తో చెప్పు** – ఇది professional గా వినిపిస్తుంది: "My current CTC is X lakhs – that's Y fixed plus Z variable." ⚠️ **అబద్ధం చెప్పకు.** Payslips + Form 16 verify చేస్తారు.

Q3. "What is your expected CTC?"

ఇదే critical moment. Range ఇవ్వు, justification తో: "I'm looking at 12 to 14 lakhs fixed, based on my React experience and the current market for this role. But I'm open to discussing it based on the overall package and the growth path."

⚠️ **"Whatever is company standard" అని చెప్పకు** – ఇది negotiation power మొత్తం వదిలేయడం.

Q4. "What is your notice period?"

నిజం చెప్పు + flexibility చూపించు: "It's 60 days officially, but I can likely negotiate it down to 30-45 days with buyout." – **Buyout option mention చెయ్యడం big plus**, ఎందుకంటే Infosys కి fast onboarding కావాలి.

Q5. "Are you comfortable with [location]? Are you open to relocation?"

Infosys locations: Bangalore, Pune, Hyderabad, Chennai, Mysore, Bhubaneswar, Chandigarh, Nagpur. **Rigid "no" చెప్పకు.** "My preference is [X], but I'm open to relocating for the right role." Hybrid/WFO policy గురించి అడుగు – Infosys ఇప్పుడు ఎక్కువగా **office-based / hybrid**.

Q6. "Do you have any offers in hand?"

ఉంటే చెప్పు (leverage). లేకపోతే **అబద్ధం చెప్పకు** – వాళ్ళు offer letter అడగొచ్చు. "I'm in advanced stages with a couple of other companies" అనేది safe, honest middle ground (అది నిజమైతే).

Q7. "Why are you looking for a change?"

Positive framing మాత్రమే. ఎప్పుడూ current company ని తిట్టకు.  "I've been on the same product for two years and I'm looking for larger-scale projects and client exposure."  "My manager is bad / no hike / too much work."

Q8. "Are you okay with rotational/night shifts?" (client మీద ఆధారపడి)

"I'm open to overlapping with client hours. If it's a permanent night shift I'd want to understand the details, but I'm flexible."

నువ్వు అడగాల్సిన questions (ఇవి తప్పకుండా అడుగు)

1. "How many interview rounds are there?"
2. "Is there a coding assessment before the technical round?"
3. **"What's the exact tech stack – is the backend Node or Java?"** ← §3 చూడు
4. "What's the job level / designation for this position?" ← negotiation కి ముఖ్యం
5. "Which location and what's the work model – hybrid or full office?"
6. "What's the client domain – banking, retail, healthcare?"

Gotchas (సాధారణ తప్పులు)

1. **Call కి unprepared గా attend చెయ్యడం.** Recruiter ఎప్పుడైనా call చెయ్యొచ్చు. Busy అయితే – "Can we reschedule to [time]? I want to give this my full attention." ఇది **negative కాదు, professional.**
2. **Expected CTC ని తక్కువ చెప్పడం** భయంతో. Range ఇవ్వు.
3. **Notice period ని ఎక్కువ చెప్పడం** flexibility చూపించకుండా.
4. వాళ్ళు ఏమీ అడగకపోతే నువ్వు ఏమీ అడగకపోవడం – questions అడగడం interest చూపిస్తుంది.

Key Points

- ఇది **filter call**, technical కాదు – కానీ **CTC anchor ఇక్కడే set అవుతుంది.**
- CTC: **range + justification.** "Company standard" కాదు.

- Notice period: **నిజం + buyout option.**
- Stack confirm చెయ్యి (Node vs Java).
- Resume, payslips ready ఉంచు – వాళ్ళు email లో అడుగుతారు.

Interview దృష్టి

Call తర్వాత వెంటనే recruiter కి ఒక short email పంపు – updated resume attach చేసి, discussed points confirm చేసి. ఇది Infosys recruiters కి బాగా నచ్చుతుంది (వాళ్ళు 50 candidates handle చేస్తారు, నువ్వు వాళ్ళ pipeline లో top కి వస్తావు).

6. Resume – Infosys కి ఎలా tune చెయ్యాలి

వివరణ

Resume రెండు పనులు చేస్తుంది, మరియు అవి **వేరు వేరు audiences** కోసం:

1. **ATS (Applicant Tracking System) ని దాటడం** – keyword matching software
2. **Interviewer కి question map ఇవ్వడం** – అతను నీ resume చూసే questions అడుగుతాడు

రెండోది చాలామంది గమనించరు. నీ resume లో నువ్వు ఏం రాస్తావో, **interview అదే మీద జరుగుతుంది.** అంటే resume అనేది నీ interview యొక్క **syllabus** – మరియు ఆ syllabus ని **నువ్వే రాస్తున్నావు.**

Real-life Scenario

Resume ఒక **restaurant menu** (§8 లో pitch కి వాడిన అదే analogy, ఇక్కడ ఇంకా బలంగా వర్తిస్తుంది). నువ్వు menu లో "Kubernetes, AWS, GraphQL, Docker, Angular, Vue" అని రాస్తే – interviewer వాటిలో దేని గురించైనా అడగొచ్చు. నీకు React మాత్రమే బాగా వచ్చి, మిగిలినవి ఒకసారి tutorial చూసినవి అయితే – **నువ్వే నీ interview ని sabotage చేసుకున్నావు.**

Rule: నీ resume లో ఉన్న ప్రతి technology గురించి నువ్వు 5 minutes మాట్లాడగలగాలి. లేకపోతే దాన్ని తీసేయ్.

ATS కోసం – structure

- ☐ **Simple, single-column format.** Tables, text boxes, columns, graphics, icons – ATS వీటిని చదవలేదు. **Fancy Canva template = rejected before a human sees it.**
- ☐ **.docx లేదా simple .pdf .** Image-based PDF వద్దు.
- ☐ **Standard section headings:** Summary, Technical Skills, Professional Experience, Education, Projects. Creative headings ("My Journey", "What I Bring") వద్దు.
- ☐ **JD నుండి keywords match చెయ్యి.** JD లో "React.js", "Redux", "REST APIs" అని ఉంటే – నీ resume లో **అదే words** ఉండాలి. ATS exact match చూస్తుంది; "React" ≠ "ReactJS" కొన్ని systems లో.
- ☐ **Dates:** **MMM YYYY – MMM YYYY** format (Jan 2023 – Present). **ఇవి నీ relieving letter తో exact match అవ్వాలి** (§30 BGV warning).
- ☐ **2 pages max.** 3 YOE కి **1-2 pages**, ఎక్కువ కాదు.

- Header లో: పేరు, phone, email, LinkedIn, GitHub, location. **Photo వద్దు, age వద్దు, marital status వద్దు.**

Content – Infosys కి ఏం highlight చెయ్యాలి

1. Summary (3 lines, top లో)

"Frontend developer with 3 years of experience building React applications, currently owning the reporting module of a B2B logistics platform used by 4,000+ users. Strong in React, TypeScript, Redux Toolkit and performance optimisation, with backend experience in Node.js and Express. Experienced working directly with product owners and client stakeholders in an Agile delivery model."

⚠️ ఆ చివరి వాక్యం ముఖ్యం – "client stakeholders", "Agile delivery" – **Infosys ఇవి వెతుకుతుంది.** Service company కి client-facing experience ఒక differentiator.

2. Technical Skills – grouped, honest

Languages:	JavaScript (ES6+), TypeScript, HTML5, CSS3, SQL
Frontend:	React, Redux Toolkit, React Router, React Hook Form, Tailwind CSS
Backend:	Node.js, Express, REST APIs
Databases:	MongoDB, PostgreSQL
Testing:	Jest, React Testing Library
Tools:	Git, Webpack/Vite, Jira, Postman, Chrome DevTools
Practices:	Agile/Scrum, Code Review, CI/CD

⚠️ "Familiar with" category పెట్టకు. Interviewer దాన్ని కూడా అడుగుతాడు. తెలిసినవి మాత్రమే రాయు.

3. Experience – bullets, ప్రతిదీ impact తో

❌ బలహీనమైన bullets (చాలామంది ఇలా రాస్తారు):

- Worked on React components
- Involved in bug fixing and enhancements
- Participated in daily standups
- Used Redux for state management

✅ బలమైన bullets – formula: Action verb + ఏం చేశావు + ఏం ఫలితం (number తో)

- **Owned** the reporting module end to end for a logistics platform serving **4,000+ daily users**
- **Reduced** report interaction time from **2.5s to under 200ms** by introducing list virtualisation and memoising the filter pipeline
- **Built** a shared component library **consumed by 3 teams**, cutting new-screen development time by roughly **40%**

- **Migrated** the module to TypeScript incrementally, eliminating a recurring class of runtime type errors
- **Reviewed** PRs and **mentored 1 junior developer** on React patterns and testing
- **Collaborated** directly with the product owner and client stakeholders in **sprint demos**

ప్రతి bullet లో ఒక number పెట్టాలని ప్రయత్నించకు – కుదిరిన చోట పెట్టు. కానీ కనీసం 3-4 bullets లో numbers ఉండాలి.

4. Education – degree, college, year, percentage/CGPA. **3 YOE కి ఇది కిందన ఉండాలి**, పైన కాదు.

Infosys కి ప్రత్యేకంగా జోడించాల్సినవి

ఈ 5 things resume లో ఉంటే Infosys కి ఎక్కువ appeal అవుతుంది:

1. **Client interaction** – "worked directly with client stakeholders", "presented in sprint demos"
2. **Agile/Scrum** – Infosys delivery model ఇదే
3. **Code review / mentoring** – JL4 (Technology Analyst) కి push చెయ్యడానికి
4. **Domain** – banking, insurance, retail, healthcare, logistics. **Infosys clients domain-specific** – నీ domain match అయితే అది strong signal
5. **Notice period** – resume చివర్లో ఒక line: "Notice Period: 60 days (negotiable)" – recruiters దీన్ని వెతుకుతారు, ఇది వాళ్ళ time save చేస్తుంది

Gotchas (సాధారణ తప్పులు)

1. **రాని skills రాయడం.** ఇది అతిపెద్ద తప్పు. Resume లో ఉన్న ప్రతిదీ fair game. **Kubernetes ఒకసారి touch చేశావా? తీసేయ్.**
2. **Fancy template.** ATS parse చెయ్యలేదు. **Single column, plain.**
3. **Responsibilities రాయడం, achievements కాదు.** "Responsible for developing components" ❌ vs "Built X which achieved Y" ✅
4. **Dates approximate గా రాయడం.** BGV లో mismatch = red flag (§30).
5. **అన్ని applications కి ఒకే resume.** JD keywords ప్రకారం **skills section ని tune చెయ్యి** (అబద్ధం కాదు – order మార్చడం, JD భాష వాడటం).
6. **Photo, age, marital status, "Declaration: I hereby declare..."** – ఇవి outdated, తీసేయ్.
7. **GitHub link ఇచ్చి అందులో ఏమీ లేకపోవడం.** Link ఇస్తే – pinned repos ఉండాలి, README ఉండాలి. లేకపోతే link ఇవ్వకు.

Key Points

- Resume = నీ interview యొక్క **syllabus**. నువ్వే దాన్ని రాస్తున్నావు – **జాగ్రత్తగా రాయు.**
- ప్రతి skill గురించి **5 minutes మాట్లాడగలగాలి.** లేకపోతే తీసేయ్.
- ATS: **single column, standard headings, JD keywords, exact dates.**
- Bullets: **action verb + impact + number.**
- Infosys కి: **client interaction, Agile, mentoring, domain** highlight చెయ్యి.
- **1-2 pages.**

Interview దృష్టి

Resume final చేసాక ఈ test చెయ్యి – "Resume Audit":

నీ resume ప్రింట్ తీసుకుని, ప్రతి line మీద ఒక pen తో ✓ లేదా ✗ పెట్టు:

- ✓ = దీని గురించి నేను 5 minutes confident గా మాట్లాడగలను, ఒక example తో
- ✗ = మాట్లాడలేను

ప్రతి ✗ ని resume నుండి తీసేయ్, లేదా interview కి ముందు దాన్ని ✓ చెయ్యి.

ఇది 20 minutes పని. ఇది చెయ్యకపోతే, interviewer నీ resume లోని బలహీనమైన line ని కచ్చితంగా కనుక్కుంటాడు – అది వాళ్ళ job.

Part 2 – Round 1: Technical Interview

7. Technical round structure – 1 hour లో ఏం జరుగుతుంది

TECHNICAL ROUND · గంట యొక్క విభజన

TECHNICAL ROUND – 1 గంటలో ఏం జరుగుతుంది

0-10 నిమిషాలు Intro + pitch	10-30 నిమిషాలు Project deep-dive	30-45 నిమిషాలు React / JS questions	45-55 నిమిషాలు Live coding	55-60 నిమిషాలు మీ ప్రశ్నలు
--------------------------------	-------------------------------------	--	-------------------------------	-------------------------------

30 నిమిషాలు = project – ఇదే round ని నిర్ణయిస్తుంది

అందుకే ఒక project ని పూర్తిగా సిద్ధం చేసుకోవాలి: architecture, మీ పాత్ర, ఒక కష్టమైన bug, ఒక trade-off నిర్ణయం, ఒక తప్పు నుంచి నేర్చుకున్నది. "నేను ఈ feature చేశాను" సరిపోదు – "ఎందుకు అలా చేశాను" కావాలి.

చివరి 5 నిమిషాలు వృథా చేయకండి

"ప్రశ్నలు ఏమైనా ఉన్నాయా?" – "లేవు" అనడం చెడ్డ signal.
Team ఏం చేస్తుంది · tech stack ఎందుకు ఇది · మొదటి 3 నెలల్లో ఏం ఆశిస్తారు – ఇవి అడగండి.

వివరణ

Infosys technical round 1 hour, Microsoft Teams లో. Interviewer సాధారణంగా ఒక **Senior Developer / Technology Lead** – ఒక client project నుండి, నిన్ను తన team కి తీసుకోవాలా వద్దా అని decide చేస్తున్నాడు.

Typical time breakdown:

Time	ఏం జరుగుతుంది	నీ goal
0-5 min	Introduction, "tell me about yourself"	Tone set చెయ్యడం
5-25 min	Project deep-dive – నీ resume మీద	ఇక్కడే ఎక్కువ marks
25-45 min	JavaScript + React + Redux questions	Fundamentals prove చెయ్యడం
45-55 min	Live coding (CodeSandbox) (కొన్నిసార్లు)	Working code

Time	ఏం జరుగుతుంది	నీ goal
55-60 min	"Any questions for me?"	Interest చూపించడం

గమనించు: Project deep-dive కి 20 minutes – ఇది interview లో అతిపెద్ద block. చాలామంది React questions కి 90% time prep చేసి, project కి 10% చేస్తారు. **అది తలకిందులు.**

Real-life Scenario

Interviewer ఒక **manager hiring a plumber** లాంటివాడు. అతను "pipe అంటే ఏమిటి?" అని అడగడు. అతను అడిగేది: "నువ్వు గతంలో ఏ పని చేశావు? ఒక leak ఎలా fix చేశావు? Customer ఇబ్బంది పెడితే ఏం చేశావు?" – తర్వాత ఒకటి రెండు technical questions వేసి నీకు నిజంగా వచ్చా అని confirm చేస్తాడు.

Project story = నీ portfolio. Technical questions = verification.

అడిగే topics – probability ranking

Candidate reports ఆధారంగా, ఎక్కువ నుండి తక్కువ probability:

Rank	Topic	Probability	ఎక్కడ చదవాలి
1	నీ project	100%	§9 (ఈ doc)
2	React hooks (useState, useEffect)	~95%	§13 + React_Telugu.md
3	JavaScript closures, this, hoisting	~90%	§10 + JavaScript_Telugu.md
4	Virtual DOM, reconciliation, keys	~85%	§13
5	Redux / state management	~80%	§15
6	Event loop, promises, async/await	~80%	§10, §11
7	Performance optimization (memo, lazy, code split)	~75%	§13, §14
8	props vs state, lifecycle methods	~75%	§13
9	API calls (fetch/axios), error handling	~70%	§21
10	Git	~60%	§17
11	Live coding task	~50%	§18
12	CSS / responsive / flexbox	~40%	§16
13	Node/Express/Mongo (Full Stack అయితే)	~40%	§19

Rank	Topic	Probability	ఎక్కడ చదవాలి
14	SQL query	~35%	§20
15	DSA problem	~30%	§23
16	Testing (Jest/RTL)	~25%	§22

ఈ table నీ study plan. Top 8 topics మీద 80% time పెట్టు.

Gotchas (సాధారణ తప్పులు)

1. **Interviewer ని interrupt చెయ్యడం.** Question పూర్తిగా విను, ఒక క్షణం ఆగు, తర్వాత answer చెయ్యి.
2. **చాలా పొడవుగా మాట్లాడటం.** ఒక answer **60-90 seconds**. అంతకంటే ఎక్కువైతే interviewer bored అవుతాడు. Detail కావాలంటే అతనే అడుగుతాడు.
3. **Silence ని భయపడటం.** Coding problem లో ఆలోచిస్తున్నప్పుడు – **loud గా ఆలోచించు**: "Okay, so I need to... let me think about the edge case where the array is empty."
4. **Camera off ఉంచడం.** Camera on ఉంచు.

Key Points

- 1 hour: **project 20 min + tech 20 min + coding 10 min + Q&A 5 min.**
- **Project prep = highest ROI.**
- Top 8 topics = interview లో 80%.
- Difficulty: **easy to medium** – trick questions తక్కువ, breadth ఎక్కువ.

Interview దృష్టి

చివర్లో "Do you have any questions for me?" అన్నప్పుడు **ఎప్పుడూ "No" అనకు**. ఇవి అడుగు:

1. "What does the team's tech stack look like day to day?"
2. "What would I be working on in the first three months?"
3. "How is the team structured – how many devs, how do you handle code reviews?"
4. "Is there scope to work on the architecture side over time?"

8. "Tell me about yourself" – 90-second pitch

90-SECOND PITCH • నాలుగు భాగాలు

90-SECOND PITCH – నిర్మాణం

ఇప్పుడు	పేరు, role, ఎన్ని ఏళ్ళు, ఏ stack	15s
గతం	ముఖ్యమైన 1-2 projects, మీ పాత్ర	40s
ప్రభావం	ఒక కీలక ఫలితం (సంఖ్యతో)	20s
ఇప్పుడు ఎందుకు	ఈ role ఎందుకు కావాలి	15s

చేయకూడనివి

Resume ని పునరుద్ధరించే కంటే చదవడం – వాళ్ళ దగ్గర resume ఉంది.
పుట్టిన ఊరు, కళాశాల marks, వ్యక్తిగత వివరాలు – అడిగితేనే.
3 నిమిషాలు దాటడం – మొదటి impression అక్కడే పోతుంది.

వివరణ

ఇది **ప్రతి round లో మొదటి question** (technical, managerial, HR – అన్నిటిలో). ఇది "warm-up" కాదు – ఇది **frame-setting**. నువ్వు ఇక్కడ ఏ topics mention చేస్తావో, interviewer ఆ topics మీదే తర్వాత questions అడుగుతాడు.

అంటే – **నీకు బాగా వచ్చిన విషయాలు mention చెయ్యి, రాని విషయాలు mention చెయ్యకు**. ఇది నీ చేతిలో ఉన్న ఏకైక control point.

Real-life Scenario

ఇది ఒక **restaurant menu** లాంటిది. నువ్వు menu లో ఏం రాస్తావో customer అదే order చేస్తాడు. నీ pitch లో "I've worked with WebSockets and micro-frontends" అని చెప్తే, interviewer వెంటనే "Tell me about micro-frontends" అని అడుగుతాడు. **నీకు అది deep గా రాకపోతే నువ్వే నీ గొయ్యి తవ్వకున్నట్టు.**

అందుకే pitch లో **నువ్వు 10 minutes మాట్లాడగలిగే విషయాలు మాత్రమే పెట్టు.**

Template – 5 parts, 90 seconds

Part 1 – Present (15 sec): ఎవరు, ఎంత experience, ఏ stack **Part 2 – Current role (30 sec):** ఏ product, ఎంత scale, నీ ownership **Part 3 – Ek concrete achievement (25 sec):** ఒక number తో **Part 4 – Technical identity (10 sec):** నీ strength ఏమిటి **Part 5 – Why here (10 sec):** ఎందుకు Infosys

Code (fill-in-the-blank template)

```
// PART 1 – Present
"I'm [పేరు], a frontend developer with 3 years of experience,
working primarily with React, JavaScript, and TypeScript."

// PART 2 – Current role + ownership
"Currently I'm at [company] on [product – ఒక వాక్యంలో ఏం చేస్తుందో].
I own the [module పేరు] module end to end – that covers
```

[2-3 concrete things: e.g. the checkout flow, the reporting dashboard, the shared component library]."

```
// PART 3 – Achievement with a number ← ఇదే pitch లో బలమైన భాగం
"One thing I'm proud of: [problem] was [specific pain].
I [what you did – memoisation / code splitting / virtualisation /
caching / query optimisation], which brought [metric] from [X] to [Y]."
```

```
// PART 4 – Technical identity
"My strongest areas are React performance, state management with Redux,
and building reusable component systems.
On the backend I work with Node, Express, and MongoDB."
```

```
// PART 5 – Why here
"I'm looking for larger-scale, client-facing projects, which is
exactly what attracted me to this role at Infosys."
```

Complete example (ఇలా వినిపించాలి)

"I'm a frontend developer with 3 years of experience, mainly React, JavaScript and TypeScript. Currently I'm at [company], working on a B2B logistics dashboard that around 4,000 internal users use daily to track shipments.

I own the reporting module end to end – the data grid, the filter system, and the export flow. One thing I'm proud of: our main report table was rendering about 8,000 rows and the page was freezing for two to three seconds on every filter change. I introduced list virtualisation with react-window and memoised the filter pipeline, which brought the interaction time down to under 200 milliseconds. That went out to production and the support tickets around 'dashboard is slow' basically stopped.

My strongest areas are React performance, Redux state management, and building reusable components. On the backend I work with Node, Express and MongoDB – I've built and maintained REST APIs for our own module.

I'm now looking for larger-scale, client-facing work with more architectural involvement, which is what drew me to this role."

ఎందుకు ఇది పని చేస్తుంది:

-  Concrete numbers (4,000 users, 8,000 rows, 2-3s → 200ms)
-  Ownership language ("I own", "I introduced")
-  Business impact ("support tickets stopped") – service company కి ఇది బంగారం
-  Follow-up hooks ఇచ్చాడు: virtualisation, memoisation, Redux – అన్న అతనికి తెలిసినవి
-  90 seconds

Comparison – బలహీనమైన vs బలమైన pitch

బలహీనం (✗)	బలమైన (✓)
"I did my B.Tech in 2021 from..."	Education mention చెయ్యకు (వాళ్ళు resume లో చూశారు)
"I have worked on many technologies like React, Angular, Vue, Node, Python, AWS..."	3-4 core skills మాత్రమే – depth signal
"We developed an application for the client"	"I own the reporting module end to end"
"It improved the performance"	"2-3 seconds down to under 200 milliseconds"
4 minutes పొడవు	90 seconds
"I want a better opportunity"	"Larger-scale, client-facing projects"

Gotchas (సాధారణ తప్పులు)

- 1. Personal details చెప్పడం** – "I'm from Hyderabad, my father is..." – అవసరం లేదు, HR round లో కూడా కాదు.
- 2. Education నుండి మొదలుపెట్టడం.** 3 years experience ఉన్నప్పుడు college irrelevant. **Present నుండి మొదలుపెట్టు.**
- 3. రాని technology mention చెయ్యడం.** Resume లో ఉన్నా, pitch లో highlight చెయ్యకు.
- 4. Numbers లేకపోవడం.** ఒక్క number కూడా లేని pitch = generic. **కనీసం 2 numbers** పెట్టు.
- 5. బట్టీ పట్టినట్టు చెప్పడం.** Structure గుర్తుపెట్టుకో, wording కాదు. Practice చెయ్యి కానీ robot లా కాదు.

Key Points

- **90 seconds.** అంతకు మించకూడదు.
- Structure: Present → Role/ownership → Achievement (numbers తో) → Strengths → Why here
- **నీకు వచ్చిన topics మాత్రమే mention చెయ్యి** – ఇవే follow-up questions అవుతాయి.
- Ownership language: "I built", "I own", "I decided".
- 5 సార్లు **loud గా** practice చెయ్యి, timer పెట్టుకుని.

Interview దృష్టి

Managerial round లో ఇదే question వస్తుంది కానీ **emphasis మార్పు** – technical details తగ్గించి, **ownership + team + client** పెంచు:

"...I own the reporting module, and I also mentor one junior developer on the team. I work directly with our product owner to scope requirements, and I've been the point person for our US client's weekly demo calls."

9. Project deep-dive – ఇదే round ని decide చేస్తుంది

PROJECT DEEP-DIVE • STAR తో ఐదు ప్రశ్నలు

PROJECT DEEP-DIVE – వాళ్ళు ఏం వెతుకుతున్నారు

Architecture	ఎలా కట్టారు, ఎందుకు అలా
మీ పాత్ర	మీరు ఏం చేశారు – "we" కాదు, "I"
Trade-off	ఏది ఎంచుకున్నారు, ఏది వదిలేశారు
కష్టమైన bug	ఎలా debug చేశారు
మళ్ళీ చేస్తే	ఏం మారుస్తారు – ఇది maturity signal

STAR నిర్మాణం – ప్రతి కథకీ

Situation (సందర్భం) → Task (నా బాధ్యత) → Action (నేను ఏం చేశాను) → Result (ఫలితం, సంఖ్యతో)
రెండు నిమిషాలు మించకుండా. Result lo ఒక సంఖ్య ఉండాలి – "40% వేగం", "బగ్లు సగం".

వివరణ

Technical round లో **20-25 minutes** నీ project మీద మాట్లాడతారు. ఇది Infosys interview యొక్క **అతిపెద్ద single block**, కానీ చాలామంది దీనికి prepare అవ్వరు – "నా project నాకు తెలుసు కదా" అనుకుంటారు.

సమస్య: నీకు project తెలుసు, కానీ దాన్ని **structured గా, 5 minutes లో, interviewer కి అర్థమయ్యేలా** చెప్పడం వేరే skill. అది practice చెయ్యాలి.

Real-life Scenario

నువ్వు ఒక సినిమా చూశావు. ఎవరైనా "ఆ సినిమా ఎలా ఉంది?" అని అడిగితే – నువ్వు మొదటి scene నుండి చివరి scene వరకు చెప్పవు కదా? నువ్వు చెప్పేది: "ఇది ఒక thriller, hero ఒక journalist, అతను ఒక scam కనిపెడతాడు, climax లో twist ఉంటుంది." – **structure ఉంటుంది.**

Project కూడా అంతే. "మేము React వాడాము, Redux వాడాము, API calls చేశాము" అనేది scene-by-scene narration. **Structure కావాలి.**

Project pitch structure – 5 layers

ఈ order లో చెప్పు (ప్రతి layer కి 1 వాక్యం చాలు, interviewer అడిగితే deep వెళ్ళు):

Layer 1 – Business context (ఏమిటి, ఎవరికీ):

"It's a B2B logistics dashboard used by about 4,000 warehouse operators to track shipments in real time."

Layer 2 – Architecture (ఎలా కట్టారు):

"React on the front end with Redux Toolkit for state, a Node/Express API layer, MongoDB for storage, and Redis for caching the frequently-hit lookup data. Deployed on AWS behind CloudFront."

Layer 3 – నీ ownership (అతి ముఖ్యం):

"I own the reporting module – that's the data grid, the filtering system, and the CSV/PDF export. I also maintain our shared component library that the other three teams consume."

Layer 4 – ఒక hard problem నువ్వు solve చేసినది:

"The hardest problem I hit was..." – §9 లోని STAR template వాడు (Part 3, §25).

Layer 5 – Impact + learnings:

"That change cut the report load from 6 seconds to under 1 second, and the pattern got adopted by two other teams."

Interviewer అడిగే follow-up questions – ఇవి 100% వస్తాయి

ఈ ప్రతి question కీ నీ project కి specific answer ready ఉంచు. Generic answer చెప్తే పట్టుబడతావు.

#	Question	ఏం చూస్తున్నారు
1	"What was your specific contribution?"	Ownership – అతి ముఖ్యం
2	"How many people were on the team?"	Scale sense
3	"Why did you choose Redux over Context API?"	Decision reasoning
4	"How did you handle API errors and loading states?"	Practical maturity
5	"How did you handle authentication?"	Security awareness
6	"What was the biggest technical challenge?"	Problem solving
7	"How did you optimise performance?"	Senior signal
8	"How did you test it?"	Quality mindset
9	"What would you do differently if you rebuilt it?"	Self-awareness – senior signal
10	"How did you deploy? What was the CI/CD?"	End-to-end awareness
11	"How did you handle state that had to be shared across routes?"	Architecture
12	"How many users? What kind of load?"	Scale sense

Q3 కి model answer (ఇది తరచుగా అడుగుతారు)

"Why Redux and not Context API?"

"We started with Context, actually. It worked fine while the state was small. The problem showed up as the dashboard grew – Context re-renders every consumer whenever the provider value changes, and we had a single provider holding filters, user preferences and cached shipment data together. A filter change was re-rendering the entire tree.

We had two options: split the context into several providers with memoised values, or move to Redux. We went with Redux Toolkit because we also needed devtools for debugging production issues, and we had async flows – polling and optimistic updates – where RTK Query gave us caching and request de-duplication for free. Context is great for low-frequency values like theme or auth; it's not a state management library, and we were treating it like one."

ఇది ఎందుకు మంచి answer: trade-off చూపించాడు, alternative consider చేశాడు, **"Context is not a state management library"** అనే mature statement ఉంది, concrete reason (devtools, RTK Query) ఉంది.

Gotchas (సాధారణ తప్పులు)

1. **"We" అనడం.** ప్రతి "we" ని "I" గా మార్చు – నీ contribution ఎక్కడో అక్కడ. "We built the app" ❌ → "I built the reporting module; the team built the rest" ✅
2. **Numbers లేకపోవడం.** Users ఎంతమంది? Rows ఎన్ని? Response time ఎంత? Team size? **ఈ numbers ముందే రాసుకో.**
3. **Resume లో లేని project చెప్పడం లేదా resume లో ఉన్న project deep గా తెలియకపోవడం.** Resume లో ఉన్న ప్రతి project = fair game.
4. **"What would you do differently?" కి "nothing" అనడం.** ఇది arrogance లేదా reflection లేకపోవడం. **ఒక honest answer ready ఉంచు:** "I'd introduce TypeScript from day one – we retrofitted it in year two and that migration cost us weeks."
5. **NDA/confidential అని మొత్తం refuse చెయ్యడం.** Client పేరు చెప్పకపోవడం fine, కానీ **architecture, నీ role, technical decisions** చెప్పొచ్చు. "I can't name the client, but the domain is insurance claims processing and the architecture was..."

Key Points

- **20-25 minutes** ఈ block కి – interview లో అతిపెద్దది.
- 5 layers: Business → Architecture → **నీ ownership** → Hard problem → Impact.
- పైన ఉన్న **12 follow-up questions కి answers రాసుకో** – ఇది non-negotiable homework.
- **Numbers ముందే memorize చెయ్యి:** users, team size, data volume, before/after metrics.
- "I", "we" కాదు.

Interview దృష్టి

Homework (ఇది ఇప్పుడే చెయ్యి): ఒక page తీసుకుని ఇవి రాయ –

1. నీ project ఒక వాక్యంలో (business context)
2. Architecture diagram (frontend → API → DB → cache)
3. నీ ownership – 3 bullet points
4. 3 hard problems + వాటిని ఎలా solve చేశావు (STAR format)
5. 6 numbers: users, team size, data scale, 2 before/after metrics, uptime/traffic
6. పైన 12 questions కి one-paragraph answers

ఈ ఒక్క page నీ interview లో 40% marks.

10. JavaScript question bank – Infosys depth

వివరణ

Infosys JavaScript questions **conceptual**, trick కాదు. వాళ్ళు "explain closures" అని అడుగుతారు, "implement a JavaScript engine" అని కాదు. కానీ **గుంత ఉంటే వెంటనే పట్టుకుంటారు** – ఎందుకంటే follow-up అడుగుతారు.

Rule: ప్రతి answer = **definition + why it exists + one example**. `JavaScript_Telugu.md` లో ప్రతి topic deep గా ఉంది – ఇక్కడ **Infosys కి ఎంత చెప్పాలి** అనేది ఉంది.

Depth calibration: ★ = basics, తప్పకుండా అడుగుతారు · ★★ = ఎక్కువగా అడుగుతారు · ★★ ★ = follow-up గా వస్తుంది, ఇది వస్తే నువ్వు senior లా కనిపిస్తావు

A. Scope, Hoisting, this (★★★ – దాదాపు ఖాయం)

Q1 ★ var , let , const తేడా?

`var` is function-scoped and hoisted with an initial value of `undefined`. `let` and `const` are block-scoped and hoisted too, but they sit in the **temporal dead zone** – accessing them before the declaration throws a `ReferenceError` instead of giving you `undefined`. `const` prevents reassignment of the binding, but it does not make objects immutable – you can still mutate properties of a `const` object. Also, `var` at the top level attaches to `window`; `let` / `const` don't.

Q2 ★★ Hoisting అంటే ఏమిటి?

During the compilation phase the engine allocates memory for declarations before any code runs. Function declarations are hoisted fully – you can call them before they appear. `var` is hoisted and initialised to `undefined`. `let` / `const` are hoisted but uninitialised (TDZ). Function *expressions* and arrow functions follow the rules of the variable they're assigned to, so calling one before its line throws.

Q3 ★★ ★ Closure అంటే ఏమిటి? – Infosys favourite

A closure is a function bundled together with the lexical environment it was created in. Even after the outer function returns, the inner function still holds a live reference to those outer variables, so they aren't garbage collected.

```
function counter() {  
  let count = 0;           // ఈ variable outer function return అయ్యాక కూడా బతికే  
  ఉంటుంది  
  return () => ++count;     // ఈ inner function 'count' మీద closure  
}  
const inc = counter();  
inc(); // 1  
inc(); // 2 ← count reset అవ్వలేదు
```

ఎక్కడ వాడతాము: data privacy (module pattern), function factories, `once`, memoisation, debounce/throttle, and every React hook – `useState` keeps its value across renders through closures.

Q4 ★★ Closures memory leak కి కారణం అవుతాయా?

They can. A closure keeps its entire enclosing scope alive, so if you close over a large object or a DOM node and the closure itself is long-lived – an event listener you never remove, a `setInterval` you never clear – that memory can't be reclaimed. The fix is to remove listeners, clear timers, and null out references you no longer need. In React that's exactly what the `useEffect` cleanup function is for.

Q5 ★★★ `this` ఎలా పని చేస్తుంది?

`this` is decided by **how a function is called**, not where it's defined. Four rules, in priority order:

1. `new Foo()` → `this` is the newly created object
2. `fn.call/apply/bind(obj)` → explicitly set to `obj`
3. `obj.method()` → `this` is `obj` (implicit binding)
4. Plain `fn()` → `globalThis` in sloppy mode, `undefined` in strict mode

Arrow functions are the exception – they have no `this` of their own. They take it lexically from the enclosing scope at definition time, and `call / bind` can't change it. That's why arrows are the right choice for callbacks inside a class method, and the wrong choice for an object method that needs `this` to be the object.

Q6 ★★ `call`, `apply`, `bind` తేడా?

All three set `this` explicitly. `call(thisArg, a, b)` invokes immediately with individual arguments. `apply(thisArg, [a, b])` invokes immediately with an array. `bind(thisArg, ...)` doesn't invoke – it returns a **new function** with `this` permanently locked, and it supports partial application. Memory hook: **c**all = **c**omma, **a**pply = **a**rray, **b**ind = **b**ound-for-later.

B. Types, Coercion, Equality (☆☆)

Q7 ☆ == vs === ?

`===` compares type and value with no conversion. `==` performs type coercion first, which produces surprises like `'' == 0 → true` and `null == undefined → true` (but `null == 0 → false`). **Always use `===`**; the one common exception is `x == null`, which is a neat way to check for `null` or `undefined` in a single comparison.

Q8 ☆☆ JavaScript లో data types ఎన్ని?

Seven primitives – `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint` – and everything else is an object (arrays, functions, dates, all objects). Primitives are copied by value; objects are copied by reference.

Q9 ☆☆ Shallow copy vs deep copy? – Infosys ఇది అడుగుతుంది

A shallow copy duplicates only the top level – nested objects are still shared by reference, so mutating a nested property affects both copies. `Object.assign({}, obj)` and the spread `{...obj}` are shallow.

A deep copy clones the whole tree. Options: `structuredClone(obj)` – built into modern browsers and Node, handles Dates, Maps, Sets and circular references; `JSON.parse(JSON.stringify(obj))` – quick but loses `undefined`, functions, `Date` objects (become strings), and throws on circular references; or lodash's `cloneDeep`.

In React this matters constantly – state updates must be immutable, so you spread each level you're changing rather than mutating in place.

Q10 ☆ null vs undefined ?

`undefined` means a variable has been declared but never assigned – it's the engine's default. `null` is an explicit "intentionally empty" value that a developer assigns. Quirk to know: `typeof undefined` is `"undefined"` but `typeof null` is `"object"`, which is a long-standing bug in the language kept for backward compatibility.

Q11 ☆☆ Falsy values ఏవి?

Exactly eight: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy – including `[]`, `{}`, `"0"` and `"false"`, which trips people up.

C. Event Loop & Async (☆☆☆ – తప్పకుండా అడుగుతారు)

Q12 ☆☆☆ Event loop ఎలా పని చేస్తుంది?

JavaScript has one call stack, so it runs one thing at a time. Async work is handed off to the runtime – the browser's Web APIs or libuv in Node – and when it finishes, the callback goes into a queue.

The event loop's job: when the call stack is empty, it first drains the **entire microtask queue** (promise callbacks, `queueMicrotask`, `MutationObserver`), then takes **one** task from the macrotask queue (`setTimeout`, `setInterval`, I/O, UI events), then drains microtasks again, and repeats.

The key point for interviews: microtasks always beat macrotasks. A

`Promise.resolve().then()` scheduled after a `setTimeout(fn, 0)` still runs first.

Q13 ☆☆☆ `setTimeout(fn, 0)` వెంటనే run అవుతుందా?

No. It's queued as a macrotask and can only run once the call stack is empty and all pending microtasks have drained. The `0` is a *minimum* delay, not a guarantee – browsers also clamp nested timeouts to about 4ms.

Q14 ☆☆☆ Callback hell అంటే ఏమిటి, ఎలా solve చేస్తారు?

Nested callbacks for sequential async work – each level indents further, error handling gets duplicated at every level, and the code becomes unreadable ("the pyramid of doom"). Promises flattened this into a `.then()` chain with a single `.catch()`, and `async/await` made it read like synchronous code with normal `try/catch`.

Q15 ☆☆☆ Promise అంటే ఏమిటి? States ఏవి?

An object representing the eventual result of an async operation. Three states: **pending**, **fulfilled**, **rejected**. Once it settles it's immutable – it can't change again. Promises solve two problems callbacks had: inversion of control (you're not handing your callback to someone else's code to call whenever they like) and the lack of composability.

Q16 ☆☆☆ `Promise.all`, `allSettled`, `race`, `any` తేడా?

Method	ఎప్పుడు resolve అవుతుంది	ఎప్పుడు reject అవుతుంది	ఎప్పుడు వాడాలి
<code>all</code>	అన్నీ fulfil అయితే	ఏ ఒక్కటి reject అయినా (fail-fast)	అన్ని results కావాలి, ఒకటి fail అయితే మొత్తం fail
<code>allSettled</code>	ఎప్పుడూ (అన్నీ settle అయ్యారే)	ఎప్పుడూ కాదు	ప్రతి result status తెలియాలి, partial failure ok
<code>race</code>	మొదట settle అయినది (fulfil లేదా reject)	మొదటిది reject అయితే	Timeout patterns
<code>any</code>	మొదట fulfil అయినది	అన్నీ reject అయితే (<code>AggregateError</code>)	Multiple mirrors, ఏదో ఒకటి పని చేస్తే చాలు

Infosys follow-up: "Dashboard లో 5 API calls ఉన్నాయి, ఒకటి fail అయినా మిగతావి చూపించాలి – ఏం వాడతావు?" → `allSettled` . `all` వాడితే ఒక్క failure మొత్తం page ని పడేస్తుంది.

Q17 ★★ `async/await` – promises కంటే ఎలా better?

It's syntactic sugar over promises – an `async` function always returns a promise, and `await` pauses that function until the promise settles without blocking the main thread. The benefits are readability, normal `try/catch` error handling, and real stack traces on failure.

The classic mistake: awaiting independent calls sequentially.

```
// ❌ 3 seconds – ఒకదాని తర్వాత ఒకటి
const a = await getUser();
const b = await getPosts();
const c = await getComments();

// ✅ 1 second – parallel గా start అయి, అన్నీ కలిపి await
const [a, b, c] = await Promise.all([getUser(), getPosts(), getComments()]);
```

ఈ ఒక్క snippet interview లో చూపిస్తే senior signal.

D. Arrays & Functional methods (★★)

Q18 ★★ `map`, `filter`, `reduce` తేడా?

All three are non-mutating and return something new. `map` transforms each element and returns a new array of the **same length**. `filter` returns a new array containing only the elements that pass a predicate – length is **less than or equal to** the original. `reduce` folds the array down to a **single value** – which can be a number, an object, or even another array; `map` and `filter` can both be written in terms of `reduce` .

Q19 ★★ `forEach` vs `map` ?

`forEach` returns `undefined` and exists purely for side effects; `map` returns a new array and is meant for transformation. Neither can be broken out of early – you need a `for...of` or `some / every` for that. **Don't use `map` when you're ignoring the return value** – it allocates an array for nothing, and reviewers read it as a misunderstanding.

Q20 ★★ `slice` vs `splice` ?

`slice(start, end)` is **non-mutating** – it returns a shallow copy of a portion. `splice(start, deleteCount, ...items)` **mutates** the original array, removing and/or inserting in place, and returns the removed elements. Memory hook: **splice** has the **p** for "permanent damage". In React state, use `slice / spread`, never `splice`.

Q21 ★★ `find` vs `filter`, `some` vs `every` ?

`find` returns the **first matching element** (or `undefined`) and stops early; `filter` scans everything and returns an array. `some` returns `true` if at least one element passes (stops at the first hit); `every` returns `true` only if all pass (stops at the first failure). Use `find / some` when you only need one answer – they short-circuit.

E. ES6+ features (★★)

Q22 ★★ Spread vs rest operator?

Same `...` syntax, opposite jobs. **Spread** expands an iterable into individual elements – `[...arr1, ...arr2]`, `{...obj, key: value}`, `fn(...args)`. **Rest** collects the remaining items into an array or object – `function f(a, ...others)`, `const {a, ...rest} = obj`. Rule of thumb: on the **right** side of `=` or in a call it's spread; on the **left** side or in a parameter list it's rest.

Q23 ★ Destructuring అంట్?

Pulling values out of arrays or objects into variables in one statement, with support for defaults, renaming, and nesting: `const {name, age = 18, address: {city} = {}} = user;`. Used constantly in React for props and hook returns.

Q24 ★★ Optional chaining మరియు nullish coalescing?

`?.` short-circuits to `undefined` instead of throwing when something in the chain is `null` or `undefined` – `user?.address?.city`. It works on calls too: `callback?.()`. `??` returns the right side only when the left is `null` or `undefined` – unlike `||`, which also falls through on `0`, `''` and `false`. That difference is a real bug source: `const count = props.count || 10` gives 10 when `count` is 0, but `props.count ?? 10` correctly gives 0.

Q25 ★ Template literals?

Backtick strings with `${}` interpolation, multi-line support, and tagged templates for custom processing (that's how styled-components works).

F. Objects & Prototypes (★★)

Q26 ★★ Prototypal inheritance అంటే ఏమిటి?

Every object has a hidden link (`[[Prototype]]`), reachable via `Object.getPrototypeOf` or the legacy `__proto__` to another object. When you access a property that isn't on the object itself, the engine walks up this **prototype chain** until it finds the property or reaches `null`. This is how `arr.map` works – `map` lives on `Array.prototype`, not on your array.

`class` in ES6 is **syntactic sugar over exactly this** – JavaScript doesn't have classical inheritance under the hood.

Q27 ★★ `Object.freeze` deep freeze చేస్తుందా?

No – it's shallow. The top-level properties become non-writable, but nested objects are still mutable. For a deep freeze you recurse over the properties and freeze each one.

Q28 ★ `Map` vs plain object?

`Map` accepts **any type as a key** (objects, functions), preserves insertion order, has a `.size` property, and is directly iterable. A plain object coerces all keys to strings/symbols and inherits from `Object.prototype`, so keys like `toString` can collide. Use `Map` for frequent additions/deletions and non-string keys; use an object for simple records and JSON-shaped data.

G. Misc (★★)

Q29 ★★ Debouncing vs throttling? – Infosys favourite

Both limit how often a function runs, but differently. **Debounce** waits until the events *stop*: it resets a timer on every call and only fires after a quiet period – ideal for a search-as-you-type input, where you want one API call after the user pauses. **Throttle** guarantees a maximum rate: it fires at most once per interval regardless of how many events arrive – ideal for scroll, resize, or mousemove handlers.

Implementations are in §12 – వాటిని రాయమని అడుగుతారు.

Q30 ★★ Event bubbling, capturing, delegation?

An event travels down from the root to the target (**capturing**), fires on the target, then travels back up (**bubbling**) – bubbling is the default phase your handlers run in.

`stopPropagation()` halts the travel; `preventDefault()` cancels the browser's default action – they're different things.

Event delegation exploits bubbling: instead of attaching a listener to 500 list items, you attach one to the parent and read `event.target`. That means less memory and it automatically works for items added later. **React uses exactly this internally** – it attaches listeners at the root container rather than on each element (at the document level before React 17, at the root container from 17 onwards).

Q31 ★★ Memory management / garbage collection?

JavaScript is garbage collected using a **mark-and-sweep** algorithm: starting from roots (global object, current call stack), the collector marks everything reachable and frees the rest. Reference counting alone would leak on circular references, which is why mark-and-sweep is used.

Common leak sources: event listeners never removed, `setInterval` never cleared, detached DOM nodes still referenced in a variable, growing global caches, and closures holding large objects. In React, that's what `useEffect` cleanup exists for.

Q32 ★★ localStorage vs sessionStorage vs cookies?

–	Size	Expiry	Server కి వెళ్తుందా
localStorage	~5-10MB	Manual గా clear చేసేవరకు	✗
sessionStorage	~5MB	Tab close అయితే పోతుంది	✗
Cookies	~4KB	Set చేసిన expiry	✅ ప్రతి request తో

Security follow-up: "JWT token ఎక్కడ store చేస్తావు?" → "localStorage" is readable by any JavaScript on the page, so it's vulnerable to XSS. The safer pattern is an **httpOnly, Secure, SameSite cookie** – JavaScript can't read it, which kills the XSS token-theft path, though you then need CSRF protection." – ఈ answer Infosys లో బాగా score చేస్తుంది.

Gotchas (సాధారణ తప్పులు)

1. **Definition మాత్రమే చెప్పి ఆగిపోవడం.** ప్రతి answer కి "ఎప్పుడు వాడతాము" + ఒక example జోడించు.
2. **Closure ని "function inside a function" అని చెప్పడం.** అది incomplete – "function + its lexical environment, which survives after the outer function returns" అనాలి.
3. **this ని arrow function context లో తప్పు చెప్పడం.** Arrow కి own this లేదు – "arrow function this ని parent నుండి తీసుకుంటుంది" అనేది సరైన phrasing.
4. **Promise.all ని అన్నిటికీ suggest చెయ్యడం.** Partial failure ok అయితే allSettled .

Key Points

- Infosys JS = **conceptual, breadth-first**. Trick questions తక్కువ.
- **Big four తప్పకుండా:** closures, this , event loop, hoisting.
- ప్రతి answer: **definition** → **why** → **example**.
- Deep dive కావాలంటే → JavaScript_Telugu.md .

Interview దృష్టి

ఈ 8 questions ఖాయంగా వస్తాయి – వీటిని బట్టి కాదు, **loud గా practice** చెయ్యి: var/let/const · closures · this · event loop · == vs === · shallow vs deep copy · debounce vs throttle · map/filter/reduce

11. JavaScript output-based puzzles – 12 verified snippets

వివరణ

"What does this code print?" – Infosys technical round లో ఇవి తప్పకుండా వస్తాయి. ఇవి నీ **mental model** ని test చేస్తాయి, memory ని కాదు.

✅ ఈ section లోని ప్రతి output node లో run చేసి verify చేయబడింది – guess కాదు.

Answer ఎలా చెప్పాలి: output చెప్పి ఆగకు. ఎందుకు అని explain చెయ్యి. Output తప్పు చెప్పి reasoning సరిగ్గా చెప్తే కూడా marks వస్తాయి. Output సరిగ్గా చెప్పి "నాకు తెలుసు, గుర్తుంది" అంటే marks రావు.

Puzzle 1 ★★☆☆ – var in a loop (అత్యంత famous)

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

```
}
```

Output: 3 3 3

ఎందుకు: `var` is function-scoped, so all three callbacks close over the **same single** `i`. The loop finishes synchronously and leaves `i` at 3; only then does the event loop run the timeout callbacks, and all three read that same variable.

Fix 1 – `let` : block-scoped, so each iteration gets a **fresh binding**. **Fix 2 – IIFE:**

`setTimeout(((x) => () => console.log(x))(i), 0)` – capture the value per iteration.

Puzzle 2 ★★ – same loop with `let`

```
for (let j = 0; j < 3; j++) {  
  setTimeout(() => console.log(j), 0);  
}
```

Output: 0 1 2

ఎందుకు: `let` creates a new binding **per iteration** – the spec literally copies the value into a fresh environment record each time round. Each closure captures its own `j`.

Puzzle 3 ★★★ – micro vs macro task

```
console.log('A');  
setTimeout(() => console.log('B'), 0);  
Promise.resolve().then(() => console.log('C'));  
console.log('D');
```

Output: A D C B

ఎందుకు: `A` and `D` are synchronous – they run first. Then the call stack empties. The event loop drains the **microtask queue** first, so `C` (a promise callback) runs. Only then does it take one **macrotask** – `B`. ఒక్క వాక్యంలో: *sync* → *microtasks* → *macrotasks*.

Puzzle 4 ★★ – Promise executor is synchronous

```
console.log('start');  
const p = new Promise((resolve) => {  
  console.log('executor');  
  resolve('resolved');  
});
```

```
p.then(v => console.log(v));
console.log('end');
```

Output: start executor end resolved

పొందుకు: The **executor function runs synchronously** the moment you call `new Promise` – that surprises a lot of people. Only the `.then` callback is deferred to the microtask queue, so it lands after `end`.

Puzzle 5 ★★☆☆ – `this` in method vs arrow

```
const obj = {
  name: 'infy',
  regular() { return this.name; },
  arrow: () => this.name,
};
console.log(obj.regular());
console.log(obj.arrow());
```

Output: infy then undefined

పొందుకు: `regular` is called as `obj.regular()`, so implicit binding makes `this = obj`. `arrow` has **no `this` of its own** – it takes it lexically from where it was *defined*, which is the enclosing module/global scope, not the object. Objects don't create a scope.

Rule: object methods కి **arrow function వాడకు** if you need `this`.

Puzzle 6 ★★☆☆ – hoisting & TDZ

```
console.log(typeof a); // ?
var a = 5;

console.log(b); // ?
let b = 5;
```

Output: undefined, then a **ReferenceError**

పొందుకు: `var a` is hoisted and initialised to `undefined`, so `typeof a` is `"undefined"`. `let b` is hoisted too but sits in the **temporal dead zone** until its declaration executes – touching it before that throws `ReferenceError: Cannot access 'b' before initialization`.

Trap: `typeof` is normally safe on undeclared variables, but **it does NOT protect you from the TDZ** – `typeof b` before `let b` also throws.

Puzzle 7 ★★ – coercion rapid-fire

```
console.log(1 + '2');           // '12'
console.log('3' - 1);           // 2
console.log([] + {});           // '[object Object]'
console.log(0.1 + 0.2 === 0.3); // false
console.log([] == false);       // true
console.log(null == undefined); // true
console.log(null === undefined); // false
console.log(null == 0);         // false
console.log(typeof NaN);        // 'number'
console.log(NaN === NaN);       // false
```

ఎందుకు (ముఖ్యమైనవి):

- `+` is overloaded – if **either** operand is a string it concatenates; `-` has no string meaning so it coerces to number.
- `0.1 + 0.2` is `0.30000000000000004` – IEEE-754 floating point can't represent these exactly. **Fix:** compare with an epsilon, or use integers (paise/cents) for money.
- `[] == false` → `[]` becomes `''` → `0`, and `false` becomes `0` → true. **This is why `===` exists.**
- `null == undefined` is true but `null == 0` is false – `null` only loosely equals `undefined`, nothing else.
- `NaN === NaN` is false by IEEE-754 design. Use `Number.isNaN(x)` (not the global `isNaN`, which coerces first).

Puzzle 8 ★★ – losing `this`

```
const o = { n: 'X', get() { return this.n; } };
const f = o.get;
console.log(f());           // ?
console.log(f.call(o));     // ?
```

Output: `undefined`, then `X`

ఎందుకు: Extracting the method breaks the implicit binding – `f()` is now a plain call, so `this` is `globalThis` (sloppy mode) and `globalThis.n` doesn't exist. **In strict mode / ES modules `this` would be `undefined` and it would throw a `TypeError` instead.** `f.call(o)` restores the binding explicitly.

ఇదే React class components లో `this.handleClick = this.handleClick.bind(this)` ఎందుకు రాయాల్సి వచ్చిందో దానికి కారణం.

Puzzle 9 ★★ – full ordering (hard one)

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve()
  .then(() => { console.log('3'); return Promise.resolve(); })
  .then(() => console.log('4'));
queueMicrotask(() => console.log('5'));
console.log('6');
```

Output: 1 6 3 5 4 2

ఎందుకు:

- 1, 6 – synchronous.
- Microtask queue drains: 3 runs first (it was queued first), 5 next.
- 4 comes **after** 5 because returning a promise from `.then` costs extra microtask ticks before the next `.then` is scheduled.
- 2 last – it's a macrotask, and macrotasks only run after the microtask queue is empty.

ఇది **hard**. Infosys లో ఇంత deep గా అడగకపోవచ్చు, కానీ వస్తే నువ్వు ready.

Puzzle 10 ★★ – the `parseInt` classic

```
console.log([1, 2, 3].map(String)); // ['1','2','3']
console.log(['1','2','3'].map(Number)); // [1, 2, 3]
console.log(['10','10','10'].map(parseInt)); // [10, NaN, 2] ← ఇదే trap
console.log([1,2,3] + [4,5]); // '1,2,34,5'
```

ఎందుకు `[10, NaN, 2]`: `map` passes **three** arguments to the callback – (`element`, `index`, `array`). `parseInt` takes (`string`, `radix`), so it receives the index as the radix:

- `parseInt('10', 0)` → radix 0 is treated as 10 → **10**
- `parseInt('10', 1)` → radix 1 is invalid → **NaN**
- `parseInt('10', 2)` → binary "10" → **2**

Fix: `arr.map(s => parseInt(s, 10))` – always pass the radix, and don't pass a multi-arg function directly to `map`.

`[1,2,3] + [4,5]`: `+` on two objects calls `toString()` on both, which joins with commas – `'1,2,3' + '4,5'`.

Puzzle 11 ★★ – reference vs value (React 5 critical)

```
let x = { v: 1 };
let y = x;
y.v = 2;
console.log(x.v);           // 2 ← same object

const src = { a: 1, nested: { b: 2 } };
const copy = { ...src };
copy.nested.b = 99;
console.log(src.nested.b);  // 2 ← spread is SHALLOW
```

ఎందుకు: Objects are held by reference. `y = x` copies the reference, not the object. Spread makes a **shallow** copy – the top level is new, but `nested` is still the same object in both.

React connection: ఇదే reason ఎందుకు nested state update చేసేటప్పుడు ప్రతి level ని spread చెయ్యాలి:

```
setState(prev => ({ ...prev, nested: { ...prev.nested, b: 99 } }));
```

ఒక level మాత్రమే spread చేస్తే original mutate అవుతుంది → React re-render trigger చెయ్యదు → "state మారింది కానీ UI update అవ్వట్లేదు" bug. **ఇది Infosys interview లో అడిగే classic React bug.**

Puzzle 12 ★★★ – async/await ordering

```
async function f1() { console.log('f1 start'); await f2(); console.log('after await'); }
async function f2() { console.log('f2'); }

console.log('script start');
f1();
Promise.resolve().then(() => console.log('microtask'));
console.log('script end');
```

Output: script start · f1 start · f2 · script end · after await · microtask

ఎందుకు: An `async` function runs **synchronously until it hits the first `await`**. So `f1 start` and `f2` print immediately. At the `await`, the rest of `f1` is queued as a microtask – and because that happened **before** the `.then` line executed, `after await` is queued first and therefore runs first.

Mental model: `await X` ≈ "run everything up to here now, then wrap the rest in `X.then(rest)`".

Gotchas (సాధారణ తప్పులు)

1. **Output చెప్పి ఆగిపోవడం.** ఎప్పుడూ "ఎందుకు" జోడించు. Interviewer output కంటే reasoning కి ఎక్కువ marks ఇస్తాడు.
2. **తొందరపడటం.** 10 seconds ఆలోచించు. Loud గా trace చెయ్యి: "Okay, these two lines are synchronous, so those print first..."
3. **Sloppy vs strict mode తేడా మర్చిపోవడం** (Puzzle 8). Modern React code ES modules – always strict.
4. **Number.isNaN బదులు global isNaN వాడటం.** `isNaN('abc') → true` (coerces), `Number.isNaN('abc') → false`. **Number.isNaN వాడు.**

Key Points

- **Sync → microtasks → macrotasks** – ఈ ఒక్క rule 5 puzzles solve చేస్తుంది.
- `var` = one shared binding; `let` = one per iteration.
- Arrow functions కి own `this` లేదు.
- Spread/ `Object.assign` = **shallow**.
- `map(parseInt)` = ఎప్పుడూ trap.
- **Reasoning > answer.**

Interview దృష్టి

Puzzle ఇచ్చినప్పుడు ఇలా చెయ్యి:

1. Code ని **loud గా చదువు** ("so we have a for loop with var...")
2. Sync lines identify చెయ్యి
3. Microtasks ఏవి, macrotasks ఏవి చెప్పు
4. తర్వాత output చెప్పు
5. తప్పుతే – "Let me re-trace that" – recover అవ్వచ్చు, **panic అవ్వకు**

12. Polyfills – చేతితో రాయాల్సినవి

వివరణ

"Implement `map` without using `map`" / "Write your own debounce" – ఇవి Infosys live coding లో వచ్చే **అత్యంత common tasks**. కారణం: ఇవి నీకు `this`, closures, `apply` నిజంగా అర్థమయ్యాయా అని ఒకసారి test చేస్తాయి.

✅ ఈ section లోని ప్రతి implementation `node` లో assertions తో test చేయబడింది – అన్నీ pass అయ్యాయి.

Priority order (time తక్కువ ఉంటే ఈ order లో నేర్చుకో): **debounce → throttle → deepClone → reduce → bind → Promise.all → flatten → memoize**

1. map polyfill ★★

```
Array.prototype.myMap = function (cb, thisArg) {
  if (typeof cb !== 'function') throw new TypeError(cb + ' is not a function');
  const out = new Array(this.length);
  for (let i = 0; i < this.length; i++) {
    if (i in this) out[i] = cb.call(thisArg, this[i], i, this); // sparse arrays
  }
  return out;
};

[1, 2, 3].myMap(x => x * 2); // [2, 4, 6] ✓
```

Interviewer చూసేవి: (a) callback కి **మూడు arguments** పంపావా (element, index, array), (b) **thisArg** support చేశావా, (c) **i in this** తో **sparse arrays** (holes) handle చేశావా. ఈ మూడూ చేస్తే **senior signal**.

2. filter polyfill ★★

```
Array.prototype.myFilter = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this && cb.call(thisArg, this[i], i, this)) out.push(this[i]);
  }
  return out;
};

[1, 2, 3, 4].myFilter(x => x % 2 === 0); // [2, 4] ✓
```

3. reduce polyfill ★★★ (ఇదే most-asked)

```
Array.prototype.myReduce = function (cb, initial) {
  let acc = initial, start = 0;

  if (arguments.length < 2) { // ⚠ initial value ఇవ్వలేదా?
    if (this.length === 0) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[0]; // మొదటి element ని accumulator గా వాడు
    start = 1;
  }
  for (let i = start; i < this.length; i++) acc = cb(acc, this[i], i, this);
  return acc;
};

[1,2,3,4].myReduce((a, b) => a + b); // 10 ✓
[1,2,3,4].myReduce((a, b) => a + b, 10); // 20 ✓
[].myReduce((a, b) => a + b); // TypeError ✓
```

ఇక్కడే 90% మంది తప్పు చేస్తారు: `if (!initial)` అని check చేస్తారు. అది bug – ఎవరైనా `0`, `''`, `false` ని initial value గా ఇస్తే అది falsy కాబట్టి ignore అవుతుంది. `arguments.length < 2` వాడాలి. ఈ ఒక్క detail చెప్తే interviewer గుర్తుపెట్టుకుంటాడు.

4. bind polyfill ★★

```
Function.prototype.myBind = function (thisArg, ...boundArgs) {
  const fn = this;
  return function (...callArgs) {
    // `new` తో పిలిస్తే thisArg ని ignore చేసి, కొత్త object ని వాడాలి
    return fn.apply(this instanceof fn ? this : thisArg, [...boundArgs, ...callArgs]);
  };
};

const o = { n: 'infy' };
function greet(g, p) { return g + ' ' + this.n + p; }
greet.myBind(o, 'hi')('!'); // 'hi infy!' ✓
```

Key details: (a) **partial application** – bind time args + call time args merge అవ్వాలి, (b) `this instanceof fn` check – bound function ని `new` తో పిలిస్తే `thisArg` ని ignore చెయ్యాలి. ఈ రెండో point చెప్తే నువ్వు spec చదివావు అని అర్థం.

5. call polyfill ★★

```
Function.prototype.myCall = function (thisArg, ...args) {
  thisArg = thisArg ?? globalThis;
  const key = Symbol('fn'); // Symbol → existing property ని overwrite చెయ్యదు
  thisArg[key] = this; // function ని object మీద temporary గా పెట్టు
  const result = thisArg[key](...args); // implicit binding → this = thisArg
  delete thisArg[key]; // cleanup
  return result;
};

greet.myCall(o, 'yo', '?'); // 'yo infy?' ✓
```

Trick: `Symbol` ఎందుకు? String key (`thisArg.fn = this`) వాడితే object లో అదే పేరుతో property ఉంటే overwrite అవుతుంది. **Symbol collision-free.**

6. Promise.all polyfill ★★★

```
function myPromiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = new Array(promises.length);
    let completed = 0;
```

```

    if (promises.length === 0) return resolve([]); // ⚠️ empty array edge case

    promises.forEach((p, i) => {
      Promise.resolve(p).then(
        allow
        value => {
          results[i] = value; // ⚠️ index తో - order preserve
        },
        reject // ఏ ఒక్కటి fail అయినా వెంటనే
      );
    });
  });
}

await myPromiseAll([1, Promise.resolve(2), 3]); // [1, 2, 3] ✅
await myPromiseAll([]); // [] ✅

```

మూడు edge cases interviewer చూస్తాడు: (1) empty array → వెంటనే resolve, (2)

`Promise.resolve(p)` wrap – plain values కూడా పని చెయ్యాలి, (3) `results[i]` – **completion order కాదు, input order** లో results రావాలి.

Follow-up: "allSettled ఎలా మార్చావు?" → reject handler లో `reject` బదులు `results[i] = {status:'rejected', reason}` పెట్టి counter increment చెయ్యి. అంతే.

7. debounce ★★★★★ (highest priority)

```

function debounce(fn, delay) {
  let timer;
  function debounced(...args) {
    clearTimeout(timer); // ప్రతి call కి పాత timer cancel
    timer = setTimeout(() => fn.apply(this, args), delay);
  }
  debounced.cancel = () => clearTimeout(timer); // cleanup కి (React unmount)
  return debounced;
}

// verified: వరుసగా 3 calls → 1 execution మాత్రమే ✅

```

⚠️ **function వాడాను, arrow కాదు** – ఎందుకంటే `this` ని caller నుండి forward చెయ్యాలి. Arrow అయితే `this` lexical అవుతుంది, DOM handler గా వాడితే తప్పు.

.cancel ఎందుకు: React లో `useEffect` cleanup లో దీన్ని పిలవాలి, లేకపోతే component unmount అయ్యాక timer fire అయి "setState on unmounted component" leak వస్తుంది.

ఇది mention చేస్తే senior signal.

React లో వాడకో:

```
const debouncedSearch = useMemo(() => debounce(onSearch, 300), [onSearch]);
useEffect(() => debouncedSearch.cancel, [debouncedSearch]); // cleanup
```

8. throttle ★★★★★

```
function throttle(fn, limit) {
  let lastCall = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastCall >= limit) {
      lastCall = now;
      fn.apply(this, args);
    }
  };
}
```

// verified: వరుసగా 3 calls → 1 execution; 60ms తర్వాత మళ్ళీ call → 2వ execution ✓

Debounce vs throttle ఒక్క వాక్యంలో: *Debounce waits for silence; throttle enforces a maximum rate.* Search input → debounce. Scroll/resize → throttle.

9. deepClone ★★★★★

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value; // primitives
  if (seen.has(value)) return seen.get(value); // ⚠ circular
  refs

  if (value instanceof Date) return new Date(value);
  if (value instanceof RegExp) return new RegExp(value.source, value.flags);

  const out = Array.isArray(value) ? [] :
Object.create(Object.getPrototypeOf(value));
  seen.set(value, out); // recurse ముందు
  register

  for (const key of Reflect.ownKeys(value)) { // symbols కూడా
    out[key] = deepClone(value[key], seen);
  }
  return out;
}

// verified: nested mutation isolated ✓ · circular ref handled ✓ · Date preserved ✓
```

ఎందుకు WeakMap : circular references (`obj.self = obj`) – లేకపోతే infinite recursion → stack overflow. WeakMap వాడితే garbage collection block అవ్వదు.

Interviewer follow-up: "JSON.parse(JSON.stringify()) ఎందుకు వాడకూడదు?"

"It loses `undefined` values and functions entirely, converts `Date` objects into strings, drops `Map` / `Set`, and throws on circular references. It's fine for plain JSON-shaped data. For anything else, modern runtimes have `structuredClone()` built in – that's what I'd reach for in production; I'd only hand-roll this if I needed custom behaviour."

10. flatten ★★

```
function myFlat(arr, depth = 1) {
  return arr.reduce((acc, v) =>
    Array.isArray(v) && depth > 0
      ? acc.concat(myFlat(v, depth - 1))
      : acc.concat([v]), // ⚠ [v] – concat(v) అయితే array ని spread చేస్తుంది!
    []);
}

myFlat([1, [2, [3, [4]]]], Infinity); // [1, 2, 3, 4] ✓
myFlat([1, [2, [3, [4]]]]);           // [1, 2, [3, [4]]] ✓ (default depth = 1)
```

⚠ నేను ఈ code రాసేటప్పుడు ఇక్కడే bug వచ్చింది – `acc.concat(v)` రాశాను, కానీ `concat` ఒక array ని argument గా ఇస్తే దాన్ని ఒక level spread చేస్తుంది. అంటే depth limit పని చెయ్యదు. `acc.concat([v])` రాయాలి. **Test రాయకపోతే ఇది పట్టుబడేది కాదు** – interview లో కూడా ఇదే trap.

11. once మరియు memoize ★★

```
function once(fn) {
  let called = false, result;
  return function (...args) {
    if (!called) { called = true; result = fn.apply(this, args); }
    return result; // ⚠ తర్వాత calls కి cached result తిరిగి ఇవ్వాలి
  };
}

function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args); // ⚠ limitation: objects, functions కి పని చెయ్యదు
    if (cache.has(key)) return cache.get(key);
    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}
```

// verified: once → underlying fn 1 సారి మాత్రమే call ✓ · memoize → cache hit ✓

Memoize follow-up: "Cache ఎప్పటికీ పెరుగుతూ పోతుంది కదా?" → "Yes, that's an unbounded cache and a memory leak in a long-running app. In production I'd bound it – an LRU with a max size, or a `WeakMap` when the key is an object so entries can be collected."

Gotchas (సాధారణ తప్పులు)

1. **Callback కి index, array పంపకపోవడం.** `cb(this[i])` కాదు, `cb.call(thisArg, this[i], i, this)`.
2. **reduce లో if (!initial).** `arguments.length < 2` వాడు.
3. **Debounce/throttle లో arrow function వాడటం.** `this` forward అవ్వదు.
4. **Promise.all లో empty array handle చెయ్యకపోవడం** – infinite pending promise.
5. **concat array ని spread చేస్తుంది** అని మర్చిపోవడం (flatten bug).
6. **Cleanup mention చెయ్యకపోవడం** – `.cancel`, `clearTimeout`, `WeakMap`. ఇవే junior/senior తేడా.

Key Points

- Priority: **debounce** → **throttle** → **deepClone** → **reduce** → **bind** → **Promise.all**.
- ప్రతి polyfill లో **edge cases loud** గా చెప్పు – empty array, no initial value, circular refs, `this` binding.
- **Production లో ఏం వాడతావో చెప్పు** (`structuredClone`, `lodash`) – "నేను ఇది hand-roll చెయ్యను, కానీ ఎలా పని చేస్తుందో తెలుసు" అనేది perfect answer.
- **Cleanup/memory గురించి మాట్లాడు** – ఇది 3 YOE ని 5 YOE లా చూపిస్తుంది.

Interview దృష్టి

Polyfill అడిగినప్పుడు ఈ order follow చెయ్యి:

1. **"Let me clarify the signature first"** – arguments ఏవి, return ఏమిటి
2. Naive version రాయ (working code first)
3. **"Now let me handle the edge cases"** – empty input, `this`, index args
4. **"In production I'd use X"** – ఇది maturity చూపిస్తుంది

Working code + edge case awareness > perfect code silently written.

13. React question bank – Infosys-specific, 40 questions

వివరణ

ఈ list Infosys కి ప్రత్యేకంగా report చేయబడిన questions నుండి తీసుకున్నది (candidate experiences + Infosys-specific question banks). వీటిని **మూడు levels** గా విభజించాను – నువ్వు 3 YOE కాబట్టి **Level 1 అన్నీ instant గా రావాలి**, Level 2 confident గా, Level 3 లో కనీసం సగం.

Depth calibration: Infosys React questions **conceptual + practical**. వాళ్ళు "React Fiber లో work loop ఎలా పని చేస్తుంది?" అని అడగరు. కానీ "ఈ list slow గా ఉంది, ఎలా fix చేస్తావు?" అని అడుగుతారు. **Theory కంటే application.**

Level 1 – Fundamentals (ఇవి 100% వస్తాయి, instant గా రావాలి)

R1. React అంటే ఏమిటి? ఇతర frameworks కంటే advantage ఏమిటి?

A library for building user interfaces through composable components. Its core ideas: **declarative** – you describe what the UI should look like for a given state and React figures out the DOM operations; **component-based** – small reusable pieces; and a **virtual DOM** with reconciliation so updates are batched and minimal. It's a library, not a framework – routing, state management and data fetching are your choice, which is why it composes well but needs more decisions up front than Angular.

R2. JSX అంటే ఏమిటి?

A syntax extension that lets you write HTML-like markup inside JavaScript. It isn't understood by browsers – Babel compiles it into `React.createElement(type, props, ...children)` calls (or `jsx()` calls with the modern runtime), which produce plain JavaScript objects describing the UI. **Follow-up వస్తుంది:** "Why must component names be capitalised?" → "Lowercase tags compile to string literals – `'div'` – which React treats as DOM elements. Capitalised names compile to a variable reference, so React knows it's a component and must be in scope."

R3. Virtual DOM అంటే ఏమిటి? ఎలా పని చేస్తుంది?

A lightweight JavaScript object tree that mirrors the real DOM. When state changes, React builds a new virtual tree, **diffs** it against the previous one (reconciliation), computes the minimum set of changes, and applies them to the real DOM in one batch. **ఎందుకు వేగం:** direct DOM manipulation triggers layout and paint each time, which is expensive. Diffing plain objects in memory is cheap, and batching means one DOM write instead of many. ⚠️ **నిజాయితీ point (senior signal):** "The virtual DOM isn't faster than perfectly hand-optimised DOM code – it's faster than the naive re-render-everything approach, and it buys you a declarative programming model without giving up reasonable performance."

R4. Props vs State?

Props	State
Parent నుండి వస్తాయి	Component లోపలే ఉంటుంది
Read-only (immutable)	<code>setState</code> /setter తో మార్చొచ్చు
Component ని configure చేస్తాయి	సమయంతో మారే data
Props మారితే re-render	State మారితే re-render
Function arguments లాంటివి	Local variables లాంటివి (renders మధ్య బతికేవి)

R5. Functional vs Class components?

Class components use ES6 classes, `this.state`, `this.setState` and lifecycle methods. Functional components are plain functions that use hooks. Since React 16.8, functional components can do everything classes could – and they're the recommended approach now: less boilerplate, no `this` confusion, and logic reuse through custom hooks instead of HOCs and render props. ⚠️ **Infosys legacy codebases లో class components ఉంటాయి** – lifecycle methods తెలియాలి. §R11 చూడు.

R6. `useState` మరియు `useEffect` explain చెయ్యి?

`useState` gives a function component a piece of state that survives re-renders, returning the current value and a setter. Two things worth knowing: updates are **asynchronous and batched**, so use the functional form `setCount(c => c + 1)` when the new value depends on the old one; and passing a function to `useState(() => expensive())` makes the initial computation lazy.

`useEffect` runs side effects after render – data fetching, subscriptions, manual DOM work. The dependency array controls when: `[]` runs once after mount, `[a, b]` runs whenever `a` or `b` change, and omitting it runs after **every** render. The optional returned function is the **cleanup**, which runs before the next effect and on unmount – that's where you remove listeners and clear timers.

R7. Keys ఎందుకు అవసరం?

Keys tell React which item in a list is which across renders, so it can match elements between the old and new trees instead of guessing. Without stable keys React falls back to position, and reordering or deleting causes it to update the wrong DOM nodes. ⚠️ **Index ని key గా వాడొచ్చా?** "Only when the list is static – never reordered, filtered, or added to in the middle. Otherwise it causes real bugs: if you delete the first item, every index shifts, so React thinks the content changed rather than that an item was removed. The classic symptom is an input's typed value jumping to the wrong row." – ఈ concrete bug example చెప్తే బాగా score చేస్తావు.

R8. React events ఎలా handle చేస్తుంది?

React wraps native events in a **SyntheticEvent** – a cross-browser normalised wrapper with the same API. React doesn't attach a listener to each element; it attaches them at the **root container** and uses event delegation (at `document` before React 17, at the root container from 17 onwards). That's why you call `e.preventDefault()` instead of returning `false`, and why `stopPropagation` on a React handler doesn't stop a native listener attached outside React.

R9. Controlled vs uncontrolled components?

In a **controlled** component the form value lives in React state and every keystroke goes through `onChange` – React is the single source of truth, which makes validation, conditional disabling and formatting straightforward. In an **uncontrolled** component the DOM holds the value and you read it with a `ref` when you need it – less code and fewer renders, useful for simple forms and file inputs. **Practical answer:** "I default to controlled because most real forms need live validation. For large forms where per-keystroke re-renders became a problem, I've used react-hook-form, which is uncontrolled under the hood and only re-renders on submit or error."

R10. Conditional rendering ఎలా చేస్తావు?

`&&` for simple show/hide, ternary for either/or, early `return null` to render nothing, and a lookup object or `switch` when there are several states. ⚠️ **&& గోతి:**
`{items.length && <List />}` renders a literal `0` on screen when the array is empty, because `0` is falsy but still a valid React child. Use `{items.length > 0 && <List />}`. ఇది Infosys favourite gotcha question.

R11. Lifecycle methods ఏవి? Hooks తో ఎలా map అవుతాయి?

Class	Functional (hooks)
constructor	useState initialiser
componentDidMount	useEffect(fn, [])
componentDidUpdate	useEffect(fn, [deps])
componentWillUnmount	useEffect(() => () => cleanup, []) ← return చేసే function
shouldComponentUpdate	React.memo
getDerivedStateFromError	Error boundary (ఇంకా class మాత్రమే)

Infosys ఇది exact గా అడిగింది: "How to use `componentWillUnmount()` in a functional component?"

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  const onResize = () => setWidth(window.innerWidth);
  window.addEventListener('resize', onResize);

  return () => {                                // ← ఇదే componentWillUnmount
    clearInterval(id);
    window.removeEventListener('resize', onResize);
  };
}, []);
```

జోడించు: "The cleanup doesn't only run on unmount – it also runs before every re-run of the effect, which is what prevents stale subscriptions when dependencies change."

R12. React Router దేనికి?

Client-side routing for single-page apps – it swaps components based on the URL without a full page reload, so navigation is instant and application state survives. Core pieces: `BrowserRouter`, `Routes / Route`, `Link` (instead of `<a>`, which would reload), `useNavigate` for programmatic navigation, `useParams` for route params, and nested routes with `Outlet`. Protected routes are usually a wrapper component that checks auth and redirects.

Level 2 – Intermediate (confident గా రావాలి)

R13. `useMemo` vs `useCallback` vs `React.memo` ?

All three are about avoiding unnecessary work. `useMemo(fn, deps)` caches a computed **value**. `useCallback(fn, deps)` caches a **function reference** – it's literally `useMemo(() => fn, deps)`. `React.memo(Component)` is a HOC that skips re-rendering when props are shallow-equal.

The connection matters: `React.memo` compares props shallowly, so passing an inline arrow function as a prop breaks it – a new reference every render means the memo never hits. `useCallback` is what makes `React.memo` actually work.

⚠️ **ఎప్పుడు వాడకూడదు (ఇది చెప్తే senior signal):** "These aren't free – they cost memory and a dependency comparison every render. Wrapping every function in `useCallback` makes code noisier and can be net slower. I reach for them when there's a measured problem: an expensive computation, a memoised child that's re-rendering, or a dependency of another hook."

R14. Custom hooks అంటే ఏమిటి?

A function starting with `use` that calls other hooks – the mechanism for sharing **stateful logic** (not UI) between components. Before hooks this needed HOCs or render props, which produced deeply nested "wrapper hell".

```
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id); // value మారితే పాత timer cancel
  }, [value, delay]);
  return debounced;
}
```

Important nuance: two components using the same custom hook get **separate state** – the hook shares logic, not data.

R15. Context API అంటే ఏమిటి? ఎప్పుడు వాడాలి?

A way to pass values down the tree without threading props through every intermediate component (prop drilling). `createContext`, a `Provider` at the top, and `useContext` wherever you need it. ⚠️ **Big caveat (Infosys ఇష్టపడే answer):** "Context re-renders **every consumer** whenever the provider's value changes – there's no selector, so consumers can't subscribe to just one slice. That makes it a good fit for low-frequency values like theme, locale or the auth user, and a poor fit for frequently-changing state. Two things I watch for: memoise the provider value so it isn't a new object every render, and split into multiple contexts rather than one giant one."

R16. HOC (Higher-Order Component) అంటే ఏమిటి?

A function that takes a component and returns an enhanced component – `withAuth(Dashboard)`. It was the pre-hooks pattern for cross-cutting concerns like auth checks, logging and data injection (`connect` in old Redux is the classic example). Downsides: wrapper nesting, prop name collisions, and losing static methods and refs unless you forward them. **Custom hooks replaced most HOC use cases**, though HOCs still make sense when you need to wrap the rendering itself, not just the logic.

R17. Error boundaries?

A class component implementing `static getDerivedStateFromError` (to render a fallback) and/or `componentDidCatch` (to log). It catches render-phase errors anywhere in its subtree and shows a fallback instead of unmounting the whole app. ⚠️ **ఏం catch చెయ్యదు:** event handler errors, async code (`setTimeout` , promises), server-side rendering, and errors thrown in the boundary itself. Those need normal `try/catch`. ⚠️ **Still class-only** – there's no hook equivalent. In practice most teams use `react-error-boundary`.

R18. React లో performance ఎలా optimise చేస్తావు? – ★ Infosys ఇది తప్పకుండా అడుగుతుంది

ఇది **checklist** గా చెప్పు, ఒక్క technique కాదు:

1. **Measure first** – React DevTools Profiler to find what's actually re-rendering, and Lighthouse for load performance. Optimising without measuring is guessing.
2. **Cut wasted renders** – `React.memo` on expensive children, `useMemo` / `useCallback` for the values and callbacks they depend on, and correct `key` usage.
3. **Code splitting** – `React.lazy` + `Suspense`, route-based first, then heavy components like charts and editors.
4. **Virtualisation** – `react-window` / `react-virtuoso` for long lists; render 20 rows instead of 8,000.
5. **Debounce expensive handlers** – search inputs, resize, scroll.
6. **State placement** – keep state as local as possible; lifting it too high re-renders half the tree.
7. **Bundle size** – analyse with `source-map-explorer`, avoid importing whole libraries (`import debounce from 'lodash/debounce'`, not the full lodash), tree-shaking.
8. **Assets** – image optimisation, lazy loading below the fold, correct caching headers.

Then anchor it: "In my project the biggest single win was virtualisation – the report grid went from 6 seconds to under 1."

R19. Lazy loading మరియు code splitting? – Infosys ఇది exact గా అడిగింది

Code splitting breaks the bundle into chunks that load on demand instead of shipping everything up front. **Lazy loading** is loading a chunk at the moment it's needed. In React: `React.lazy(() => import('./Chart'))` wrapped in `<Suspense fallback={...}>`. The dynamic `import()` is what tells the bundler to create a separate chunk.

```
const Dashboard = React.lazy(() => import('./Dashboard'));

<Suspense fallback=<Spinner />>
  <Route path="/dashboard" element=<Dashboard /> />
</Suspense>
```

Where to split: routes first (biggest win, easiest), then heavy libraries (charting, rich text editors, PDF viewers), then modals and tabs that most users never open. **Pair it with an error boundary** – a chunk can fail to load on a flaky network, and without a boundary that's a white screen.

R20. `useRef` దేనికి?

Two distinct uses. First, **accessing DOM nodes** – focus, scroll, measure, integrate a non-React library. Second, a **mutable box that survives renders without causing one** – timer IDs, previous values, "has this mounted yet" flags. **The key property:** changing `ref.current` does **not** trigger a re-render. That's the whole point – it's the escape hatch for values you need to persist but that shouldn't drive the UI. ⚠️ **Anti-pattern:** using a ref to hold something the UI displays. If it's rendered, it belongs in state.

R21. Async operations React లో ఎలా handle చేస్తావు?

For anything server-related I'd reach for **TanStack Query** (or RTK Query) rather than hand-rolling – you get caching, deduplication, background refetch, retries and loading/error states out of the box, and it removes most of the boilerplate. If it's hand-rolled, the shape is: `useEffect` with a **cleanup that cancels**, because otherwise a slow first request can resolve after a fast second one and overwrite fresh data with stale data – the classic race condition.

```
useEffect(() => {
  const controller = new AbortController();
  let active = true;

  (async () => {
    try {
      setLoading(true);
      const res = await fetch(url, { signal: controller.signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`); // ⚠️ fetch 404 కి
      throw చెయ్యదు!
      const json = await res.json();
      if (active) setData(json);
    } catch (e) {
      if (active && e.name !== 'AbortError') setError(e);
    } finally {
      if (active) setLoading(false);
    }
  })();

  return () => { active = false; controller.abort(); }; // ← race condition
  fix
}, [url]);
```

⚠️ ఈ `res.ok` check చాలా ముఖ్యం: `fetch` only rejects on network failure – a 404 or 500 still resolves. Axios throws on non-2xx by default; `fetch` doesn't. ఇది Infosys అడిగే follow-up.

R22. Fetch vs Axios?

`fetch` is built in, no dependency, but it needs manual JSON parsing, doesn't throw on HTTP error status, has no built-in timeout, and no interceptors. Axios gives automatic JSON transforms, throws on non-2xx, supports request/response **interceptors** (which is how you attach auth tokens and handle 401 refresh centrally), timeouts, and upload progress. **Practical answer:** "For a real app I prefer Axios mainly for interceptors – having token refresh and error normalisation in one place rather than at every call site. For something small, `fetch` with a thin wrapper is fine."

R23. Sibling components మధ్య communication ఎలా? – Infosys ఇది అడిగింది

The standard answer is **lifting state up**: move the shared state to the closest common parent, pass the value down to one sibling as a prop and a setter down to the other. The siblings never talk directly – the parent owns the state. **When that gets painful** – the common parent is many levels up, so you'd be prop-drilling through components that don't care – the options are Context for low-frequency shared values, or a store (Redux/Zustand) for anything frequently updated or genuinely global. **అనకూడనిది**: an event bus or direct refs between siblings. Interviewers read that as fighting the framework.

R24. Large state ని ఎలా manage చేస్తావు?

I'd start by **classifying the state**, because different kinds want different tools:

- **Server state** (API data) → TanStack Query / RTK Query – caching and invalidation are the actual problem here
- **Global client state** (auth, cart, permissions) → Redux Toolkit or Zustand
- **Low-frequency global values** (theme, locale) → Context
- **Local UI state** (modal open, input value) → `useState` right where it's used
- **URL state** (filters, pagination, tab) → the URL itself, via search params – free shareability and back-button support

"The most common mistake I see is putting everything in one global store, including things only one component uses. That makes every change a potential global re-render and makes the store impossible to reason about." **ఈ classification answer చాలా బాగా score చేస్తుంది** – చాలామంది "Redux వాడతాను" అని మాత్రమే చెప్తారు.

Level 3 – Experienced (కనీసం సగం రావాలి)

R25. Reconciliation algorithm ఎలా పని చేస్తుంది?

A full tree diff would be $O(n^3)$, which is unusable, so React uses a heuristic $O(n)$ algorithm built on two assumptions: (1) **elements of different types produce different trees** – if a `<div>` becomes a `<span>`, React tears down the whole subtree and rebuilds it rather than diffing; (2) **keys identify which children are stable across renders**. Same type means React keeps the DOM node and just updates the changed attributes, then recurses into children. **Practical consequence**: conditionally swapping a component's type unmounts it and loses its state. That's why `{isEdit ? <Input/> : <Display/>}` resets state, but rendering the same type with different props doesn't.

R26. React Fiber అంటే ఏమిటి?

The rewrite of React's core in v16. Before Fiber, rendering was a synchronous recursive walk that couldn't be interrupted – a large update blocked the main thread and dropped frames. Fiber turned the tree into a **linked list of units of work** that React can pause, resume, prioritise and abandon. That unlocked concurrent features: `useTransition` for marking updates as non-urgent, `useDeferredValue`, `Suspense`, and automatic batching in React 18. **Two phases:** the **render phase** (building the work-in-progress tree – interruptible, must be pure, which is why StrictMode double-invokes to catch side effects) and the **commit phase** (applying to the DOM – synchronous and uninterruptible). ⚠️ **Infosys లో ఇది అరుదు.** వస్తే bonus. Deep dive → `React_Telugu.md`.

R27. React 18 లో ఏం మారింది?

Automatic batching – before 18, React only batched updates inside its own event handlers; updates inside promises, `setTimeout` and native handlers each caused a separate render. Now everything batches. **Concurrent features** – `useTransition`, `useDeferredValue`. `createRoot` replaced `ReactDOM.render`. **StrictMode double-invokes effects** in development to surface missing cleanup. Plus streaming SSR with `Suspense`.

R28. Components ని reusable గా ఎలా చేస్తావు?

Composition over configuration. Concretely: accept `children` and use **compound components** rather than a wall of boolean props (`<Modal><Modal.Header><Modal.Body></Modal>`); expose an `as / asChild` prop for the underlying element; **spread the rest props** and forward the ref so the component behaves like a real DOM element; extract behaviour into a custom hook so the logic is reusable independently of the markup; and keep the component **presentational** – no data fetching inside a reusable UI component. **Red flag I watch for:** a component with 15 boolean props (`isPrimary`, `isSmall`, `hasIcon`, `isLoading` ...). That usually means it should be several components, or should accept children.

R29. Accessibility (a11y) ఎలా ensure చేస్తావు?

Semantic HTML first – a real `<button>` gives you keyboard activation, focus and screen reader semantics for free, where a clickable `<div>` gives you none of it. Beyond that: labels tied to inputs, alt text, ARIA only where semantics genuinely fall short, visible focus indicators, keyboard navigation and focus trapping in modals, colour contrast at 4.5:1, and `aria-live` for async status messages. Tooling: `eslint-plugin-jsx-a11y`, axe DevTools, and actually tabbing through the page. **Infosys context:** enterprise and government clients often have **WCAG 2.1 AA contractual requirements** – mentioning that shows you understand why it matters commercially, not just technically.

R30. React app ని security threats నుండి ఎలా protect చేస్తావు?

React escapes interpolated values by default, so `{userInput}` can't inject markup – that covers most XSS. The risks that remain: `dangerouslySetInnerHTML` (sanitise with DOMPurify if you must use it), `javascript: URLs` in `href` from user data, tokens in `localStorage` (prefer httpOnly cookies), secrets in the frontend bundle (anything in `REACT_APP_*` is public), and dependency vulnerabilities (`npm audit`). Plus the things the server must do: CSP headers, CSRF tokens or SameSite cookies, HTTPS, and **validating on the server** – client-side validation is UX, never security. Deep dive → `Security_Telugu.md`.

R31. React app ని ఎలా debug చేస్తావు?

React DevTools for the component tree, props and state; the **Profiler** tab to see what re-rendered and why. For network issues, the Network tab and Redux DevTools for action/state history with time-travel. In production, source maps plus an error tracker like Sentry so I get real stack traces and the user's action breadcrumbs. **A specific technique worth mentioning:** for "why did this re-render?", the Profiler's "why did this render" data, or a `usePrevious` hook diffing props, beats guessing.

R32. Complex forms ఎలా handle చేస్తావు?

For anything beyond a couple of fields I use **react-hook-form with a Zod (or Yup) schema**. The reasons: it's uncontrolled internally so it doesn't re-render on every keystroke, the schema gives me one source of truth for validation that I can **share with the backend**, and it handles field arrays, nested objects and multi-step wizards without custom plumbing. Hand-rolling gets painful fast once you need per-field async validation, dirty/touched tracking and submission state.

R33. Deployment మరియు CI/CD ఎలా handle చేస్తావు?

Git flow with feature branches and PR review; CI on every PR running lint, type-check, tests and a build. On merge to main, build and deploy – for a static React app that's an S3 + CloudFront upload with a cache invalidation, or a platform like Vercel/Netlify. Environment config through build-time env vars per environment, never committed secrets. Long-cached hashed asset filenames with a short-cached `index.html`. Plus error tracking and a rollback path.

R34. Version control (Git) ఎలా manage చేస్తావు? → §17 చూడు.

Gotchas (సాధారణ తప్పులు)

1. **"Virtual DOM is faster than real DOM"** అని flat గా చెప్పడం. **Nuanced answer ఇవ్వు** (R3) – interviewers ఈ nuance గమనిస్తారు.
2. **"React is a framework"** అనడం. **Library.**
3. **useEffect cleanup mention చెయ్యకపోవడం.** ప్రతి effect answer లో cleanup mention చెయ్యి.
4. **useMemo / useCallback ని "always use for performance" అనడం.** ఎప్పుడు వాడకూడదో చెప్పడమే senior signal.
5. **Performance question కి ఒక్క technique చెప్పడం. Checklist ఇవ్వు** (R18), measure-first తో మొదలుపెట్టు.
6. **Index as key ని "never use" అనడం. "Only for static lists" అనేది సరైన, nuanced answer.**

Key Points

- Level 1 (R1-R12) = **100% probability**, instant recall.
- **R18 (performance)** మరియు **R24 (state management)** = అత్యధిక marks ఇచ్చే questions. వీటిని బాగా prepare చెయ్యి.
- ప్రతి answer లో **"my project లో ఇలా వాడాను"** జోడించు.
- **Nuance = seniority.** "Always X" కాదు, "X when Y, because Z".
- Deep dive → `React_Telugu.md` (63 topics).

Interview దృష్టి

Infosys interviewer ఒక concept అడిగినప్పుడు, **అతను అడగకముందే** trade-off చెప్పు:

Q: "What is Context API?" A: "[definition]... **The thing to be careful about is** that it re-renders every consumer, so I use it for theme and auth rather than for high-frequency state."

ఈ **"the thing to be careful about is"** phrase నీ interview లో 3-4 సార్లు వాడు. ఇది నువ్వు production లో పని చేశావని చూపిస్తుంది.

14. React scenario questions – నిజంగా అడిగినవి

వివరణ

Level 3 questions కంటే ఇవి ఎక్కువ ముఖ్యం. Infosys interviewers theory కంటే "ఈ situation లో నువ్వు ఏం చేస్తావు?" అని అడగడానికి ఇష్టపడతారు – ఎందుకంటే client project లో అదే పని.

Answer format: ఎప్పుడూ ఇలా → **clarify** → **options** → **నీ choice** + **ఎందుకు** → **trade-off**.

S1 ★★★★★ "A list of 10,000 items is rendering slowly. How do you fix it?"

"First I'd confirm where the time is actually going with the Profiler – is it the initial mount, or a re-render on every keystroke? Those need different fixes.

If it's the sheer number of DOM nodes, the fix is **virtualisation** – `react-window` or `react-virtuoso` render only the ~20 rows in the viewport plus a small buffer, so the DOM stays small no matter how big the list. That's usually the single biggest win.

Alongside that: **pagination or infinite scroll** so we don't fetch 10,000 records at once; `React.memo` **on the row component** with `useCallback` for the handlers passed into it, otherwise every parent render re-renders all rows; **stable keys** from the record ID, not the index; and **debouncing the search input** so filtering doesn't run per keystroke.

If the filter itself is expensive I'd wrap it in `useMemo` keyed on the query and the source data.

The trade-off with virtualisation is that Ctrl+F and native scroll-anchoring stop working the way users expect, and it complicates variable-height rows – so I'd only bring it in once the list is genuinely large."

ఇది ఎందుకు మంచి answer: measure-first, primary fix + supporting fixes, **trade-off** mention.

S2 ★★★★★ "How would you implement pagination or infinite scroll?" – Infosys ఇది అడిగింది

Pagination:

```
function useProducts() {
  const [page, setPage] = useState(1);
  const [data, setData] = useState({ items: [], total: 0 });
  const [loading, setLoading] = useState(false);
  const LIMIT = 20;

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    fetch(`/api/products?page=${page}&limit=${LIMIT}`, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error(`HTTP ${r.status}`); return r.json();
```

```

    })
    .then(setData)
    .catch(e => { if (e.name !== 'AbortError') console.error(e); })
    .finally(() => setLoading(false));
    return () => controller.abort(); // page వేగంగా మారితే race condition
    రాకూడదు
  }, [page]);

  return { ...data, page, setPage, loading, totalPages: Math.ceil(data.total / LIMIT)
};
}

```

Infinite scroll – IntersectionObserver తో (scroll listener కంటే మంచిది):

```

const sentinelRef = useRef(null);

useEffect(() => {
  const el = sentinelRef.current;
  if (!el || !hasMore || loading) return;

  const observer = new IntersectionObserver(
    ([entry]) => { if (entry.isIntersecting) setPage(p => p + 1); },
    { rootMargin: '200px' } // కిందికి రాకముందే load మొదలుపెట్టు
  );
  observer.observe(el);
  return () => observer.disconnect(); // ← cleanup ముఖ్యం
}, [hasMore, loading]);

// JSX: <div ref={sentinelRef} /> ← list చివర్లో

```

జేడించాల్సినవి (ఇవి చెప్తే బాగా score చేస్తావు):

- "IntersectionObserver over a scroll listener, because scroll fires constantly and needs throttling, while the observer is handled by the browser off the main thread."
- "Offset pagination breaks on live data – if a record is inserted while the user is paging, they see duplicates or skip records. For feeds I'd use **cursor-based pagination** instead."
- "The URL should hold the page number so the page is shareable and the back button works."
- "In production I'd use TanStack Query's useInfiniteQuery rather than hand-rolling this – it handles caching, dedupe and scroll restoration."

S3 ★★★★★ "The same payment functionality exists in our React web app. How do we reuse it in a mobile app without rewriting it?" – Infosys managerial round లో నిజంగా అడిగిన question

ఇది ఒక **architecture question**, coding question కాదు. Structured గా answer చెయ్యి:

"It depends on how much we want to share and what 'mobile app' means here. I'd frame it as three layers, because they're reusable to very different degrees.

The business logic layer is the most reusable – payment validation, amount calculation, the API client, retry logic, state machines for the payment flow. That's plain TypeScript with no DOM dependency, so I'd extract it into a **shared package** in a monorepo and both apps consume it. That's usually 60-70% of the actual complexity and it moves with zero rewriting.

The UI layer can't be shared directly – React DOM renders `<div>`, React Native renders `<View>`. But if the mobile app is React Native, we share the logic and the state management and rewrite only the presentation, which is the smaller part. If it's a native iOS/Android app, we can't share the UI at all.

For the fastest route with the least rewrite, there are two pragmatic options: host the payment flow as a **web view** the mobile app embeds – one implementation, one place to maintain PCI compliance, at the cost of a slightly less native feel; or use the payment provider's **native SDK** on mobile while sharing our backend API, which is what I'd actually recommend for payments specifically.

And the point I'd raise with the team: payments are exactly where duplicating logic is most dangerous – a rounding or currency bug that exists in one place and not the other is very hard to catch. So whatever we do with UI, the calculation and validation logic should live in exactly one shared module."

ఇది ఎందుకు excellent answer: layered thinking, multiple options with trade-offs, ఒక recommendation, మరియు **business risk** (PCI, currency bugs) mention – ఇది managerial round లో బంగారం.

S4 ★★ "Design a dynamic form that changes based on user input"

"I'd drive it from a **schema rather than hard-coded JSX** – an array of field definitions with type, name, label, validation rules, and a `showIf` condition referencing other fields' values. The renderer maps each field type to a component, and a `visibleFields` computation filters by the conditions against current values.

That gives you three things: the backend or a config can change the form without a deploy, validation lives with the field definition, and adding a new field type is one entry in the component map.

I'd build it with react-hook-form and a Zod schema generated from the same definitions, so validation and rendering can't drift apart. The main things to get right are **clearing the value of a field when it becomes hidden** – otherwise you submit data the user can't see – and keeping the conditions declarative so they stay testable."

S5 ★★ "How would you handle authentication in React?"

"The frontend's job is routing and UX; the security lives on the server.

On login, the server sets an **httpOnly, Secure, SameSite=Lax cookie** containing the session or refresh token. I prefer that over `localStorage` because JavaScript can't read an httpOnly cookie, which removes the XSS token-theft path – though it means we need CSRF protection, which SameSite plus a CSRF token handles.

In the app, an **AuthContext** holds the current user and loading state; a **ProtectedRoute** wrapper checks it and redirects to login with the attempted URL preserved so we can send them back after. Role-based UI is a permission check in the same context – but with the understanding that **hiding a button is UX, not security**; the server must authorise every request regardless.

For expiry, an **Axios response interceptor** catches a 401, calls the refresh endpoint once, and retries the queued requests – with a guard so parallel 401s don't fire multiple refreshes. If refresh fails, clear state and redirect to login."

⚠️ ఈ answer లో "hiding a button is UX, not security" అనే line చెప్పు – ఇది security maturity signal.

S6 ★★ "A third-party charting library needs to be integrated. How?"

"First check whether it has an official React wrapper – if so, use it. If it's an imperative library like D3 or Chart.js, the pattern is: a `useRef` for the container node, a `useEffect` that instantiates the chart on mount, a second effect that calls the library's update method when data changes rather than recreating it, and a **cleanup that calls `destroy()`** – that's the part people forget, and it leaks memory and leaves stale canvases on every remount.

I'd also wrap it in a `React.lazy` import, because charting libraries are heavy and most pages don't need them in the initial bundle. And I'd keep React out of the chart's DOM subtree – the library owns those nodes, React shouldn't try to reconcile them."

S7 ★★ "How do you handle API errors and loading states across the app?"

"I'd want it consistent rather than re-implemented per component. Three levels:

Transport level – an Axios interceptor that normalises every error into one shape, so components never deal with the difference between a network failure, a 500 with a JSON body, and a 500 with an HTML body. It also handles 401 refresh centrally.

Data level – TanStack Query gives every call `isLoading`, `isError` and `error` uniformly, plus retries with backoff for transient failures. Retrying a 500 makes sense; retrying a 400 doesn't, so the retry predicate matters.

UI level – a shared `<QueryBoundary>` that renders a skeleton while loading and an error state with a retry button on failure, so the pattern is one component rather than an `if (loading) ... if (error) ...` block in fifty files. Plus an error boundary above it for render-time crashes, and a toast for mutation failures.

One detail I care about: **skeletons that match the final layout**, so there's no layout shift when data arrives."

Gotchas (సాధారణ తప్పులు)

1. వెంటనే **code** రాయడం మొదలుపెట్టడం. ముందు **1-2 clarifying questions** అడుగు: "How large is the dataset?", "Is this data changing live?" – ఇది senior behaviour.
2. ఒక్క **solution** చెప్పడం. **2-3 options + నీ choice + ఎందుకు** ఇవ్వు.
3. **Trade-off** చెప్పకపోవడం. ప్రతి scenario answer చివర్లో **"the trade-off is..."** జోడించు.
4. **Library** పేరు మాత్రమే చెప్పడం. "I'd use react-window" సరిపోదు – **ఎందుకు, ఎలా, ఏం cost** చెప్పు.

Key Points

- Scenario questions = **theory కంటే ఎక్కువ weight** Infosys లో.
- Format: **clarify** → **options** → **choice + why** → **trade-off**.
- **S1, S2, S3 తప్పకుండా prepare చెయ్యి** – ఇవి Infosys లో నిజంగా అడిగినవి.
- ప్రతి answer లో **production reality** mention చెయ్యి ("in production I'd use X because...").

Interview దృష్టి

Scenario question వచ్చినప్పుడు, ఈ ఒక్క వాక్యంతో మొదలుపెట్టు:

"Let me ask a couple of clarifying questions first – how large is the dataset, and is this data updating in real time?"

ఇది నిన్ను వెంటనే **senior candidate** category లో పెడుతుంది. Junior candidates వెంటనే solution చెప్పడం మొదలుపెడతారు.

15. Redux & State Management

వివరణ

Infosys interviews లో **Redux ~80% probability**. కారణం: enterprise React codebases దాదాపు అన్నీ Redux మీద నడుస్తాయి, మరియు Infosys projects legacy-heavy.

⚠ **ముఖ్యం:** నీ project లో Redux వాడకపోతే, "నాకు Redux రాదు" అని చెప్పకు. Concepts నేర్చుకుని, ఒక చిన్న demo app కట్టు. ఇది **2 రోజుల పని**, కానీ interview లో 15 minutes block.

D1 ★★☆☆ "What is Redux? Why use it?"

"Redux is a predictable state container. The whole app state lives in a **single immutable store**, the only way to change it is by **dispatching an action** (a plain object describing what happened), and **reducers** are pure functions that take the current state and an action and return the next state.

The value isn't the global variable – it's the **constraints**. Because every change goes through a described action and a pure function, you get a full history of what happened, time-travel debugging, and state changes that are trivially testable. In a large team that traceability is worth more than the boilerplate costs.

When I wouldn't use it: a small app, or one where most 'state' is really server data. For server data I'd use TanStack Query or RTK Query, because the actual problem there is caching and invalidation, not state transitions."

D2 ★★☆☆ "Redux three principles?"

1. **Single source of truth** – one store for the whole app
2. **State is read-only** – the only way to change it is dispatching an action
3. **Changes are made by pure functions** – reducers must be pure: same input, same output, no mutation, no side effects, no API calls, no `Date.now()` or `Math.random()`

D3 ★★☆☆ "Redux data flow explain చెయ్యి"

```
UI event (button click)
  ↓ dispatch(action)
Action { type: 'cart/itemAdded', payload: { id: 42 } }
  ↓
Middleware (thunk / saga – async, logging ఇక్కడ)
  ↓
Reducer (pure) → (state, action) => newState
  ↓
```

Store updates



useSelector subscribed components re-render

One-line version: "Unidirectional" – dispatch an action, a middleware chain can intercept it, a pure reducer produces the next state, and subscribed components re-render. Data only ever flows one direction, which is what makes it debuggable."

D4 ★★☆☆ "Redux Toolkit ఎందుకు? Classic Redux తో సమస్య ఏమిటి?"

"Classic Redux needed a lot of ceremony for one feature – action type constants, action creators, a switch-based reducer, manual immutable spreading at every nesting level, plus store setup with middleware and devtools wiring. It was easy to get wrong and tedious to read.

Redux Toolkit is the official answer and it's what the Redux docs now recommend by default:

- `createSlice` generates action creators and types from the reducer names
- **Immer** is built in, so you write `state.items.push(item)` and it produces an immutable update under the hood – that removes the single biggest source of Redux bugs
- `configureStore` sets up thunk, devtools and dev-mode checks that catch accidental mutation
- `createAsyncThunk` handles the pending/fulfilled/rejected lifecycle
- **RTK Query** handles server state – caching, dedupe, invalidation

Same architecture, roughly a third of the code."

```
// Redux Toolkit – ఒక complete slice
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [], status: 'idle' },
  reducers: {
    itemAdded(state, action) {
      const existing = state.items.find(i => i.id === action.payload.id);
      if (existing) existing.qty += 1; // ⚠️ mutation లా కనిపిస్తుంది –
      else state.items.push({ ...action.payload, qty: 1 }); // Immer దీన్ని
      // immutable update గా మారుస్తుంది
    },
    itemRemoved(state, action) {
      state.items = state.items.filter(i => i.id !== action.payload);
    },
  },
});
```

```
export const { itemAdded, itemRemoved } = cartSlice.actions;
export default cartSlice.reducer;
```

⚠ **Immer గురించి follow-up ఖాయం:** "Isn't that mutating state?" → "It looks like it, but Immer gives the reducer a **draft proxy**. You mutate the draft, and Immer produces a new immutable state from the recorded changes. The actual store state is never mutated. The one rule is: **either mutate the draft or return a new value, never both in the same reducer.**"

D5 ★★★★★ "Redux vs Context API – ఎప్పుడు ఏది?" – ఇది చాలామంది ఖాయం

–	Context API	Redux (Toolkit)
ఏమిటి	Dependency injection mechanism	State management library
Re-render	Provider value మారితే అన్ని consumers	<code>useSelector</code> – subscribed slice మాత్రమే
Selectors	✗ లేవు	✓ ఉన్నాయి (memoised)
DevTools / time travel	✗	✓
Middleware (async, logging)	✗	✓
Boilerplate	తక్కువ	RTK తో మధ్యస్థం
ఎప్పుడు	Theme, locale, auth user – అరుదుగా మారేవి	Frequently-updating shared state, complex async, పెద్ద team

The line that scores: "Context is not a state management library – it's a way to avoid prop drilling. People compare them as alternatives, but they solve different problems. Context has no selector mechanism, so every consumer re-renders on any change to the provider value. That's fine for a theme; it's a performance problem for a cart that updates on every keystroke."

D6 ★★ "Middleware అంటే ఏమిటి? Thunk vs Saga?"

"Middleware sits between dispatching an action and the reducer receiving it, so it can intercept, delay, transform or log. That's where async lives, because reducers must stay pure.

Redux Thunk lets an action creator return a function instead of an object, so you can dispatch, await, and dispatch again. It's simple, it's what RTK includes by default, and it covers the vast majority of cases.

Redux Saga uses generator functions and describes side effects declaratively. It's much more powerful for complex orchestration – debouncing, cancellation, racing, retry loops, long-running background flows – and it's very testable because effects are plain objects. The cost is a real learning curve with generators.

My default is thunks, and I'd only reach for saga if we genuinely had complex cancellable workflows. For plain data fetching I'd skip both and use RTK Query."

D7 ★★ "useSelector re-render ఎప్పుడు trigger చేస్తుంది? Optimise ఎలా?"

"`useSelector` runs the selector after every dispatched action and compares the result to the previous one with **reference equality (===)** by default. So a selector returning a **new object or array each time** – like `state => state.items.filter(...)` or `state => ({ a, b })` – re-renders on every single action, even unrelated ones.

Three fixes: select **primitives** where possible and call `useSelector` several times; use `createSelector` from Reselect (built into RTK) to memoise derived data; or pass `shallowEqual` as the second argument when you must return an object."

```
// ❌ ప్రతి action కి re-render – కొత్త array reference
const activeUsers = useSelector(state => state.users.list.filter(u => u.active));

// ✅ memoised selector
const selectActiveUsers = createSelector(
  [state => state.users.list],
  list => list.filter(u => u.active) // inputs మారితేనే మళ్ళీ compute
);
const activeUsers = useSelector(selectActiveUsers);
```

ఈ answer చెప్పగలిగితే నువ్వు Redux ని నిజంగా production లో వాడవని interviewer కి తెలుస్తుంది.

D8 ★★ "Redux కి alternatives ఏవి?"

Zustand – minimal, hook-based, no provider needed, selector support built in; my default for new small-to-medium apps. **Jotai/Recoil** – atomic state, good when many independent small pieces of state need to be shared. **MobX** – observable/reactive, less boilerplate but more magic. **TanStack Query** – not a general state library, but it removes the need for most global state because most of it was server cache. **RTK Query** – the same idea inside Redux. "The industry direction is: server state to a query library, and only genuinely-client global state to a store. That's shrunk what Redux needs to hold considerably."

Gotchas (సాధారణ తప్పులు)

1. **Reducer లో mutation** (plain Redux లో). RTK లో Immer వల్ల ok, కానీ ఎందుకు ok అని explain చెయ్యగలగాలి.
2. **Reducer లో API call / side effect.** Reducers must be **pure** – middleware లో చెయ్యి.
3. **అన్నిటినీ Redux లో పెట్టడం.** Modal open/close, input value → local `useState`.
4. **useSelector లో object/array return చెయ్యడం** memoisation లేకుండా (D7).
5. **"Redux is dead" అనడం.** అది తప్పు – RTK actively maintained, enterprise లో ubiquitous. Infosys interviewer కి ఇది offensive గా కూడా అనిపించొచ్చు.

Key Points

- **RTK నేర్చుకో, classic Redux కాదు** – కానీ classic ఎందుకు మారిందో తెలియాలి.
- **D5 (Redux vs Context)** = దాదాపు ఖాయం. Table + "Context is not a state management library" line.
- **D7 (useSelector optimisation)** = senior signal.
- Redux రాకపోతే – **2 రోజుల్లో ఒక చిన్న cart app కట్టు** (RTK + async thunk).

Interview దృష్టి

"Have you used Redux?" అని అడిగినప్పుడు, నీ project లో లేకపోతే:

"In my current project we use Context plus TanStack Query, because most of our shared state turned out to be server cache. But I've worked with Redux Toolkit – I built [small project] with slices, async thunks and memoised selectors – and I understand where it earns its keep: when you need traceable state transitions and devtools across a large team."

నిజాయితీ + demonstrated knowledge = pass. బట్టీ కొట్టిన theory = follow-up లో పట్టుబడతావు.

16. HTML, CSS & Browser – underrated కానీ అడుగుతారు

వివరణ

Frontend role కి CSS questions ~40% probability. తక్కువే కానీ – ఇక్కడ fail అయితే "ఇతను React నేర్చుకున్నాడు కానీ fundamentals లేవు" అనే impression వస్తుంది. **2 hours prep చాలు.**

C1 ★★ Box model?

Every element is a box of four layers: **content** → **padding** → **border** → **margin**. With the default `box-sizing: content-box`, `width` applies to the content only, so padding and border add on top – a 200px-wide box with 20px padding renders 240px. With `box-sizing: border-box`, `width` includes padding and border, which is far more predictable. **Almost every project sets `*, *::before, *::after { box-sizing: border-box; }` in the reset** for exactly that reason.

C2 ★★★ Flexbox vs Grid?

Flexbox is one-dimensional – it lays out along a single axis (a row or a column) and is content-driven, so items size themselves and the container distributes the leftover space. Best for navbars, toolbars, centring, and distributing a row of cards.

Grid is two-dimensional – rows and columns at the same time, and it's layout-driven: you define the structure and place items into it, including overlapping ones. Best for page layouts, dashboards, and any card grid where you want consistent tracks.

They compose – a Grid page layout with Flexbox inside each cell is the normal combination, not an either/or.

C3 ★★ CSS specificity?

The cascade resolves conflicts by specificity, counted as (inline, IDs, classes/attributes/pseudo-classes, elements): inline style `1,0,0,0` > ID `0,1,0,0` > class `0,0,1,0` > element `0,0,0,1`. Equal specificity means the later rule wins.

`!important` overrides all of it – which is why it's a smell: it usually means the architecture lost, and the next person has to escalate with another `!important`.

C4 ★★ position values?

`static` (default, in normal flow), `relative` (offset from its normal position but still occupies its space – mainly used to create a positioning context), `absolute` (removed from flow, positioned against the nearest positioned ancestor), `fixed` (positioned against the viewport, stays on scroll), `sticky` (relative until it hits a scroll threshold, then fixed – the classic sticky header). ⚠ **sticky gotcha:** it silently fails if any ancestor has `overflow: hidden / auto`, or if you forget to set `top`. **ఇది real-world debugging question గా అడగొచ్చు.**

C5 ★★ Responsive design ఎలా చేస్తావు?

Mobile-first with `min-width` media queries, so the base styles are the simplest case and each breakpoint adds complexity. Relative units – `rem` for typography so it respects the user's browser setting, `% / fr / vw` for layout. `clamp()` for fluid type without breakpoints. Flexbox and Grid with `auto-fit / minmax` so a card grid reflows without any media query at all. Responsive images with `srcset`, and the viewport meta tag. Then actually testing on a real device, not just a resized window.

C6 ★★ `display: none` vs `visibility: hidden` vs `opacity: 0` ?

–	Space అక్రమిస్తుందా	Clickable	Screen reader కి వినిపిస్తుందా	Transition అవుతుందా
<code>display: none</code>	✗	✗	✗	✗
<code>visibility: hidden</code>	✓	✗	✗	✓ (partially)
<code>opacity: 0</code>	✓	✓ ఇంకా clickable!	✓ ఇంకా చదువుతుంది	✓

`opacity: 0` trap: element invisible కానీ ఇంకా **clickable మరియు screen readers చదువుతాయి** – ఇది accessibility bug మరియు "ghost click" bug కి కారణం. Hide చెయ్యాలంటే `pointer-events: none` కూడా జోడించు, లేదా `visibility: hidden` వాడు.

C7 ★★ Pseudo-class vs pseudo-element?

A pseudo-**class** targets a state of an existing element – `:hover`, `:focus`, `:nth-child`, `:disabled`. A pseudo-**element** creates a virtual element that isn't in the DOM – `::before`, `::after`, `::placeholder`. Convention: one colon for classes, two for elements.

C8 ★★ Semantic HTML ఎందుకు?

Using `<header>`, `<nav>`, `<main>`, `<article>`, `<button>` instead of `<div>` everywhere. It matters for **accessibility** (screen readers navigate by landmarks; a real `<button>` is keyboard-activable and focusable for free), **SEO**, and maintainability. **The concrete example to give:** a `<div onClick>` isn't reachable by keyboard, doesn't respond to Enter or Space, and isn't announced as a button – you'd have to add `role`, `tabIndex` and key handlers to reproduce what `<button>` gives you free.

C9 ★★ Browser లో page ఎలా render అవుతుంది? (critical rendering path)

HTML is parsed into the **DOM**, CSS into the **CSSOM**, the two combine into the **render tree** (visible nodes only), then **layout/reflow** computes geometry, then **paint** fills pixels, then **composite** assembles layers. **The performance point:** changing geometry (width, top, margin) triggers **layout** → **paint** → **composite** – expensive. Changing colour triggers **paint** → **composite**. Changing `transform` or `opacity` only triggers **composite**, which the GPU handles. **That's why animations should use `transform` and `opacity`, not `top` / `left` / `width`.** – ఈ ఒక్క point interview లో చాలా బాగా score చేస్తుంది.

C10 ★ CSS-in-JS / Tailwind / CSS Modules?

CSS Modules – scoped class names at build time, zero runtime, plain CSS. **Tailwind** – utility classes, no naming decisions, excellent bundle size after purging, at the cost of verbose markup. **CSS-in-JS** (styled-components/emotion) – colocated, dynamic styles from props, but runtime cost and it interacts awkwardly with React Server Components. **The current default I'd pick for a new project is Tailwind or CSS Modules**, since the industry has moved away from runtime CSS-in-JS for performance reasons.

Key Points

- **Flexbox vs Grid** = most-asked CSS question.
- **box-sizing: border-box** ఎందుకు – తెలియాలి.
- **transform / opacity animations** – ఇది senior signal.
- Semantic HTML → accessibility ← ఈ connection చెప్పు.
- 2 hours prep చాలు, ఎక్కువ time వృథా చెయ్యకు.

17. Git – Infosys technical round లో regular question

వివరణ

Git questions **~60% probability**. ఒక candidate technical round లో "JavaScript, React, Redux, and Git" అని explicitly report చేశాడు. ఇవి conceptual – commands బట్టి కాదు, ఎప్పుడు ఏది

G1 ★★ ★ merge vs rebase ?

Merge creates a new commit joining two branches. History is non-linear but it's a truthful record of what actually happened, and it's non-destructive. **Rebase** replays your commits on top of the target branch, producing linear history – but it **rewrites commit hashes**, creating new commits.

The golden rule: never rebase a branch that others have pulled. Rewriting shared history forces everyone else into a painful recovery. My working pattern is: rebase my own local feature branch onto main to keep it current and clean, then **merge** it into main via a PR.

G2 ★★ ★ git reset vs git revert ?

`reset` moves the branch pointer backwards – it **rewrites history**, so it's for local commits only. `--soft` keeps changes staged, `--mixed` (default) keeps them in the working directory, `--hard` **discards them entirely**. `revert` creates a **new commit that undoes** a previous one, leaving history intact – that's the safe choice for anything already pushed to a shared branch. **One-liner:** "Reset for local mistakes, revert for anything that's already public."

G3 ★★ ★ fetch vs pull ?

`fetch` downloads remote changes into your remote-tracking branches without touching your working directory – you can inspect what changed first. `pull` is `fetch` followed by `merge` (or `rebase` with `--rebase`), applying changes immediately. **I habitually fetch first on a shared branch** so a surprise merge doesn't land on top of uncommitted work.

G4 ★★ ★ Merge conflict ఎలా resolve చేస్తావు?

"Git marks the conflicting regions with `<<<<<<`, `=====`, `>>>>>>`. I open the file, look at both sides, and decide what the code should actually be – which sometimes means neither side verbatim. Then remove the markers, `git add` the file, and continue the merge or rebase. **The practical part is prevention:** pull frequently so conflicts stay small, keep feature branches short-lived, and after resolving, **run the tests** – a conflict resolved so it merely compiles is a very common way to ship a bug."

G5 ★★ git stash ఎప్పుడు?

When you need to switch context mid-change – an urgent bug on another branch – without committing half-finished work. `git stash push -m "wip"`, switch, fix, come back, `git stash pop`. `git stash list` to see them; `-u` to include untracked files.

Gotcha: stashes are easy to forget and aren't pushed anywhere, so I try not to leave them sitting for days.

G6 ★★ cherry-pick ఎప్పుడు?

Applying one specific commit from another branch – most often taking a hotfix from a release branch back into main, or pulling one commit out of a branch that isn't ready to merge as a whole. It creates a **new commit with a new hash**, so overusing it produces duplicate-looking history.

G7 ★★ మీ team Git workflow ఏమిటి? – ఇది తప్పకుండా అడుగుతారు

Answer with your **actual** workflow. A standard, safe one: "We use feature branches off `develop`, named `feature/JIRA-123-short-description`. Commits follow conventional commits – `feat:`, `fix:`, `chore:`. A PR needs at least one approval and green CI – lint, type-check, tests, build. We squash-merge so each feature is one clean commit on `develop`. Releases cut a `release/` branch, and hotfixes branch from `main` and get merged back into both." ⚠️ **మీ team లో process లేకపోతే కూడా** – "Our process was lighter than that; here's what I'd propose" అని చెప్పు. **Process awareness ఉందని చూపించడం ముఖ్యం** – Infosys process-driven company.

G8 ★ git squash ఎందుకు?

Combining several commits into one – usually with interactive rebase or a squash-merge – so the shared branch has one meaningful commit per feature instead of "fix typo", "wip", "actually fix it". It makes `git log`, `git bisect` and reverts far more useful.

Key Points

- `merge` vs `rebase` = most-asked. **Golden rule** చెప్పు.
- `reset` vs `revert` = second most-asked.
- **§G7 (your workflow)** కి నీ నిజమైన answer ready ఉంచు.
- Commands బట్టి కాదు – ఎప్పుడు, ఎందుకు.
- Deep dive → `Linux_Git_Telugu.md`.

18. Machine coding / Live coding (CodeSandbox)

వివరణ

Infosys technical round లో ~50% probability live coding ఉంటుంది. Candidates report చేసినది: **CodeSandbox** లో ఇస్తారు, screen share చేస్తారు, easy-to-medium difficulty. Tasks నిజంగా ఇచ్చినవి:

"Code a simple component and render a component to that component" · "Call the API endpoint using fetch or Axios from the second component to the first component" ·
"Write a React component that fetches data from an API and displays it in a list" ·
"Implement a counter using hooks" · "Write a component that accepts an array of items as props and displays each in a div"

ఇవి కష్టం కాదు. కానీ pressure లో, ఎవరో చూస్తున్నప్పుడు రాయడం practice అవసరం.

Real-life Scenario

ఇది driving test లాంటిది. నీకు driving వచ్చు – కానీ examiner పక్కన కూర్చుని చూస్తున్నప్పుడు clutch తప్పుతుంది. **Solution: examiner ఉన్నట్టు practice చెయ్యడం.** CodeSandbox తెరిచి, timer పెట్టుకుని, **loud గా మాట్లాడుతూ** ఈ tasks రాయ. కనీసం 5 సార్లు.

Task 1 ★★☆☆ – API fetch + list (అత్యంత common)

```
import { useState, useEffect } from 'react';

export default function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    async function load() {
      try {
        setLoading(true);
        setError(null);
        const res = await fetch('https://jsonplaceholder.typicode.com/users', {
          signal: controller.signal,
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`); // ⚠️ fetch 404 కి throw
        setUsers(await res.json());
      } catch (err) {
        if (err.name !== 'AbortError') setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    load();
    return () => controller.abort(); // ⚠️ cleanup – race condition + leak
  }, []);
}
```

```

}, []);

if (loading) return <p>Loading...</p>;
if (error) return <p role="alert">Something went wrong: {error}</p>;
if (!users.length) return <p>No users found.</p>; // ⚠️ empty state

return (
  <ul>
    {users.map(u => (
      <li key={u.id}>{u.name} – {u.email}</li> /* ⚠️ stable key, index కాదు */
    ))}
  </ul>
);
}

```

Interviewer ఈ 6 things చూస్తాడు (చాలామంది 2-3 మాత్రమే చేస్తారు):

1. ✅ **loading state**
2. ✅ **error state** – మరియు `res.ok` check
3. ✅ **empty state** ← ఇది చాలామంది మర్చిపోతారు
4. ✅ **cleanup / abort**
5. ✅ **stable key** (`u.id`, index కాదు)
6. ✅ Loud గా narrate చెయ్యడం

Bonus line to say out loud: *"In a real project I'd move this into a custom `useUsers` hook, or use TanStack Query – but let me write it directly here so the logic is visible."* – ఇది instant senior signal.

Task 2 ★★ – Parent → Child props + child → parent callback

Infosys exact గా అడిగింది: "Code a component and render a component to that component" + "call the API from the second component to the first".

```

// Parent – state ని own చేస్తుంది
function Parent() {
  const [selected, setSelected] = useState(null);
  const items = [{ id: 1, name: 'Alpha' }, { id: 2, name: 'Beta' }];

  return (
    <div>
      <h2>Selected: {selected?.name ?? 'none'}</h2>
      <ItemList items={items} onSelect={setSelected} /> /* data down, callback up */
    </div>
  );
}

// Child – presentational, state లేదు
function ItemList({ items, onSelect }) {
  return (
    <ul>
      {items.map(item => (

```

```

    <li key={item.id}>
      <button onClick={() => onSelect(item)}>{item.name}</button>
    </li>
  )}
</ul>
);
}

```

చెప్పాల్సిన line: "Data flows down through props and events flow up through callbacks – that's the one-way data flow React is built on. The child stays presentational, which makes it reusable and testable."

Task 3 ★★ – Counter with hooks

```

function Counter({ step = 1 }) {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(c => c - step)} aria-label="decrement"></button>
      <span>{count}</span>
      <button onClick={() => setCount(c => c + step)} aria-label="increment">+
    </button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

```

⚠ **ఇక్కడ ఒక్క detail మీద interviewer దృష్టి పెడతాడు:** `setCount(c => c + step)` – **functional update.** `setCount(count + step)` కూడా పని చేస్తుంది కానీ, **ఒకే handler లో రెండు సార్లు call చేస్తే** stale value వాడుతుంది. దీన్ని loud గా చెప్పు: "I'm using the functional form because updates are batched – if I called this twice in one handler, `count` would be stale both times and it'd only increment once."

Task 4 ★★ – Search with debounce (medium, వస్తే)

```

function SearchUsers() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  useEffect(() => {
    if (!query.trim()) { setResults([]); return; }

    const controller = new AbortController();
    const timer = setTimeout(async () => { // ← debounce: 400ms ఆగి, తర్వాత
      call
      try {
        const res = await fetch(`/api/search?q=${encodeURIComponent(query)}`,
          { signal: controller.signal });
        if (res.ok) setResults(await res.json());
      }
    }
  }, 400);
}

```

```

    } catch (e) { if (e.name !== 'AbortError') console.error(e); }
    }, 400);

    return () => { clearTimeout(timer); controller.abort(); }; // query మరితే రెండు
cancel
    }, [query]);

    return (
      <>
        <input value={query} onChange={e => setQuery(e.target.value)}
placeholder="Search..." />
        <ul>{results.map(r => <li key={r.id}>{r.name}</li>)}</ul>
      </>
    );
  }
}

```

ఇక్కడ cleanup రెండు పనులు చేస్తుంది – pending timer ని cancel చేస్తుంది (debounce), మరియు in-flight request ని abort చేస్తుంది (race condition). **ఈ రెండూ చెప్తే excellent.**

Gotchas (సాధారణ తప్పులు)

1. **మౌనంగా code రాయడం.** Interviewer నీ **ఆలోచన** చూడాలనుకుంటున్నాడు. నిరంతరం **narrate** చెయ్యి: "First I'll set up the state for loading and error..."
2. **Loading/error/empty states మర్చిపోవడం.** ఇవే junior/mid తేడా.
3. **key={index}** వాడటం.
4. **Cleanup మర్చిపోవడం.**
5. **Perfect code కోసం ఆగిపోవడం. Working first, then improve.** "Let me get it working, then I'll clean it up."
6. **Console error వస్తే panic అవ్వడం.** "Let me check what that is" – debugging చూపించడం **positive signal**.

Key Points

- Tasks **easy-to-medium** – API fetch + list, props, counter, debounced search.
- **6-point checklist** (Task 1) ప్రతి component కి apply చెయ్యి.
- **Narrate constantly.** Silence = negative.
- **Working code > elegant code.**
- CodeSandbox తెరిచి **5 సార్లు practice** చెయ్యి, timer తీ, loud గా.

Interview దృష్టి

Live coding మొదలుపెట్టేముందు ఈ 3 వాక్యాలు చెప్పు – ఇవి నీకు ఆలోచించే time ఇస్తాయి, మరియు **senior** గా కనిపిస్తావు:

1. "Let me make sure I understand the requirement – you want X that does Y, correct?"
2. "I'll start with the basic structure and get it working, then handle the edge cases."
3. "I'm going to talk through what I'm doing as I go."

19. Node.js / Express / MongoDB – Full Stack అయితే

వివరణ

⚠️ **ముందు §3 చదువు** – Infosys "Full Stack" చాలాసార్లు Java. Node role confirm అయితే మాత్రమే ఈ section. Probability ~40%.

N1 ★★☆☆ "Node.js అంటే ఏమిటి? ఎందుకు వాడాలి?"

"A JavaScript runtime built on V8 that runs outside the browser. What makes it distinctive is the **event-driven, non-blocking I/O model**: instead of one thread per request blocking on database and network calls, a single thread handles many concurrent connections and hands I/O off to the OS, picking up the result when it's ready.

That makes it excellent for **I/O-bound** workloads – APIs, real-time apps, proxies – and poor for **CPU-bound** work, because a heavy computation blocks the single event loop thread and stalls every other request. For that you'd use worker threads or a separate service.

The practical benefits on a team: one language across the stack, shared validation code and types between client and server, and npm's ecosystem."

N2 ★★☆☆ "Node event loop phases?"

libuv runs the loop through phases in order: **timers** (`setTimeout` / `setInterval`), **pending callbacks**, **idle/prepare**, **poll** (retrieve new I/O events – this is where most work happens), **check** (`setImmediate`), and **close callbacks**. Between every phase, Node drains the **microtask queues**: `process.nextTick` first (highest priority), then promise callbacks. **The interview-friendly summary**: " `nextTick` beats promises, promises beat timers, and `setImmediate` runs after I/O whereas `setTimeout(fn, 0)` runs before it in the next iteration – which is why their relative order in the main module is actually non-deterministic."

N3 ★★☆☆ "Middleware in Express?"

Functions with the signature (`req`, `res`, `next`) that run in order for each request. They can read and modify `req` / `res`, end the response, or call `next()` to pass control on. It's how cross-cutting concerns are layered: logging, `express.json()` body parsing, CORS, authentication, rate limiting, and finally the route handler.

`app.use()` registers middleware – globally, or scoped to a path prefix. **Error middleware is special: it takes four arguments** – (`err`, `req`, `res`, `next`). Express identifies it by arity, and it must be registered **last**. You reach it by calling `next(err)`.

```
// Error handling middleware – ఎప్పుడూ చివరన register చెయ్యాలి
app.use((err, req, res, next) => {
```

```

    console.error(err.stack);
    const status = err.status || 500;
    res.status(status).json({
      error: status === 500 ? 'Internal server error' : err.message, // ⚠️ 500 details
      leak చెయ్యకు
    });
  });

// ⚠️ async route లో throw చేస్తే Express 4 దాన్ని పట్టుకోదు – wrap చెయ్యాలి
const asyncHandler = fn => (req, res, next) => Promise.resolve(fn(req, res,
next)).catch(next);
app.get('/users/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({ error: 'Not found' });
  res.json(user);
}));

```

❗ **asyncHandler wrapper mention చెయ్యి** – Express 4 లో async errors silently hang అవుతాయి. ఇది తెలిస్తే production experience ఉందని అర్థం. (Express 5 దీన్ని native గా handle చేస్తుంది.)

N4 ⭐⭐ "express.static() డేనికీ?"

Serves static assets – the folder you pass becomes the web root, so `express.static('public')` makes `public/logo.png` available at `/logo.png`. In production you'd normally put a CDN or nginx in front rather than serving assets from Node.

N5 ⭐⭐ "MongoDB vs SQL – ఎప్పుడు ఏది?"

MongoDB stores flexible JSON-like documents with no fixed schema, scales horizontally by sharding, and is a good fit when the data is naturally document-shaped, the schema evolves, or you read whole aggregates together – catalogues, content, event logs. **SQL** gives you a fixed schema, real joins, and **multi-row ACID transactions** – the right choice when relationships matter and correctness is non-negotiable, like financial records or anything with heavy reporting. "MongoDB has supported multi-document transactions since 4.0, so the old 'no transactions' argument is out of date – but relational integrity and complex joins are still where SQL is simply better."

N6 ★★ "MongoDB indexing? Text search?"

An index is a B-tree over a field that turns a full collection scan into a lookup. Without one, a query on a million documents reads all million. You diagnose with `.explain('executionStats')` – look for `COLLSCAN`, which means no index was used. Compound indexes follow the **prefix rule**: an index on `{a: 1, b: 1}` serves queries on `a` and on `a+b`, but not on `b` alone. **Text search**: create a text index – `db.articles.createIndex({ title: 'text', body: 'text' })` – then query with `{ $text: { $search: "react hooks" } }`. There's a limit of one text index per collection. For anything serious (fuzzy matching, ranking, facets) I'd use Atlas Search or Elasticsearch instead. **Cost to mention**: indexes speed reads and **slow writes**, and they take disk space – so you index for actual query patterns, not everything.

N7 ★★ "MERN architecture flow explain చెయ్య?"

"React renders the UI and sends HTTP requests. Express routes those requests through middleware – auth, validation – into controllers holding the business logic. Mongoose models define schemas and talk to MongoDB. The response comes back as JSON and React renders it. In a real deployment I'd add: JWT or session auth at the middleware layer, request validation with Zod or Joi at the boundary, Redis for caching hot reads and for rate limiting, and structured logging."

N8 ★★ "Node లో error handling?"

Operational errors (a failed request, a bad input) are handled and returned as a proper status; programmer errors (a bug) should crash the process and let the supervisor restart it. Concretely: `try/catch` around `await`, a centralised Express error middleware, `process.on('unhandledRejection')` and `uncaughtException` handlers that log and exit cleanly, and PM2 or Kubernetes to restart. **Never swallow an error silently** – an empty `catch {}` is how incidents become unexplainable.

Key Points

- **N1 (why Node)** మరియు **N3 (middleware)** = most-asked.
- **CPU-bound weakness** స్వయంగా mention చెయ్య – balanced view.
- `asyncHandler`, `.explain()`, index write-cost – ఇవి **production experience signals**.

20. DBMS & SQL

వివరణ

Probability ~35%. Full Stack అయితే ఎక్కువ. Infosys enterprise clients SQL-heavy కాబట్టి basics రావాలి. Deep dive → `DBMS_Telugu.md`.

Q1 ★★☆☆ "JOIN types?"

INNER – rows matching in both tables. **LEFT** – all rows from the left, with **NULL** s where the right has no match (this is the one you use to find "customers with no orders").

RIGHT – the mirror image. **FULL OUTER** – everything from both sides. **CROSS** – the cartesian product.

Q2 ★★☆☆ Common query patterns – ఇవి రాయమని అడుగుతారు

```
-- 2nd highest salary (అత్యంత classic)
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Nth highest – window function (modern, better answer)
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rn FROM employees
) t WHERE rn = 2;

-- Duplicate emails కనుక్కో
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- ప్రతి department లో అత్యధిక salary ఉన్నవాళ్ళు
SELECT d.name, e.name, e.salary FROM employees e
JOIN departments d ON d.id = e.dept_id
WHERE e.salary = (SELECT MAX(salary) FROM employees WHERE dept_id = e.dept_id);

-- ఒక్క order కూడా లేని customers
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

Q3 ★★☆☆ " WHERE vs HAVING ?"

WHERE filters **individual rows before** grouping; **HAVING** filters **groups after** aggregation. So you can't use an aggregate in **WHERE** – **WHERE COUNT(*) > 1** is invalid, it has to be **HAVING COUNT(*) > 1**.

Q4 ★★☆☆ " DELETE vs TRUNCATE vs DROP ?"

DELETE removes rows (optionally with a **WHERE**), is logged per row, fires triggers, and can be rolled back. **TRUNCATE** removes all rows quickly by deallocating pages – no **WHERE** , no triggers, resets identity. **DROP** removes the table itself.

Q5 ★★ "Normalisation? 1NF/2NF/3NF?"

Organising tables to remove redundancy and update anomalies. **1NF** – atomic values, no repeating groups. **2NF** – 1NF plus no partial dependency on part of a composite key. **3NF** – 2NF plus no transitive dependency (non-key columns depending on other non-key columns). "In practice, teams normalise to 3NF for transactional systems and **deliberately denormalise** for read-heavy reporting, because joins get expensive. It's a read/write trade-off, not a rule."

Q6 ★★ "Index అంటే ఏమిటి? Trade-off?"

A B-tree structure that makes lookups on a column logarithmic instead of a full scan. The cost: every insert, update and delete must also maintain the index, plus disk space. So you index columns used in `WHERE`, `JOIN` and `ORDER BY` – not every column.

Q7 ★★ "ACID properties?"

Atomicity – all or nothing. **Consistency** – constraints hold before and after. **Isolation** – concurrent transactions don't see each other's partial work. **Durability** – committed data survives a crash.

Q8 ★ "Primary key vs unique key vs foreign key?"

A **primary key** uniquely identifies a row – one per table, no `NULL`s. A **unique key** also enforces uniqueness but allows a `NULL` and you can have several. A **foreign key** references another table's primary key and enforces referential integrity.

Key Points

- **2nd highest salary** మరియు **duplicates** – ఇవి తప్పకుండా practice చెయ్యి.
- `WHERE` vs `HAVING` = most-asked concept.
- Window functions తెలిస్తే mention చెయ్యి – modern signal.

21. API, REST, HTTP & Security basics

వివరణ

Probability ~**70%** (API/HTTP), security basics ~40%. Frontend developer కి కూడా ఇవి అడుగుతారు.

A1 ★★★★★ "REST principles?"

Stateless (the server keeps no client session between requests – every request carries what it needs), resource-based URLs (nouns, not verbs – `/users/42` , not `/getUser?id=42`), the HTTP methods carrying the semantics, standard status codes, and a uniform interface. Layered and cacheable.

A2 ★★★★★ "HTTP methods – idempotent ఏవి?"

Method	పని	Idempotent	Safe
GET	చదవడం	✓	✓
POST	సృష్టించడం	✗	✗
PUT	పూర్తిగా replace	✓	✗
PATCH	పార్షియల్గా update	సాధారణంగా ✗	✗
DELETE	తొలగించడం	✓	✗

Idempotent = అదే request ని n సార్లు పంపినా, ఒకసారి పంపినట్లే result. `DELETE /users/42` రెండో సారి 404 ఇవ్వచ్చు, కానీ **state అదే** – కాబట్టి idempotent. **ఇది ఎందుకు ముఖ్యం:** network failure అయినప్పుడు idempotent requests ని safe గా retry చెయ్యచ్చు; POST ని retry చేస్తే duplicate records వస్తాయి. (అందుకే payment APIs **idempotency keys** వాడతాయి.)

A3 ★★★★★ "Status codes?"

2xx – 200 OK, 201 Created (with a `Location` header), 204 No Content. **3xx** – 301 permanent, 302/307 temporary, 304 Not Modified (caching). **4xx client** – 400 malformed, **401 unauthenticated** (who are you?), **403 unauthorised** (I know who you are, you can't do this), 404, 409 conflict, 422 validation failed, 429 rate limited. **5xx server** – 500, 502, 503, 504. ⚠️ **401 vs 403 తేడా అడుగుతారు** – ఇది classic.

A4 ★★ "CORS అంటే ఏమిటి? ఎందుకు వస్తుంది?"

"The browser's **same-origin policy** blocks JavaScript from reading a response from a different origin – different scheme, host or port. CORS is the mechanism for a server to opt in, by sending `Access-Control-Allow-Origin` and related headers.

The key thing people get wrong: CORS is enforced by the browser, not the server. The request often reaches the server and executes; the browser just refuses to hand the response to your JavaScript. That's also why Postman never shows a CORS error.

For non-simple requests (custom headers, `PUT / DELETE`, JSON content type) the browser first sends a **preflight OPTIONS** request. And **the fix belongs on the server** – a browser extension or disabling web security only papers over it locally."

A5 ★★ "Authentication vs Authorisation?"

Authentication is **who you are** (login, 401 when it fails). Authorisation is **what you're allowed to do** (roles and permissions, 403 when it fails).

A6 ★★ "JWT ఎలా పని చేస్తుంది? Trade-offs?"

Three base64url parts – header, payload, signature – separated by dots. The server signs it; anyone can **read** the payload, so **never put secrets in it**. The server verifies the signature rather than looking up a session, which is what makes it stateless and horizontally scalable. **The real trade-off: you can't revoke a JWT.** Until it expires it's valid, so a logout or a ban doesn't take effect immediately. The usual mitigation is short-lived access tokens (15 minutes) plus a refresh token that is stored server-side and can be revoked. **Storage:** httpOnly cookie over `localStorage` – see §Q32.

A7 ★★ "XSS మరియు CSRF?"

XSS – injecting script into a page so it runs with the victim's session. Defence: escape output (React does this by default), sanitise any HTML you must inject, and a Content-Security-Policy header. **CSRF** – tricking a logged-in user's browser into making a state-changing request to your site from another site; the browser attaches the cookies automatically. Defence: `SameSite` cookies, anti-CSRF tokens, and checking `Origin`. **The neat contrast to state:** "XSS exploits the user's trust in the site; CSRF exploits the site's trust in the user's browser." Deep dive → `Security_Telugu.md`.

A8 ★★ "REST vs GraphQL?"

REST has multiple endpoints returning fixed shapes, which leads to over-fetching (more data than the screen needs) and under-fetching (several round trips). GraphQL has one endpoint where the client specifies exactly the fields it wants, which is a big win for mobile and for screens assembling data from many sources. The costs: caching is harder (no per-URL HTTP cache), query complexity has to be limited or a client can DoS you, and there's real server-side setup. **REST is still the right default for a CRUD API**; GraphQL earns its complexity when you have many clients with different data needs.

Key Points

- **CORS** = most-asked. "**Browser enforces it, not the server**" – ఈ line చెప్పు.
- **401 vs 403, idempotency, JWT revocation problem** – ఇవి differentiators.
- Status codes: 200/201/204/400/401/403/404/409/429/500 తెలియాలి.

22. Testing – 3 YOE కి expected level

వివరణ

Probability ~**25%** – తక్కువ, కానీ అడిగితే "**మేము tests రాయలేదు**" అనేది **weak answer**. Infosys process-driven, quality gates ఉంటాయి.

T1 ★★ "Unit vs integration vs E2E?"

Unit – one function or component in isolation, fast, many of them. **Integration** – several pieces together, e.g. a form component with its validation and a mocked API. **E2E** – the real browser driving the real app (Cypress/Playwright), slow but highest confidence. **The testing trophy** (Kent C. Dodds' model, which the React community follows): the largest investment goes into **integration** tests, because they give the best confidence-per-effort – unit tests can all pass while the app is broken.

T2 ★★ "React Testing Library ఎందుకు, Enzyme కాదు?"

"RTL tests behaviour the way a user experiences it – you query by role, label and text, not by component internals. Enzyme let you reach into state and props, which meant a refactor that changed nothing user-visible would still break the tests. **The guiding principle is 'the more your tests resemble the way your software is used, the more confidence they give you.'** Practically: `getByRole('button', {name: /submit/i})` over a test ID, `userEvent` over `fireEvent` because it simulates the real event sequence, and `findBy*` for async appearance."

```
test('shows an error when submitting an empty email', async () => {
  const user = userEvent.setup();
  render(<LoginForm onSubmit={jest.fn()} />);

  await user.click(screen.getByRole('button', { name: /sign in/i }));

  expect(await screen.findByText(/email is required/i)).toBeInTheDocument();
});
```

T3 ★★ "API calls ఎలా mock చేస్తావు?"

"I prefer **MSW (Mock Service Worker)** over mocking `fetch` or axios directly – it intercepts at the network level, so the component code under test is completely unmodified and the same handlers work in tests and in local development. Mocking the HTTP client couples your tests to the implementation detail of which client you use."

T4 ★ "Code coverage ఎంత ఉండాలి?"

"Coverage tells you what was executed, not what was verified – you can hit 100% with tests that assert nothing. I'd rather have 60% coverage on the payment and auth flows than 90% spread evenly across getters. As a team gate, 70-80% on changed lines is a reasonable signal, but the real question is whether the critical paths are covered."

⚠ Tests రాయకపోతే ఏం చెప్పాలి:

"Honestly, our coverage was thin – we had tests on the utility layer but the components weren't well covered, mostly because of delivery pressure. I've been working through Testing Library on my own projects, and it's one of the practices I'd want to bring into a team that already has the discipline for it." **నిజాయితీ + initiative = acceptable.**

"Testing is not needed" = instant red flag.

23. DSA – Infosys ఏ level అడుగుతుంది

DSA · INFOSYS LATERAL LO ఏ స్థాయి

DSA – Infosys lateral lo నిజమైన స్థాయి

వచ్చేవి (~30% సంభావ్యత)

Array / string manipulation
HashMap counting
Two pointers · simple recursion

అరుదుగా

Basic tree traversal
Simple sorting logic

దాదాపు రావు

DP · graphs · segment tree
LeetCode Hard

కాబట్టి ఎక్కడ సమయం పెట్టాలి

DSA కి 20% – LeetCode Easy + కొన్ని Medium చాటు (Arrays, Strings, HashMap).
React / JS depth కి 40% · Project narrative + STAR కి 40%.
Product company interview కాదు ఇది – అదే తేడాని గుర్తుంచుకోవడం ముఖ్యం.

వివరణ

Probability ~30%, మరియు అడిగినా **LeetCode Easy-Medium**. Infosys lateral frontend role కి **hard DSA అడగరు** – graphs, DP, segment trees ఆశించకు.

⚠ **ఇది ముఖ్యమైన strategic point:** నీ time ని DSA మీద 70% పెడితే **తప్పు allocation**. §7 లోని probability table చూడు – project + React + JS = 80% weight. DSA = 30% probability, తక్కువ depth. **DSA కి మొత్తం prep time లో ~15% చాలు.**

ఏం అడుగుతారు – realistic list

Strings & Arrays (అత్యధిక probability):

- Reverse a string / check palindrome
- Find duplicates in an array
- Second largest element
- Remove duplicates
- Count character frequency / first non-repeating character
- Anagram check
- Two Sum
- Move zeros to end
- Array rotation
- Max subarray sum (Kadane's)
- Merge two sorted arrays
- FizzBuzz (ఇంకా అడుగుతారు!)

Objects/Maps:

- Group an array of objects by a key
- Flatten a nested object
- Count word frequency in a sentence

Basic recursion:

- Factorial, Fibonacci (with memoisation)
- Flatten a nested array

JavaScript లో ఎలా solve చెయ్యాలి – 3 examples

```
// 1. First non-repeating character – O(n)
function firstUnique(str) {
  const counts = new Map();
  for (const ch of str) counts.set(ch, (counts.get(ch) ?? 0) + 1);
  for (const ch of str) if (counts.get(ch) === 1) return ch;
  return null;
}
// firstUnique('swiss') → 'w'
// ⚠️ చెప్పు: "Two passes but O(n) overall – a nested loop would be O(n²)."
```

```
// 2. Group by key – interview favourite
function groupBy(arr, key) {
  return arr.reduce((acc, item) => {
    const k = item[key];
    (acc[k] ??= []).push(item); // ??= – key లేకపోతే array create చెయ్యి
    return acc;
  }, {});
}
// groupBy([{dept: 'IT', n: 'A'}, {dept: 'HR', n: 'B'}, {dept: 'IT', n: 'C'}], 'dept')
// → { IT: [{...A}, {...C}], HR: [{...B}] }
```

```
// 3. Kadane's – max subarray sum, O(n)
function maxSubarraySum(nums) {
  let best = nums[0], current = nums[0];
  for (let i = 1; i < nums.length; i++) {
    current = Math.max(nums[i], current + nums[i]); // ఇక్కడ కొత్తగా మొదలుపెట్టాలా?
    best = Math.max(best, current);
  }
  return best;
}
// maxSubarraySum([-2,1,-3,4,-1,2,1,-5,4]) → 6
```

ఎలా approach చెయ్యాలి – 5 steps (interviewer ఇదే చూస్తాడు)

1. **Restate the problem** – "So I need to find the first character that appears exactly once, and return null if there isn't one?"
2. **Edge cases అడుగు** – "What should I return for an empty string? Is it case-sensitive?"
3. **Brute force చెప్పు, ఎందుకు సరిపోదో చెప్పు** – "The naive approach is a nested loop, O(n²). Let me do better."
4. **Optimised approach + complexity** – "A hash map gives me O(n) time, O(k) space where k is the alphabet size."
5. **Code రాసి, dry run చెయ్యి** – ఒక example తో line by line.

Steps 1-3 ని skip చెయ్యకు. చాలామంది వెంటనే code రాయడం మొదలుపెడతారు. **Approach explain చెయ్యడమే ఎక్కువ marks.**

Gotchas (సాధారణ తప్పులు)

1. **DSA మీద అతిగా time పెట్టడం.** ~15% చాలు. Project + React కి పెట్టు.
2. **Complexity చెప్పకపోవడం.** ప్రతి solution తర్వాత "That's O(n) time and O(n) space" చెప్పు – ఇది automatic గా అలవాటు చేసుకో.
3. **Edge cases మర్చిపోవడం** – empty array, single element, all-same elements, negative numbers.
4. **Silence.** ఆలోచిస్తున్నప్పుడు loud గా.

Key Points

- Level: **LeetCode Easy, అప్పుడప్పుడు Easy-Medium.**
- Focus: **strings, arrays, hash maps.** Graphs/DP అవసరం లేదు.
- **Prep time allocation: ~15%.**
- **Approach > code.** 5 steps follow చెయ్యి.
- Practice కావాలంటే → [DSA_Patterns_Telugu.md](#) (Arrays, Strings, Hashing chapters చాలు).

Part 3 – Round 2: Managerial Round

24. Managerial round అంటే ఏమిటి – ఎందుకు ఎక్కువ risky

వివరణ

Technical round crack అయ్యాక చాలామంది relax అవుతారు – "ఇక HR మాత్రమే మిగిలింది" అనుకుంటారు. **అది తప్పు.** Managerial round అనేది HR round కాదు, casual chat కాదు. ఇక్కడ ఒక **Delivery Manager / Project Manager** కూర్చుని decide చేస్తాడు: "ఈ వ్యక్తిని నా client project లో పెట్టొచ్చా?"

ఇక్కడ experienced candidates ఎక్కువగా reject అవుతారు – technical round లో కాదు.

Duration: technical round 1 hour అయితే, ఇది సాధారణంగా **15-45 minutes**. తక్కువ time కానీ ఎక్కువ weight.

Real-life Scenario

Technical round = "నీకు వంట వచ్చా?" Managerial round = "నిన్ను నా restaurant kitchen లో పెట్టొచ్చా?"

రెండోది వేరే question. వంట బాగా వచ్చిన chef కూడా – ఇతర chefs తో గొడవ పడితే, deadline లో panic అయితే, customer complaint వస్తే defensive అయితే – restaurant కి పనికిరాడు.

Manager నీలో వెతుకుతున్నది: **ownership, calm under pressure, communication, flexibility, team fit.**

ఎం test చేస్తారు

Parameter	ఎం చూస్తారు	ఎలా అడుగుతారు
Ownership	Problem నీది అనుకుంటావా, ఇతరుల మీద నెడతావా	"Tell me about a project that went wrong"
Conflict handling	Disagreement ని ఎలా handle చేస్తావు	"Tell me about a disagreement with a colleague"
Pressure	Deadline లో panic అవుతావా	"How do you handle tight deadlines?"
Client orientation	Client ముందు ఎలా ప్రవర్తిస్తావు	"Client asks for a change at the last minute..."
Communication	Structured గా మాట్లాడగలవా	ప్రతి answer లో
Flexibility	Location, shift, technology change	"Are you open to...?"
Learning	కొత్తది నేర్చుకుంటావా	"How do you keep up with technology?"
Team	Junior లకి help చేస్తావా	"Have you mentored anyone?"

గమనించు: ఇది పూర్తిగా **behavioural కాదు** – Infosys managerial round లో **technical scenario questions కూడా** ఉంటాయి (§S3 payment/mobile question ఇక్కడే వచ్చింది). Manager నీ technical judgement ని architecture level లో test చేస్తాడు, syntax level లో కాదు.

Gotchas (సాధారణ తప్పులు)

1. **"ఇది just HR chat" అనుకోవడం.** ఇది **evaluation round**, gate round.
2. **Prepare అవ్వకపోవడం.** Behavioural answers ని **ముందే రాసుకోవాలి** – spontaneous గా చెప్తే rambling వస్తుంది.
3. **"నాకు ఎప్పుడూ conflict రాలేదు" అనడం.** ఇది "team లో పని చెయ్యలేదు" లేదా "నిజం చెప్పలేదు" అని అర్థం.
4. **Current company ని తిట్టడం.** Instant red flag.
5. **Location/shift కి flat "no".** "Let me understand the details" అను.
6. **Answer ముగించకపోవడం.** STAR లో **R (Result)** చెప్పకుండా ఆగిపోవడం.

Key Points

- Managerial round = **gate round**, formality కాదు.
- 15-45 minutes, కానీ **rejection rate ఎక్కువ.**
- Technical scenarios + behavioural కలిపి ఉంటాయి.
- **STAR method తప్పనిసరి** (§25).
- **ప్రధాన message:** "నేను own చేస్తాను, calm గా ఉంటాను, clear గా communicate చేస్తాను, flexible గా ఉంటాను."

25. STAR method – నీ 5 stories ముందే ready చెయ్యి

వివరణ

STAR = **S**ituation, **T**ask, **A**ction, **R**esult. ఇది behavioural answers కి universal structure. Infosys interviewers explicitly ఈ format అశిస్తారు.

ఎందుకు structure అవసరం: structure లేకపోతే నీ answer ఇలా వినిపిస్తుంది – "అవును, ఒకసారి ఒక bug వచ్చింది, చాలా కష్టం అయింది, అప్పుడు మేము... ఏంటంటే... చివరికి fix అయింది." ఇది **weak**. Manager ఏమీ గుర్తుపెట్టుకోడు.

Real-life Scenario

STAR అనేది ఒక **సినిమా trailer** structure లాంటిది. Trailer లో: setup (ఇది ఎవరి కథ), conflict (సమస్య ఏమిటి), action (hero ఏం చేశాడు), payoff (ఫలితం). ఈ నాలుగు 60 seconds లో.

Trailer ముగిసేసరికి కథ అర్థమవ్వాలి.

నీ STAR answer కూడా అంతే – **60-90 seconds**, నాలుగు భాగాలు, ముగింపు స్పష్టం.

Format – timing తో

Part	ఎంత time	ఏం చెప్పాలి
Situation	15 sec	Context – ఎప్పుడు, ఏ project, ఏం జరుగుతోంది
Task	10 sec	నీ responsibility ఏమిటి
Action	40 sec ← ఎక్కువ భాగం	నువ్వు ఏం చేశావు, step by step
Result	20 sec	ఫలితం – సాధ్యమైతే number తో + నేర్చుకున్నది

⚠ **Action లో "we" కాదు, "I".** Manager నీ contribution వినాలనుకుంటున్నాడు. Team ని credit ఇవ్వొచ్చు, కానీ నీ actions స్పష్టంగా చెప్పు.

నీకు కావాల్సిన 5 stories (ఇవి ఇప్పుడే రాసుకో)

ఈ **5 stories** 90% behavioural questions ని cover చేస్తాయి. ఒక్కొక్కటి **ఒక page** రాసుకో, తర్వాత 90 seconds కి compress చెయ్యి.

#	Story	ఏ questions కి పనికొస్తుంది
1	ఒక కష్టమైన technical problem నువ్వు solve చేసినది	"Biggest technical challenge", "complex problem", "how do you debug"
2	ఒక tight deadline / production incident	"Tight deadline", "pressure", "crisis", "prioritisation"
3	ఒక conflict / disagreement (colleague లేదా manager తో)	"Disagreement", "difficult teammate", "pushback"
4	ఒక failure / mistake నువ్వు చేసినది	"Tell me about a failure", "a mistake you made", "what would you do differently"

#	Story	ఎ questions కి పనికొస్తుంది
5	ఒక ownership / initiative – నిన్ను ఎవరూ అడగకుండా చేసినది	"Went above and beyond", "took initiative", "improved a process", "mentored someone"

Model answer – Story 1 (technical challenge)

[S] "About a year into my current project, our main reporting dashboard started getting slow complaints from the operations team. It was rendering around eight thousand shipment rows, and every filter change froze the browser for two to three seconds.

[T] "I own the reporting module, so this was mine to fix. The constraint was that we couldn't change the requirement – the ops team genuinely needed to see and scan the full dataset, so 'just paginate it' wasn't acceptable to them.

[A] "I started by profiling with React DevTools rather than guessing, and found two separate problems. First, the whole list was re-rendering on every keystroke because the filter function ran inline in the render. Second, we were mounting eight thousand DOM nodes.

"So I did three things. I moved the filter into a `useMemo` keyed on the query and the source data. I debounced the search input at 300 milliseconds so it wasn't running per character. And the big one – I introduced list virtualisation with react-window, so we only render the rows in the viewport.

"The tricky part was that our rows had variable heights because of a wrapping address field. I worked around it by measuring and caching row heights rather than forcing a fixed height, which would have broken the design. I also had to make sure keyboard navigation and the export-all feature still worked, since those touched rows that weren't mounted.

[R] "Interaction time went from two to three seconds down to under two hundred milliseconds, and the 'dashboard is slow' tickets stopped. Two other teams picked up the same pattern for their grids afterwards. The lesson I took from it was to profile before optimising – my first instinct had been that the filter was the whole problem, and it was actually the smaller half."

ఎందుకు ఇది excellent:

- ✓ Concrete numbers (8,000 rows, 2-3s → 200ms, 300ms debounce)
- ✓ "I" throughout
- ✓ **Constraint mention చేశాడు** ("we couldn't just paginate") – ఇది depth చూపిస్తుంది
- ✓ Process చూపించాడు (profile → diagnose → fix), magic కాదు
- ✓ **Complication mention చేశాడు** (variable heights, keyboard nav) – నిజమైన story లా వినిపిస్తుంది

- ☒ Business impact ("tickets stopped") + adoption ("two other teams")
- ☒ **Learning తో ముగించాడు**

Model answer – Story 4 (failure) – ఇది అత్యంత ముఖ్యం

చాలామంది ఈ question కి భయపడతారు. భయపడకు – ఇది నీ maturity చూపించే question.

[S] "About eight months ago I pushed a change to our shared date utility to fix a formatting bug for our UK users.

[T] "It looked like a two-line change and I treated it that way.

[A] "I tested it against the UK format and it worked, so I merged it. What I hadn't checked was that the same utility was used by the export module, which relied on the old ISO output. The nightly export job failed silently – it wrote malformed dates rather than erroring – and nobody noticed for two days until a client flagged mismatched dates in their reconciliation report.

"Once we found it, I owned it immediately – I told my lead and the client-facing PM straight away rather than trying to quietly patch it, because they needed to know the report data was wrong. I wrote the fix, and I also wrote a script to identify and re-export the affected records so the client didn't have to reconcile manually.

[R] "It was fixed within a day and the client was fine once they saw we'd already corrected the historical data. But the real result was what changed afterwards: I added tests around that utility covering every consumer, and I started actually checking usages before touching anything shared – a quick grep would have caught this in thirty seconds. I also pushed for the export job to fail loudly instead of writing bad data silently, which got picked up as a broader fix. It taught me that in a shared codebase, 'small change' is about blast radius, not line count."

ఎందుకు ఇది excellent:

- ☒ నిజమైన failure – trivial కాదు, career-ending కాదు
- ☒ **Blame ఎవరి మీదా వెయ్యలేదు**
- ☒ వెంటనే escalate చేశాడు – ఇది manager కి బంగారం ("hides problems" అనేది worst trait)
- ☒ Damage repair చేశాడు (re-export script), symptom మాత్రమే కాదు
- ☒ **Systemic fix** (tests, process change) – "నేను ఇక జాగ్రత్తగా ఉంటాను" అనే బలహీన ముగింపు కాదు
- ☒ ఒక memorable line తో ముగించాడు

Gotchas (సాధారణ తప్పులు)

1. **Result చెప్పకపోవడం.** చాలామంది Action దగ్గర ఆగిపోతారు. **R తప్పనిసరి, number తో అయితే ఇంకా మంచిది.**
2. **Situation చాలా పొడవుగా చెప్పడం.** 15 seconds. Context మాత్రమే, కథ కాదు.

3. **Failure question కి fake failure చెప్పడం** – "I work too hard", "I'm a perfectionist".
Interviewers దీన్ని ద్వేషిస్తారు. నిజమైన, medium-sized failure చెప్పు.
4. **Conflict story లో ఎదుటివాడిని villain చెయ్యడం.** ఎప్పుడూ ఇద్దరికీ **valid point ఉంది** అనే framing వాడు.
5. **అన్ని questions కి ఒకే story వాడటం.** 5 vary చేసిన stories.
6. **Story బట్టి కొట్టడం. Bullet points గుర్తుపెట్టుకో,** sentences కాదు – లేకపోతే recital లా వినిపిస్తుంది.

Key Points

- **STAR: 15 / 10 / 40 / 20 seconds.** Action ఎక్కువ.
- **5 stories రాసుకో** – challenge, deadline, conflict, failure, initiative.
- ప్రతి story కి **కనీసం ఒక number.**
- "I", "we" కాదు.
- **Failure story = highest-value story.** దీన్ని బాగా prepare చెయ్యి.
- ప్రతి story ని **learning తో ముగించు.**

Interview దృష్టి

Homework (ఇది తప్పనిసరి): 5 stories రాయు. ప్రతి దానికి:

- S: 1 వాక్యం
- T: 1 వాక్యం
- A: 4-5 bullet points
- R: 1 number + 1 learning

తర్వాత **loud గా, timer తో practice** చెయ్యి. 90 seconds దాటకూడదు. **ఇది 2 గంటల పని, కానీ managerial round ని decide చేస్తుంది.**

26. Scenario question bank – 24 questions + model answers

వివరణ

ఇవి Infosys managerial round లో నిజంగా అడిగినవి + service-company standard patterns. **అన్నిటికీ answer రాయాల్సిన అవసరం లేదు** – కానీ ★★★★★ ఉన్నవాటికి తప్పకుండా.

A. Client scenarios (Infosys కి అత్యంత ముఖ్యం)

M1 ★★★★★ "A project is in UAT and the client asks for a last-minute change. How do you handle it?" – ఇది Infosys లో నిజంగా అడిగిన question

"My first move is not to say yes or no – it's to **understand the change and assess the impact** before committing to anything.

Concretely: I'd get clear on exactly what they want and, more importantly, **why** – sometimes what looks like a new feature is actually a misunderstanding of an existing one, or there's a simpler way to solve the underlying need. Then I'd estimate the real cost: development, regression testing, and what it touches. UAT is exactly when regression risk is highest, because the code has already been tested and signed off.

Then I'd take that to my lead or PM with **options rather than a problem** – typically three: we can take it in this release with a revised date; we can take a reduced version now that addresses the core need and the rest in the next sprint; or we can log it as a change request for the next release. The client gets to make a business decision with real information.

What I wouldn't do is commit on the spot, because a commitment I can't meet costs far more trust than an honest conversation about the trade-off. And whatever we agree, I'd make sure it's confirmed in writing through the proper change-request process – that protects both sides.

Having said that, I'd also weigh severity. If it's a genuine showstopper – something that would make the release unusable or breach a compliance requirement – that's a different conversation, and we'd absolutely find a way to accommodate it."

ఎందుకు ఇది **excellent answer**:

-  వెంటనే "yes" లేదా "no" అనలేదు – junior candidates ఏదీ ఒకటి చెప్తారు
-  **Impact assessment ముందు** – process maturity
-  **Options ఇచ్చాడు, decision escalate చేశాడు** – తనే decide చేసుకోలేదు (ఇది ముఖ్యం, 3 YOE కి decision authority లేదు)
-  **Change request process** mention – Infosys process-driven company, ఇది వాళ్ళ భాష
-  **UAT regression risk** – technical judgement
-  **Exception case** mention – rigid కాదు, judgement ఉంది

M2 ★★ "The client is unhappy with the delivery quality. What do you do?"

"I'd listen first and get specific – 'unhappy with quality' can mean bugs, missed requirements, or that it works but doesn't feel right, and those need completely different responses. I'd get concrete examples rather than defending.

Then I'd separate what's actually a defect from what's a gap in shared understanding – if a requirement was interpreted differently, that's not a blame conversation, it's a signal our clarification process was too thin. I'd fix the defects with clear priorities and dates, and communicate progress **proactively** rather than waiting to be asked, because at that point the client's confidence needs rebuilding as much as the code does.

Longer term, I'd look at what let it through – was there no acceptance criteria, no demo cadence, no test coverage on that path – and propose the fix. In my experience the single most effective thing is **shorter feedback loops**: a demo every sprint means the client corrects course in a week rather than at UAT."

M3 ★★ "The client asks you directly for something your manager hasn't approved. What do you do?"

"I'd acknowledge the request and take it seriously in front of the client – never dismiss it – but I wouldn't commit. Something like: 'That makes sense, let me check the impact on the current sprint and come back to you today.' Then I'd take it to my lead immediately.

The reason I wouldn't commit directly is that I don't have visibility into the whole plan – what looks free to me might collide with something else, and a commitment made outside the agreed scope creates a problem for the whole team. But I also wouldn't say 'I need to ask my manager' in a way that makes the client feel blocked. **The goal is to be responsive without being unilateral.**"

B. Deadline & pressure scenarios

M4 ★★ "You have a tight deadline and you know you can't finish everything. What do you do?"

"The one thing I wouldn't do is stay silent and hope. Late surprises are far more expensive than early bad news.

First I'd get an honest picture of what's actually left – not an optimistic one – and split it into what's essential for the release to function versus what's valuable but deferrable. Then I'd look at what can be reduced rather than dropped: a simpler implementation now with the full version later often gets 80% of the value.

Then I'd **raise it with my lead as early as I'm confident it's real**, with the options laid out: here's what I can definitely deliver by the date, here's what would need to slip, here's what would change if I got help on X. That lets them make a call – pull someone in, negotiate the date with the client, or descope – while there's still time for any of those to work.

And I'd be clear that I'm not asking them to solve it for me; I'm flagging it with a recommendation. **In my experience the thing managers actually get frustrated by isn't slipping a date – it's finding out on the deadline.**"

⚠️ **ఇక్కడ చెప్పకూడనివి:** "I'd work nights and weekends to finish it." – ఇది hero mentality, service company managers దీన్ని **risk** గా చూస్తారు (burnout, quality drop, single point of failure). Effort mention చెయ్యొచ్చు కానీ **primary answer communication + prioritisation అయి ఉండాలి.**

M5 ★★ "Production is down and you're on call. Walk me through what you do."

"Restore service first, find the root cause second – those are different priorities and mixing them up costs downtime.

Practically: confirm the scope – is it everyone or one region, which functionality. Communicate immediately in the incident channel so people aren't debugging blind or duplicating work. Then look for the fastest path to restore: if a recent deploy correlates with it, **roll back rather than trying to fix forward** under pressure. Check the obvious external causes – a dependency, a certificate, a full disk, a database connection pool.

Once service is restored, then do the actual root cause analysis properly, and write it up – timeline, cause, what we're changing so it can't recur. And be honest in the RCA; the point is the fix, not attributing it."

M6 ★★ "How do you prioritise when everything is urgent?"

"I'd push back gently on the premise – if everything is P1, nothing is, and that's usually a signal that prioritisation hasn't happened yet rather than that it's genuinely impossible.

My own filter is impact and blocking: what's blocking other people or the client comes first, because my delay multiplies into theirs. Then production issues affecting users. Then committed sprint work. Then everything else. And I'd distinguish urgent from important – a lot of what arrives labelled urgent is someone else's convenience.

If I genuinely can't tell, I'd rather spend five minutes asking my lead which of two things they'd want first than guess and be wrong for two days."

C. Conflict & team scenarios

M7 ★★☆☆ "Tell me about a disagreement with a colleague. How did you resolve it?"

STAR format వాడు:

[S] "A senior colleague and I disagreed on how to handle state for a new module. He wanted to add it to our existing Redux store; I wanted to keep it local to the feature with TanStack Query for the server data.

[T] "I was building the module, but he'd set up the store architecture and had good reasons for consistency.

[A] "Rather than argue in the standup, I asked him to spend twenty minutes with me on it. I started by making sure I actually understood his position – his concern was that scattered state patterns make onboarding harder and he'd seen that go badly before, which is a completely legitimate concern. Then I laid out mine: most of what I needed was server cache, and putting it in Redux meant hand-writing caching and invalidation we'd get for free otherwise.

"We didn't fully agree, so I suggested we timebox it – I'd build it my way for this one module, we'd review it at the end of the sprint, and if the team found it confusing we'd migrate it into the store. I also wrote a short doc on when to use which, so it was a documented decision rather than my personal preference.

[R] "The module shipped, the review went fine, and that doc became the pattern the team followed for the next two features. But honestly the more useful outcome was the working relationship – he later brought me into an architecture discussion I wouldn't otherwise have been in. **What I took from it is that most technical disagreements are actually about different risks, and once both risks are on the table the decision usually gets easier.**"

ఎందుకు excellent: ఎదుటివాడిని villain చెయ్యలేదు · అతని concern valid అని ఒప్పుకున్నాడు · public argument కాకుండా private conversation · **timebox + review** అనే practical resolution ·

M8 ★★ "A team member isn't delivering. What do you do?"

"As a peer rather than their manager, I'd start by assuming there's a reason I can't see. I'd talk to them directly and privately first – often it's that they're blocked on something and haven't wanted to say so, or the task was underspecified, or something outside work.

If I can help – pair on it, share context, unblock a dependency – I'd do that first, because that's faster than escalating and it doesn't damage the relationship.

If it continues and it's affecting the delivery, then I'd raise it with the lead – but framed around the **delivery risk rather than the person**: 'this piece is at risk, here's what I've tried.' That's their call to make, not mine, and going to a manager before talking to the person directly is how you lose a teammate's trust permanently."

M9 ★★ "Your manager gives you feedback you disagree with. How do you respond?"

"I'd listen fully before responding, and try to find the part that's true – even feedback that feels off usually has something real underneath it, and if several people see something I don't, the odds are on them.

I'd ask for a specific example, because 'be more proactive' is hard to act on and 'you didn't flag the API delay until the day before' is something I can actually fix.

If I still genuinely disagree after understanding it, I'd say so respectfully and with my reasoning – but once. Then I'd act on it. **Litigating feedback is a much worse look than acting on feedback I'm not sure about.**"

M10 ★★ "Have you mentored anyone?"

నీ నిజమైన అనుభవం చెప్పు. లేకపోతే: "Not formally, but I've onboarded two people onto our module – I wrote the setup documentation because I'd struggled with it myself, and I do a lot of the code review for the junior developer on our team. I try to review by asking questions rather than giving instructions, so they arrive at it themselves – that takes longer but it sticks." ⚠️ **3 YOЕ కి mentoring experience ఉంటే అది strong differentiator** – JL4 (Technology Analyst) కి push చెయ్యడానికి ఇది useful.

D. Technical judgement scenarios (managerial round లో వచ్చేవి)

M11 ★★★ "We have payment functionality in a React web app. How do we reuse it in a mobile app without rewriting?" → JS3 చూడు (పూర్తి model answer అక్కడ ఉంది). ఇది Infosys లో నిజంగా అడిగిన question.

M12 ★★ "How would you approach a legacy codebase with no documentation?"

"I'd resist the urge to rewrite anything for the first few weeks. Practically: start from the entry points and follow one real user journey end to end – that teaches you more than reading files alphabetically. Read the git history for the areas I'll touch; commit messages and PR discussions carry the reasoning that's missing from the docs. Talk to whoever's been there longest, early, because that knowledge walks out of the door eventually.

Then I'd write down what I learn as I go – even a rough architecture note is more than what exists. And I'd add tests around anything I'm about to change, because in an undocumented codebase tests are how you find out what the behaviour actually was.

The mistake I'd avoid is proposing a rewrite in month one. Code that looks bad often encodes years of edge cases that nobody remembers, and the person who suggests rewriting it usually hasn't found them yet."

M13 ★★ "How do you keep up with technology?"

Specific గా చెప్పు, generic కాదు. "I follow the React and TypeScript release notes directly rather than secondhand summaries, read a few engineering blogs regularly, and I build something small with anything I'm evaluating – I don't trust my opinion on a tool I haven't used. Recently that's been [something real]. On the team side, we do a short knowledge-sharing session every couple of weeks, which I've presented in twice." ⚠️

"I watch YouTube tutorials" అని మాత్రమే చెప్పకు. Specific sources + నువ్వు ఏదో కట్టావు అనేది చూపించు.

M14 ★★ "How do you ensure code quality in your team?"

"Layered, so nothing depends on people remembering. Automated: ESLint and Prettier on commit hooks, TypeScript in strict mode, and CI running lint, type-check, tests and build on every PR – if it's automated, it's not a conversation. Human: PR review with at least one approval, and I try to review for design and edge cases rather than style, since the tooling handles style. Process: acceptance criteria agreed before work starts, because most 'quality' problems I've seen were actually requirement problems."

E. Fit & motivation

M15 ★★★ "Why do you want to join Infosys?"

Specific గా చెప్పు, flattery కాదు. "Two reasons. First, scale and exposure – my current work is a single internal product, and I want to work on systems with real client stakes and larger user bases. Infosys works across a lot of domains, and that variety is something you can't get on a single product. Second, structure – I've been in a fairly ad-hoc environment, and I want to work somewhere with proper engineering process, code review discipline and defined career progression. And frankly, the client-facing exposure is something I want to build, because I think that's what separates a developer from someone who can lead delivery." ❌ **అనకూడనివి:** "Infosys is a big company", "for job security", "everyone knows Infosys".

M16 ★★ "Why are you leaving your current company?"

ఎప్పుడూ positive framing. ఎప్పుడూ current employer ని తిట్టకు. ✅ "I've been on the same product for over two years and I've learnt what there was to learn from it. I'm looking for larger scale and more client-facing work, and that opportunity isn't available where I am right now." ❌ "My manager is terrible" / "No hike" / "Too much workload" / "Politics" ⚠️ Salary నిజమైన కారణమైనా – **అది primary reason గా చెప్పకు.** Growth ని primary చెయ్యి, compensation ని secondary గా mention చెయ్యొచ్చు: "Compensation is part of it, honestly, but the bigger driver is the kind of work."

M17 ★★ "Where do you see yourself in 5 years?"

"In five years I'd want to be leading the technical direction of a module or a small team – still hands-on with code, but owning architecture decisions and mentoring. I'm not looking to move fully into management; I'd rather grow on the technical track. Practically that means getting deeper on system design and getting much better at the client-facing side, which is part of what attracted me here."

M18 ★★ "Are you comfortable with relocation / rotational shifts / working from office?"

Flat "no" చెప్పకు – flexibility చూపించు, కానీ నీ constraints నిజాయితీగా చెప్పు. "My preference is [location], and I'm currently based there. For the right role I'm open to relocating – I'd just want reasonable notice to plan it. On shifts, I'm completely fine with overlapping with client hours; if it's a permanent night shift I'd want to understand the details before committing, but I'm not ruling it out."

M19 ★★ "What are your strengths and weaknesses?"

Strength – ఒకటి, evidence తో: "Debugging complex issues. I'm methodical about it – I reproduce and narrow down rather than guessing, and I've ended up being the person the team brings in when something is intermittent. The performance issue I mentioned earlier was a good example."

Weakness – నిజమైనది, కానీ fatal కాదు, మరియు నువ్వు దానిపై పని చేస్తున్నావు: "I've historically gone too deep too early – spending a day making something clean when a rough version would have surfaced a requirement problem in an hour. I've been consciously working on shipping something rough for feedback first. It's better than it was, but it's still my instinct." ❌ అనకూడనివి: "I'm a perfectionist" / "I work too hard" / "I care too much" – interviewers ఇవి rehearsed non-answers అని instantly గుర్తిస్తారు.

M20 ★ "Do you have any offers in hand?" · **M21** ★ "What is your notice period?" → §29, §30 (HR round) చూడు.

M22 ★ "What if you're put on a project with a different technology stack – say Angular instead of React?"

⚠️ **Infosys కి ఇది ముఖ్యమైన question** – projects change, bench allocation ఉంటుంది. "I'd be fine with it. The framework is the surface – component models, state, data flow and the underlying JavaScript are the same problems, and Angular's structure isn't foreign to me conceptually. I'd want a reasonable ramp-up, and I'd expect to be slower for the first few weeks. My preference is React, and I'd say so, but I wouldn't turn down a project over it."

M23 ★ "Do you have any questions for me?"

ఎప్పుడూ "no" అనకు. Managerial round కి ఇవి:

1. "What does the project and the team look like – size, domain, how long has it been running?"
2. "What would success look like for this role in the first six months?"
3. "How does the team handle knowledge sharing and technical decisions?"
4. "What's the biggest challenge the team is facing right now?" ← ఇది బాగా impress చేస్తుంది

M24 ★ "Tell me about a time you took initiative." → Story 5 (§25) వాడు.

Key Points

- ★★★★★ questions: **M1, M4, M7, M11, M15, M16** – వీటికి తప్పకుండా answers ready చెయ్యి.
- Client questions లో **process (change request, escalation)** mention చెయ్యి – Infosys భాష అది.
- Deadline questions లో **communication first**, "I'll work weekends" కాదు.
- Conflict questions లో ఎదుటివాడి **concern valid** అని ఒప్పుకో.

- "Why leaving" లో ఎప్పుడూ **positive**.

27. ఈ round లో reject అయ్యే 6 తప్పులు

వివరణ

Managerial round లో technical knowledge test కాదు – **judgement** test. ఈ 6 తప్పులు చేస్తే technical round ఎంత బాగా చేసినా reject అవుతావు.

1. "నాకు ఎప్పుడూ conflict రాలేదు / ఎప్పుడూ deadline miss అవ్వలేదు / ఎప్పుడూ mistake చెయ్యలేదు."

Manager దీన్ని రెండిట్లో ఒకటిగా చదువుతాడు: నువ్వు నిజం చెప్పలేదు, లేదా నువ్వు నిజమైన responsibility ఉన్న పని చెయ్యలేదు. **రెండూ చెడ్డవే**. ప్రతి real project లో conflict, slip, mistake ఉంటాయి. **ఒక నిజమైనది ready ఉంచు**.

2. Blame ఇతరుల మీద వెయ్యడం.

"The requirements were bad", "the backend team was late", "QA missed it" – ఇవి నిజమైనా, **నీ answer లో focus నువ్వు ఏం చేశావు అనే దాని మీద ఉండాలి**. Manager వినాలనుకున్నది: "అది నా control లో లేదు, కాబట్టి నేను X చేశాను."

3. Hero mentality.

"I worked three weekends and finished it alone." Manager మనసులో: "ఇతను *deadline risk* ని ముందే *flag* చెయ్యలేదు. ఇతను *bus factor* 1. ఇతను *burnout* కి వెళ్ళాడు." **Effort admirable, కానీ process failure.**

4. Rigidity.

Location కి "no", shift కి "no", technology change కి "no", "I only work on React". Infosys ఒక **staffing-driven company** – projects మారతాయి. Flexibility లేకపోతే వాళ్ళు నిన్ను allocate చెయ్యలేరు.

5. Current company/manager ని తిట్టడం.

"My manager doesn't know anything", "the codebase is garbage", "nobody works there". Manager అలోచించేది: "ఇతను 2 సంవత్సరాల తర్వాత మా గురించి కూడా ఇలాగే మాట్లాడతాడు." **ఒక్క negative వాక్యం కూడా వద్దు.**

6. Structure లేకుండా rambling.

Question అడిగినప్పుడు 4 నిమిషాలు అటూ ఇటూ మాట్లాడి, point చెప్పకుండా ముగించడం.
STAR వాడు. 90 seconds. Interviewer ఇంకా కావాలంటే అడుగుతాడు.

Key Points

- ఈ 6 తప్పులు = **technical score ని రద్దు చేస్తాయి.**
- Common thread: **ownership, honesty, structure, flexibility.**
- Managerial round లో నీ ఏకైక message: "నేను responsible, calm, clear, flexible."

Part 4 – Round 3: HR Round & Negotiation

28. HR round questions + answers

వివరణ

HR round **15-30 minutes**. ఇక్కడ technical evaluation లేదు – ఇది **compensation, logistics, మరియు final fit**. కానీ ఇక్కడ కూడా reject అవ్వొచ్చు (attitude, rigidity, unrealistic demands).

ముఖ్యమైన mindset shift: ఈ round కి వచ్చావంటే వాళ్ళు నిన్ను తీసుకోవాలనుకుంటున్నారు. ఇప్పుడు నీ position బలంగా ఉంది. కానీ **desperate గా కనిపించకు, arrogant గా కూడా కాదు.**

Questions + answers

H1. "Tell me about yourself." → §8 pitch, కానీ **technical detail తగ్గించి** – role, ownership, team, career direction మీద focus.

H2. "Why do you want to leave your current job?" → §M16. **Positive framing మాత్రమే.**

H3. "Why Infosys?" → §M15.

H4. "What is your current CTC?"

నిజం, breakup తో: "X lakhs – that's Y fixed and Z variable." **అబద్ధం చెప్పకు** – payslips + Form 16 verify చేస్తారు, పట్టుబడితే offer revoke.

H5. "What is your expected CTC?" → §29 (పూర్తి script అక్కడ).

H6. "What is your notice period? Can you join earlier?"

"It's 60 days officially. I can explore a buyout to bring it down to around 30-45 days – I'd need to confirm with my current employer, but it's usually possible." **Buyout mention చెయ్యడం value add**, ఎందుకంటే Infosys కి fast onboarding కావాలి. ⚠️ **Buyout ఎవరు pay చేస్తారు అని అడుగు:** "Would Infosys support the buyout, or would that be on me?" – కొన్నిసార్లు వాళ్ళు cover చేస్తారు. **అడగకపోతే తెలియదు.**

H7. "Are you open to relocation? Which location do you prefer?" → §M18. Flexibility చూపించు.

H8. "Do you have any other offers?"

ఉంటే – leverage గా వాడు కానీ threat గా కాదు: "I do have one other offer in progress. That said, this role interests me more because of [specific reason], so I'd like to move forward here." లేకపోతే – **అబద్ధం చెప్పకు**. "I'm in discussions with a couple of other companies" (నిజమైతే), లేదా simply "Not currently – I've been selective about where I apply."

H9. "When can you join?"

Realistic date చెప్పు. Over-promise చేసి తర్వాత delay చేస్తే credibility పోతుంది.

H10. "Are you comfortable with the shift timings / work from office policy?"

Infosys ఇప్పుడు ఎక్కువగా **office-based / hybrid**. WFH expect చేస్తే **ఇప్పుడే clarify చెయ్యి** – offer accept చేశాక తెలుసుకోవడం worst case.

H11. "Do you have any questions?" – ఇవి అడుగు:

1. "Could you walk me through the CTC breakup – fixed, variable, and what the variable is tied to?"
2. "What's the designation and job level for this role?"
3. "Is there a probation period for lateral hires?"
4. "What's the appraisal cycle, and when would I be eligible for the first review?"
5. "What's the work model – how many days in office?"
6. "What does the joining formality timeline look like – BGV, documents, offer release?"

Gotchas (సాధారణ తప్పులు)

1. **Desperate గా కనిపించడం.** "I'll take whatever you offer" – negotiation power నున్నా అవుతుంది, మరియు వాళ్ళు నిన్ను తక్కువగా value చేస్తారు.
2. **Arrogant గా ఉండటం.** "I have 3 offers, match them or I'm out" – service company HR దీన్ని ఇష్టపడరు.
3. **CTC గురించి అబద్ధం.** Form 16 verify చేస్తారు. **Instant disqualification risk.**
4. **Variable ని fixed అనుకోవడం. Breakup అడుగు.**
5. **Questions అడగకపోవడం.** ఇది disinterest signal.

Key Points

- ఈ round కి వచ్చావంటే వాళ్ళకి నువ్వు కావాలి – position బలంగా ఉంది.
- CTC: **నిజం + breakup.**

- Notice period: **buyout option + ఎవరు pay చేస్తారని అడుగు.**
- **Breakup, designation, probation, appraisal cycle** – ఈ 4 తప్పకుండా అడుగు.

29. Salary negotiation – script + numbers

వివరణ

ఇది ఒక్క conversation, కానీ నీ తర్వాతి 2-3 సంవత్సరాల salary ని decide చేస్తుంది – ఎందుకంటే future hikes అన్నీ ఈ base మీద % గా వస్తాయి. **15 minutes prep, lakhs of impact.**

Real-life Scenario

Negotiation ని ఒక **tug of war** అనుకోవద్దు. ఇది ఒక **joint problem solving** – "మనం ఇద్దరం ఈ deal ని ఎలా work చెయ్యగలం?" అనే framing.

HR కి ఒక **band** ఉంటుంది (JL3A కి ఇంత నుండి ఇంత వరకు). వాళ్ళ job ఆ band లో తక్కువగా close చెయ్యడం. నీ job ఆ band లో **పైన** close చెయ్యడం. **Band దాటి అడిగితే conversation ముగుస్తుంది** – negotiate అవ్వదు.

నీ numbers ముందే లెక్కించు

Negotiation call కి ముందు ఈ 4 numbers రాసుకో:

–	ఏమిటి	ఉదాహరణ (current ₹8L అనుకుంటే)
Current CTC	Fixed + variable breakup తో	₹8L (7L fixed + 1L variable)
Target	నువ్వు నిజంగా కోరుకునేది (~40% hike)	₹11.5L
Ask (anchor)	Target కంటే కొంచెం ఎక్కువ	₹13L
Walk-away	దీని కంటే తక్కువ అయితే వద్దు	₹9.5L

Anchor ఎందుకు target కంటే ఎక్కువ: negotiation ఎప్పుడూ కిందికే వెళ్తుంది. 13 అడిగితే 11.5 వద్ద settle అవుతుంది. 11.5 అడిగితే 10 వద్ద settle అవుతుంది. **కానీ 25 అడగకు** – అది band దాటి, unserious గా కనిపిస్తుంది. **20-40% hike = credible range.**

Script – word by word

Step 1 – HR: "What are your expectations?"

"Based on my three years of React and full-stack experience, and what I've seen for this level in the market, I'm looking at **13 lakhs fixed**. I'm open to discussing the structure – I care more about the fixed component than the total number."

⚠️ **"Fixed" అనే word వాడు.** HR "12 lakhs CTC" అని చెప్పి అందులో 2L variable పెట్టొచ్చు. **Fixed మీద negotiate చెయ్యి.**

Step 2 – HR: "That's above our range for this level. We can do 10."

వెంటనే ఒప్పుకోకు. వెంటనే **refuse** కూడా చెయ్యకు. ఒక క్షణం ఆగి: "I understand there's a band. Could you tell me what the range is for this level? I'd like to understand where 10 sits within it – if there's room to move toward the upper end, I think we can make this work."

ఇది ఎందుకు పని చేస్తుంది: నువ్వు వాళ్ళ constraint ని ఒప్పుకున్నావు (adversarial కాదు), కానీ **information** అడిగావు. HR చాలాసార్లు band top ని చెప్తారు – అప్పుడు నీకు target తెలుస్తుంది.

Step 3 – HR: "10 is what we can do."

మూడు levers ఉన్నాయి. ఒక్కటి కాదు – అన్నీ try చెయ్యి:

(a) Level: "I understand the band is a constraint. Would this role be considered for **Technology Analyst (JL4)** rather than Senior Systems Engineer? Given that I've been owning a module end to end and doing code review for the team, I think the scope matches – and that would change the band."

(b) Joining bonus: "If the fixed can't move, is there room for a **joining bonus** to bridge the gap? Even a one-time component would help, especially if I'm buying out my notice period."

(c) Notice buyout: "Would Infosys cover the notice period buyout? That's a real cost on my side and it also gets me to you faster."

(d) Early review: "If we go with 10 now, would it be possible to have a **review at six months** rather than waiting for the annual cycle?"

ఈ 4 లో కనీసం ఒకటి పని చేస్తుంది. Fixed rigid అయినా, level/bonus/buyout లో movement ఉంటుంది.

Step 4 – Close

"That works for me. Could you send the detailed breakup – fixed, variable, and what the variable is tied to – so I can review it? And could you confirm the designation as well?"

⚠ **Verbal offer మీద resign చెయ్యకు. Written offer letter వచ్చేవరకు current job వదలకు.** ఇది non-negotiable rule.

Negotiation rules

1. **మొదట number చెప్పకు... అని అందరూ చెప్తారు, కానీ Infosys screening call లో వాళ్ళు అడుగుతారు.** అప్పుడు **range + justification** ఇవ్వు. Deflect చెయ్యడానికి ప్రయత్నిస్తే recruiter ఇబ్బంది పడతాడు.
2. **Silence వాడు.** వాళ్ళు number చెప్పాక **3-4 seconds మౌనంగా ఉండు.** అసాకర్యంగా ఉంటుంది కానీ చాలాసార్లు వాళ్ళే improve చేస్తారు.
3. **ఎప్పుడూ "I need" కాదు, "the market is" / "the scope is".** Personal need కాదు, **value justification.**
4. **వెంటనే accept చెయ్యకు.** మొదటి offer కి కూడా – "Let me review the breakup and come back tomorrow" అను. ఇది ఒక్క రోజు, కానీ ఎప్పుడూ నష్టం చెయ్యదు.
5. **Politely persistent.** ఒకసారి push చెయ్యి, రెండోసారి soft గా push చెయ్యి. **మూడోసారి వద్దు** – అది relationship damage.
6. **Walk-away number తెలిసి ఉండు.** అది లేకపోతే నువ్వు negotiate చెయ్యట్లేదు, అంగీకరిస్తున్నావు.

Gotchas (సాధారణ తప్పులు)

1. **CTC vs in-hand confusion.** ₹12L CTC $\approx$ ₹80-85K/month in-hand. **Breakup అడిగి calculate చెయ్యి.**
2. **Variable ని లెక్కలో వెయ్యడం.** Variable = performance + company results. **Fixed మీద plan చెయ్యి.**
3. **Band దాటి అడగడం.** 25 LPA అడిగితే call అక్కడితో ముగుస్తుంది (ఒక candidate 40 LPA చెప్పి "we can't match that" విన్నాడు).
4. **Verbal offer నమ్మి resign చెయ్యడం. Written offer letter మాత్రమే.**
5. **Level negotiate చెయ్యకపోవడం.** Salary rigid అయినా level move అవ్వచ్చు, మరియు **level = అన్ని future hikes.**

Key Points

- **4 numbers ముందే రాసుకో:** current, target, ask, walk-away.
- **Hike range: 20-40%** current మీద. Ask = target + కొంచెం.
- **"Fixed" మీద negotiate చెయ్యి,** CTC మీద కాదు.
- **4 levers:** level, joining bonus, notice buyout, early review.
- **Written offer వచ్చేవరకు resign చెయ్యకు.**

30. Documents checklist & offer timeline

వివరణ

Infosys **background verification (BGV)** చాలా strict – service company, client compliance requirements ఉంటాయి. **Documents ముందే ready ఉంచడం** offer timeline ని వారాల్లో తగ్గిస్తుంది.

Documents checklist – ఇప్పుడే ఒక folder లో పెట్టు

Employment (అత్యంత ముఖ్యం):

- ☐ **Last 3-6 months payslips (PDF)**

- ☐ **Form 16** – గత 2 సంవత్సరాలవి
- ☐ Current offer letter / appointment letter
- ☐ Latest increment / promotion letter (ఉంటే)
- ☐ Previous employers: relieving letter + experience letter (**ప్రతి company నుండి**)
- ☐ Bank statement – salary credit చూపించేది (3-6 months)

Identity:

- ☐ PAN card
- ☐ Aadhaar card
- ☐ Passport (ఉంటే – onsite కి ముఖ్యం)
- ☐ Address proof

Education:

- ☐ 10th, 12th marksheets + certificates
- ☐ Degree certificate + all semester marksheets
- ☐ Provisional certificate (degree ఇంకా రాకపోతే)

Other:

- ☐ Passport size photos
- ☐ UAN / PF number
- ☐ Certifications (ఉంటే)

⚠ BGV warnings:

- **Employment dates exact గా match అవ్వాలి.** Resume లో "Jan 2023 - present" అని రాసి, relieving letter లో "15 Feb 2023" ఉంటే – **discrepancy flag** అవుతుంది. Resume ని documents తో match చెయ్యి.
- **Gap ఉంటే ముందే explain చెయ్యి.** దాచివేద్దు – BGV లో బయటపడుతుంది, అప్పుడు అది "lied" అవుతుంది, "had a gap" కాదు.
- **Previous employer నుండి relieving letter లేకపోతే** – ఇది real problem. Payslips + bank statement + Form 16 alternative గా పని చెయ్యొచ్చు, కానీ ముందే HR కి చెప్పు.
- **CTC verify అవుతుంది** Form 16 ద్వారా. అబద్ధం = offer revoke.

Offer timeline – realistic expectations

Candidates report చేసిన timelines (ఇవి anecdotal, official policy కాదు):

Stage	Typical time
HR round → salary negotiation	అదే call లో లేదా 2-3 రోజులు
Negotiation → scholastic/documents form	1-2 రోజులు (email లో వస్తుంది)

Stage	Typical time
Documents submit → offer letter	1-4 వారాలు (BGV మీద ఆధారపడి)
మొత్తం application → offer	2-8 వారాలు

Common experiences: కొందరికి HR discussion తర్వాత 1 వారంలో offer letter వచ్చింది. కొందరికి scholastic form email లో వచ్చి, fill చేసి పంపాక 14 రోజుల్లో వచ్చింది. కొందరికి 3-4 వారాలు పట్టింది (BGV ముందు, తర్వాత offer).

Infosys offers "business requirement" ఆధారంగా release చేస్తుంది – అంటే project allocation confirm అయ్యాక. అందుకే delay కావచ్చు.

ఏం చెయ్యాలి – practical advice

1. **Documents ఇప్పుడే ready చెయ్యి.** Interview call వచ్చాక వెతకడం మొదలుపెడితే 1 వారం పోతుంది.
2. **వేరే companies తో interview కొనసాగించు** offer letter చేతికి వచ్చేవరకు. ఇది **backup కాదు, leverage కూడా.**
3. **HR ని వారానికి ఒకసారి polite గా follow up చెయ్యి.** ప్రతిరోజూ కాదు. Email లో, short గా.
4. **Offer letter వచ్చాక carefully చదువు** – designation, fixed, variable, joining date, location, bond/service agreement ఉందా.
5. **Written offer వచ్చాకే resign చెయ్యి.** ఇది మళ్ళీ చెప్తున్నాను ఎందుకంటే చాలామంది ఇక్కడ దెబ్బతింటారు.

Key Points

- **BGV strict** – documents exact గా match అవ్వాలి.
- Resume dates = relieving letter dates. **Discrepancy = red flag.**
- Timeline: **HR → offer = 1-4 వారాలు** (variable).
- **Documents folder ఇప్పుడే తయారు చెయ్యి.**
- **Written offer letter వచ్చేవరకు current job వదలకు.**

Part 5 – Execution

31. 4-week study plan

వివరణ

అత్యంత ముఖ్యమైన principle: నీ prep time ని **probability ప్రకారం** allocate చెయ్యి, comfort ప్రకారం కాదు. చాలామంది DSA చేస్తారు ఎందుకంటే అది measurable గా అనిపిస్తుంది. కానీ §7 table ప్రకారం **project + React + JS = 80% weight, DSA = 30% probability.**

సరైన allocation:

Area	Time %	ఎందుకు
Project prep + STAR stories	25%	Interview లో అతిపెద్ద block, ఎవరూ prep చెయ్యరు
React (§13, §14)	25%	~95% probability
JavaScript (§10, §11, §12)	20%	~90% probability
Redux + state (§15)	8%	~80% probability
Managerial (§26)	8%	Reject rate ఎక్కువ
Live coding practice (§18)	6%	~50% probability
DSA (§23)	4%	~30%, low depth
Git, CSS, API, Node, SQL	4%	Fill gaps

Week 1 – Foundations + Project

రోజు	ఎం చెయ్యాలి	Section
Day 1	§1-§4 చదువు (process, JL, full-stack trap). Recruiter ని stack confirm చెయ్యి. Resume update.	§1-§4
Day 2	Project one-pager రాయ – architecture, ownership, 6 numbers, 12 follow-up answers	§9
Day 3	5 STAR stories రాయ (challenge, deadline, conflict, failure, initiative). Loud గా practice.	§25
Day 4	90-second pitch రాయ + 10 సార్లు practice, timer తో	§8
Day 5	JavaScript: scope, hoisting, closures, <code>this</code> , <code>call/apply/bind</code>	§10 A
Day 6	JavaScript: event loop, promises, async/await + 12 puzzles solve చెయ్యి	§10 C, §11
Day 7	Revision + self-test. Day 1-6 లోని అన్నిటినీ loud గా చెప్పు, ఎవరూ లేకపోయినా.	–

Week 2 – React deep

రోజు	ఎం చెయ్యాలి	Section
Day 8	React Level 1 (R1-R12) – అన్నీ instant recall వచ్చేవరకు	§13 L1
Day 9	React Level 2 (R13-R24). R18 (performance) + R24 (state) ప్రత్యేకంగా	§13 L2

రోజు	ఏం చెయ్యాలి	Section
Day 10	React Level 3 (R25-R34) + <code>React_Telugu.md</code> నుండి Fiber, reconciliation	§13 L3
Day 11	Scenario questions S1-S7 – loud గా answer చెయ్యి, రాయకు	§14
Day 12	Redux Toolkit – cart app కట్టు (slice + async thunk + selector). Redux రాకపోతే ఇది తప్పనిసరి.	§15
Day 13	Polyfills – debounce, throttle, deepClone, reduce, bind చేతితో రాయి (చూడకుండా)	§12
Day 14	Mock Interview 1 (§34) – ఒక స్నేహితుడితో లేదా record చేసుకుని	§34

Week 3 – Practice + breadth

రోజు	ఏం చెయ్యాలి	Section
Day 15	Live coding: CodeSandbox లో Task 1-4, timer తో, loud గా. 2 సార్లు.	§18
Day 16	Live coding మళ్ళీ + JavaScript array/object methods, ES6	§18, §10 D/E
Day 17	Managerial: M1, M4, M7, M11, M15, M16 – answers loud గా practice	§26
Day 18	Managerial మిగిలినవి + §27 (6 mistakes) review	§26, §27
Day 19	Git + CSS + API/HTTP – ఒక్క రోజు, breadth కోసం	§16, §17, §21
Day 20	Node/Express/Mongo + SQL (Full Stack అయితే). Frontend-only అయితే DSA.	§19, §20, §23
Day 21	Mock Interview 2 (§34) – full 1 hour	§34

Week 4 – Polish + gaps

రోజు	ఏం చెయ్యాలి
Day 22	Mock 2 లో దొరికిన gaps మీద పని
Day 23	DSA – strings, arrays, hash maps. 8-10 easy problems.
Day 24	Testing (§22) + Security (§21 A7) + మిగిలిన gaps
Day 25	HR round + negotiation – §29 script practice. 4 numbers లెక్కించు.

రోజు	ఏం చెయ్యాలి
Day 26	Documents folder ready చెయ్యి (§30). Mock Interview 3 (managerial + HR).
Day 27	Full revision – §35 rapid-fire cheat sheet + project one-pager + 5 STAR stories
Day 28	Light day. Pitch + cheat sheet మాత్రమే. కొత్తది నేర్చుకోకు. బాగా నిద్రపో.

Gotchas (సాధారణ తప్పులు)

1. **Reading మాత్రమే చెయ్యడం.** ఈ doc చదవడం prep కాదు – **loud గా answer చెప్పడం** prep. చదవడం 30%, మాట్లాడటం 70%.
2. **Project prep ని చివరికి వాయిదా వేయడం.** అది **Day 2** లో ఉంది ఒక కారణంతో.
3. **DSA rabbit hole.** 4 రోజులు LeetCode చేస్తే – తప్పు allocation.
4. **Mock interview skip చెయ్యడం.** ఇది అత్యంత ముఖ్యమైన activity. Silent prep vs speaking prep తేడా చాలా పెద్దది.
5. **కొత్త topics ని చివరి 3 రోజుల్లో మొదలుపెట్టడం.** అది confidence తగ్గిస్తుంది. **Week 4 = polish only.**

32. 1-week crash plan (interview దగ్గరపడితే)

7 రోజులు మాత్రమే ఉంటే – **breadth వదిలేసి, highest-probability items మీద మాత్రమే.**

రోజు	ఏం చెయ్యాలి (4-5 గంటలు)
Day 1	Project one-pager (§9) + 90-second pitch (§8). ఇవి రెండూ non-negotiable.
Day 2	5 STAR stories (§25) + M1, M4, M7, M15, M16 (§26)
Day 3	JavaScript §10 A + C (closures, <code>this</code> , hoisting, event loop) + §11 puzzles 1-6
Day 4	React Level 1 (R1-R12) – అన్నీ. + R18 (performance), R24 (state)
Day 5	Scenarios S1, S2, S3 (§14) + Redux D1-D5 (§15) + debounce/throttle (§12)
Day 6	Live coding Task 1-3 (§18), CodeSandbox లో, timer తో, 2 సార్లు. + §35 cheat sheet
Day 7	Full mock (§34 Mock 1) + revision. కొత్తది నేర్చుకోకు.

Crash plan లో వదిలేసేవి: Fiber internals, Node deep dive, SQL beyond basics, testing, DSA beyond 3-4 problems, CSS beyond flexbox/grid. **వచ్చినా – "I haven't worked with that in depth" అని నిజాయితీగా చెప్పు.**

33. Interview ముందు రోజు + ఆ రోజు checklist

ముందు రోజు

- ☐ **కొత్తది ఏమీ నేర్చుకోకు.** Revision మాత్రమే: pitch, project one-pager, 5 STAR stories, §35 cheat sheet

- ☐ **Setup test చెయ్యి:** Microsoft Teams install + login, camera, mic, headphones, internet backup (mobile hotspot)
- ☐ **CodeSandbox తెరిచి ఒక component రాయి** – live coding వస్తే ready
- ☐ Resume ని మళ్ళీ చదువు – అందులో ఉన్న ప్రతి skill గురించి అడగొచ్చు
- ☐ Interview link, time, timezone confirm చెయ్యి
- ☐ Quiet room, decent lighting arrange చెయ్యి (light నీ ముఖం మీద పడాలి, వెనుక కాదు)
- ☐ నీ questions list ready ఉంచు (§7, §M23, §H11)
- ☐ **7-8 గంటలు నిద్రపో**

ఆ రోజు

- ☐ **30 minutes ముందు** setup ready చెయ్యి – link click చెయ్యడానికి కాదు, test చెయ్యడానికి
- ☐ **10 minutes ముందు** join చెయ్యి
- ☐ నీ ముందు ఉంచు: project one-pager, §35 cheat sheet, water
- ☐ Phone silent, notifications off, ఇతర tabs close
- ☐ **Camera ON**
- ☐ Pen + paper (coding/design question కి draw చెయ్యడానికి)

Interview సమయంలో

- ☐ **Question పూర్తిగా విను**, ఒక క్షణం ఆగు, తర్వాత answer
- ☐ **60-90 seconds per answer** – ఎక్కువ కాదు
- ☐ తెలియకపోతే: "I haven't worked with that directly, but based on my understanding of X, I'd approach it this way..." – **bluff చెయ్యకు**
- ☐ ప్రతి technical answer లో నీ **project నుండి ఒక example**
- ☐ Coding లో **loud గా ఆలోచించు**
- ☐ చివర్లో **తప్పకుండా questions అడుగు**
- ☐ **Thank you చెప్పు**, interviewer పేరు వాడి

Interview తర్వాత

- ☐ **వెంటనే రాసుకో** – ఏం అడిగారు, నువ్వు ఏం తడబడ్డావు (తర్వాతి round కి ఇది బంగారం)
- ☐ Recruiter కి short thank-you email (optional కానీ positive)
- ☐ **Rejection వచ్చినా** – ఆ notes తో తర్వాతి interview కి prepare అవ్వ

34. Mock interview scripts

వివరణ

ఇది ఈ doc లో అత్యంత ముఖ్యమైన section. చదవడం వల్ల knowledge వస్తుంది; **loud గా చెప్పడం వల్ల మాత్రమే fluency వస్తుంది**. Interview లో నువ్వు test అవుతున్నది knowledge కాదు – **pressure లో knowledge ని articulate చెయ్యగలగడం**.

ఎలా చెయ్యాలి: ఒక స్నేహితుడిని అడుగు, లేదా **phone లో record చేసుకుని** ఒక్కడివే చెయ్యి – తర్వాత విను. Recording వినడం అసౌకర్యంగా ఉంటుంది కానీ **అదే అత్యంత ఉపయోగకరమైన feedback**.

Mock 1 – Technical Round (60 minutes)

[0-5 min]

1. "Tell me about yourself."
2. "Walk me through your current project."

[5-25 min] – Project deep dive 3. "What was your specific contribution to this project?" 4. "Why did you choose Redux over Context API?" (లేదా నీ actual choice) 5. "What was the biggest technical challenge you faced?" 6. "How did you handle API errors and loading states?" 7. "How did you handle authentication?" 8. "How many users does this serve? What's the data scale?" 9. "What would you do differently if you rebuilt it today?"

[25-45 min] – Technical 10. "Explain closures with an example." 11. "What's the difference between `var`, `let` and `const`?" 12. "How does the event loop work?" → అప్పుడు: "What does this print?" **[Puzzle 3, \$11]** 13. "Explain the virtual DOM and reconciliation." 14. "Why do we need keys in lists? Can I use the index?" 15. "What's the difference between `useMemo`, `useCallback` and `React.memo`?" 16. "How would you implement `componentWillUnmount` in a functional component?" 17. "A list of 10,000 items is rendering slowly. How do you fix it?" **[\$1]** 18. "What's the difference between debounce and throttle? Write debounce." 19. "Explain `merge` vs `rebase` in Git."

[45-55 min] – Live coding 20. "Write a component that fetches users from an API and displays them in a list." **[Task 1, \$18]** 21. Follow-up: "Now add a search box that filters as the user types, without hitting the API on every keystroke."

[55-60 min] 22. "Do you have any questions for me?"

Mock 2 – Managerial Round (30 minutes)

1. "Tell me about yourself." (ownership/team version, \$8 Interview దృష్టి)
2. "Walk me through your role in your current project – how big is the team, who do you report to?"
3. "Tell me about a challenging situation in a project and how you handled it." **[Story 1]**
4. "A project is in UAT and the client requests a last-minute change. How would you handle it?" **[M1]**
5. "Tell me about a time you missed a deadline." **[Story 2]**
6. "Tell me about a disagreement with a colleague." **[Story 7 / M7]**
7. "Tell me about a mistake you made." **[Story 4]**
8. "We have payment functionality in a React web app and want to reuse it in a mobile app without rewriting it. How?" **[M11 / S3]**
9. "How do you prioritise when everything is urgent?" **[M6]**
10. "Have you mentored anyone?" **[M10]**

11. "How do you keep up with new technology?" [M13]
12. "What if you're allocated to an Angular project instead of React?" [M22]
13. "Why do you want to leave your current company?" [M16]
14. "Why Infosys?" [M15]
15. "Are you open to relocation and rotational shifts?" [M18]
16. "Do you have any questions for me?"

Mock 3 – HR Round (20 minutes)

1. "Tell me about yourself."
2. "Why are you looking for a change?"
3. "What is your current CTC? Fixed and variable?"
4. "What are your expected CTC?" → **§29 script వాడు, negotiation ని పూర్తిగా play out చెయ్యి**
5. "That's above our band. We can offer 10." → **push back practice చెయ్యి**
6. "What is your notice period? Can you join earlier?"
7. "Are you comfortable with our office location and work-from-office policy?"
8. "Do you have any other offers?"
9. "When can you join?"
10. "Do you have any questions?"

Self-evaluation – recording విన్నప్పుడు ఇవి check చెయ్యి

□	ఏమిటి
□	ప్రతి answer 90 seconds లోపు ఉందా?
□	"we" కంటే "I" ఎక్కువసార్లు వాడావా?
□	ప్రతి technical answer లో ఒక project example ఉందా?
□	ప్రతి STAR story లో Result + number ఉందా?
□	"umm", "like", "basically" ఎన్నిసార్లు? (లెక్కపెట్టు – ఇది తగ్గించు)
□	తెలియని question కి నిజాయితీగా చెప్పావా, bluff చేశావా?
□	Trade-off mention చేశావా, లేక ఒక్క solution మాత్రమే చెప్పావా?
□	Coding లో loud గా narrate చేశావా?
□	చివర్లో questions అడిగావా?

ఒక్క mock కంటే మూడు mocks చాలా మెరుగు. Mock 1 లో నువ్వు తడబడతావు – అదే దాని ఉద్దేశం. Interview లో తడబడటం కంటే mock లో తడబడటం మేలు.

35. Rapid-fire cheat sheet (చివరి 2 గంటల revision)

ఇది print చేసి లేదా interview రోజు తెరిచి ఉంచు.

JavaScript – one-liners

Question	Answer
Closure	Function + its lexical environment, alive after the outer function returns
var vs let	Function-scoped + <code>undefined</code> hoisting vs block-scoped + TDZ
this	Decided by how it's called; arrows take it lexically
Event loop	Sync → microtasks (promises) → macrotasks (setTimeout)
== vs ===	Coercion vs strict; always <code>===</code>
Shallow vs deep	Spread/ <code>Object.assign</code> = shallow; <code>structuredClone</code> = deep
Debounce	Waits for silence – search input
Throttle	Max rate – scroll/resize
call/apply/bind	Comma / Array / Bound-for-later
Promise.all	Fail-fast; <code>allSettled</code> for partial failure
Hoisting	Declarations move up; <code>let / const</code> → TDZ
Falsy (8)	<code>false 0 -0 0n "" null undefined NaN</code>
Event delegation	One listener on the parent, uses bubbling – React does this at the root

React – one-liners

Question	Answer
Virtual DOM	In-memory tree; diff + batch, not "faster than DOM"
Reconciliation	O(n) heuristic: different type = rebuild subtree; keys identify children
Keys	Stable identity; index only for static lists
Props vs state	Read-only from parent vs owned and mutable
useEffect cleanup	Runs on unmount and before every re-run
useMemo	Caches a value · <code>useCallback</code> caches a function · <code>memo</code> skips re-render
Context caveat	Re-renders all consumers; not a state management library

Question	Answer
Redux 3 principles	Single store · read-only state · pure reducers
RTK	<code>createSlice</code> + Immer + thunk + devtools, ~1/3 the code
<code>useSelector</code> trap	Returning a new object/array → re-render every action → <code>createSelector</code>
Performance	Measure → memo → split → virtualise → debounce → bundle
Code splitting	<code>React.lazy</code> + <code>Suspense</code> ; routes first
Error boundary	Class-only; doesn't catch events/async/SSR
<code>&&</code> gotcha	<code>{items.length && ...}</code> renders a literal <code>0</code>
<code>componentWillUnmount</code>	The function you return from <code>useEffect</code>

HTTP / API

Question	Answer
CORS	Browser enforces it, not the server; fix belongs server-side
401 vs 403	Unauthenticated (who are you?) vs unauthorised (you can't)
Idempotent	GET, PUT, DELETE ✓ · POST, PATCH ✗
JWT problem	Can't be revoked → short expiry + refresh token
<code>fetch</code> gotcha	Doesn't throw on 404/500 – check <code>res.ok</code>
XSS vs CSRF	Trust in the site vs trust in the browser

Git

Question	Answer
merge vs rebase	History preserved vs linear; never rebase shared branches
reset vs revert	Rewrites history (local) vs new undo commit (public)
fetch vs pull	Download only vs download + merge

Behavioural – నీ 5 stories (ఇక్కడ నీ own keywords రాసుకో)

Story	నీ one-line reminder
1. Technical challenge	_____
2. Deadline / incident	_____
3. Conflict	_____

Story	న one-line reminder
4. Failure	_____
5. Initiative	_____

న numbers (ఇక్కడ రాసుకో)

–	Value
Users / scale	_____
Team size	_____
Data volume	_____
Before → after metric 1	_____
Before → after metric 2	_____
Current CTC (fixed + variable)	_____
Target CTC	_____
Ask (anchor)	_____
Walk-away	_____

5 phrases interview లో వాడు

1. "Let me ask a couple of clarifying questions first..." – scenario questions కి
2. "The thing to be careful about is..." – trade-off చూపించడానికి
3. "In my project, we..." – ప్రతి technical answer లో
4. "In production I'd use X, but let me write it directly here" – live coding లో
5. "I haven't worked with that directly, but based on my understanding of X..." – తెలియనిదానికి

5 things ఎప్పుడూ చెయ్యకు

1. ✗ Current company/manager గురించి negative
2. ✗ "We did it" – ownership లేకుండా
3. ✗ Bluff – తెలియనిది తెలిసినట్టు
4. ✗ "I never had a conflict / never made a mistake"
5. ✗ చివర్లో "No, I don't have any questions"

ముగింపు

ఈ doc ని ఎలా వాడాలి – 3 lines లో

1. **§9 (project) + §25 (STAR stories) మొదట చెయ్యి.** ఇవి interview లో 40% marks, మరియు ఇవే ఎవరూ prepare చెయ్యనివి.
2. **§13 (React) + §10-§12 (JavaScript) రెండోది.** ఇవి 90%+ probability.
3. **§34 (mocks) skip చెయ్యకు.** చదవడం knowledge ఇస్తుంది; **మాట్లాడటం మాత్రమే offer ఇస్తుంది.**

గుర్తుపెట్టుకోవాల్సిన ఒక్క విషయం

Infosys ఒక **service company**. వాళ్ళు వెతుకుతున్నది React Fiber internals తెలిసిన genius కాదు – **fundamentals clear గా ఉన్న, clear గా మాట్లాడే, ownership తీసుకునే, flexible గా ఉండే engineer.**

నీ technical prep ఎంత బాగున్నా, **"we did it" అని చెప్తే** నువ్వు reject అవుతావు. నీ technical prep సగం ఉన్నా, **"I owned this, here's what I did, here's the result" అని చెప్తే** నువ్వు select అవుతావు.

Depth కాదు – clarity, ownership, communication. ఇదే Infosys interview యొక్క రహస్యం.

All the best. 🍀